VIỆN KHOA HỌC VÀ CÔNG NGHỆ VIỆT NAM
VIỆN CÔNG NGHỆ THÔNG TIN

PHÂN TÍCH THIẾT KẾ HƯỚNG ĐỐI TƯỢNG

PGS.TS. Đặng Văn Đức
Email: dvduc@ioit.ac.vn

Xuất bản năm 2002

Chuyển sang ebook bởi

Sinh viên lớp DHTH4LT – Trường ĐH Công Nghiệp

10/2009

Mục lục

CHƯƠNG 1 MỞ ĐẦU 7

1.1 LỊCH SỬ HƯỚNG ĐỐI TƯỢNG 8
1.2 MỘT SỐ KHÁI NIỆM CƠ BẢN 10
1.3 NGUYÊN TẮC QUẢN LÝ ĐỘ PHỨC TẠP 12
1.4 NGUYÊN TẮC MÔ HÌNH HÓA 13
1.5 KHÁI QUÁT VỀ TIẾN TRÌNH PHÁT TRIỂN PHẦN MỀM 15
1.5.1 - Các phương pháp mô hình hóa hệ thống *15*
1.5.2 - Các pha phát triển phần mềm *17*

CHƯƠNG 2 KHÁI QUÁT VỀ UML 24

2.1 GIỚI THIỆU UML 24
2.2 MÔ HÌNH KHÁI NIỆM CỦA UML 26
2.2.1 - Phần tử mô hình trong UML *26*
2.2.2 - Các quan hệ trong UML *28*
2.2.3 - Kiểu dữ liệu *29*
2.2.4 - Biểu đồ UML *29* 2.3
KIẾN TRÚC HỆ THỐNG 38
2.3.1 - Khung nhìn UC *38*
2.3.2 - Khung nhìn thiết kế *39*
2.3.3 - Khung nhìn cài đặt *39*
2.3.4 - Khung nhìn triển khai *39*
2.3.5 - Khung nhìn tiến trình *40*
2.3.6 - Cần bao nhiêu khung nhìn *40* 2.4
RATIONAL ROSE LÀ GÌ? 40 2.5
KHẢ NĂNG SỬ DỤNG UML 41 2.6
THỰC HÀNH 41

CHƯƠNG 3 MÔ HÌNH HÓA TRƯỜNG HỢP SỬ DỤNG 44

3.1 PHÂN TÍCH TRƯỜNG HỢP SỬ DỤNG (USE CASE – UC) 44
3.1.1 - UC là gì? *44*
3.1.2 - Xây dựng UC để làm gì? *44*
3.1.3 - Tìm kiếm UC như thế nào ? *45*
3.1.4 - Luồng sự kiện trong UC *48*
3.2 BIỂU ĐỒ TRƯỜNG HỢP SỬ DỤNG 50
3.3 THỰC HÀNH 54
3.3.1 - Sử dụng Rational Rose *54*
3.3.2 - Thí dụ: hệ thống bán hàng *60*

CHƯƠNG 4 MÔ HÌNH HÓA TƯƠNG TÁC ĐỐI TƯỢNG 63

4.1 ĐỐI TƯỢNG VÀ TÌM KIẾM ĐỐI TƯỢNG 63
4.2 BIỂU ĐỒ TƯƠNG TÁC 63
4.2.1 - Biểu đồ trình tự *64*
4.2.2 - Biểu đồ cộng tác *70* 4.3
KỸ THUẬT XÂY DỰNG BIỂU ĐỒ TƯƠNG TÁC 72 4.4
THỰC HÀNH 75
4.4.1 - Sử dụng Rational Rose *75*
4.4.2 - Thí dụ: hệ thống bán hàng (tiếp theo) *83*

CHƯƠNG 5 BIỂU ĐỒ LỚP VÀ GÓI 92

5.1 LỚP VÀ TIỀM KIẾM LỚP 92
5.2 BIỂU ĐỒ LỚP 94
5.2.1 - Các loại lớp trong biểu đồ *94*
5.2.2 - Stereotype của lớp *95* 5.3
GÓI 96 5.4
THUỘC TÍNH LỚP 97
5.4.1 - Tìm kiếm thuộc tính *97*
5.4.2 - Đặc tả thuộc tính *98* 5.5
THAO TÁC CỦA LỚP 100

5.6 QUAN HỆ....101
5.6.1 - Quan hệ kết hợp....101
5.6.2 - Quan hệ phụ thuộc....104
5.6.3 - Phụ thuộc tụ hợp....105
5.6.4 - Quan hệ khái quát hóa....106
5.6.5 - Gán đặc tính cho quan hệ....107
5.7 CƠ CHẾ DUY TRÌ ĐỐI TƯỢNG....112
5.8 THỰC HÀNH....115
5.8.1 - Sử dụng Rational Rose....115
5.8.2 - Thí dụ: Hệ thống bán hàng (tiếp theo) 125

CHƯƠNG 6 BIỂU ĐỒ CHUYỂN TRẠNG THÁI VÀ BIỂU ĐỒ HOẠT ĐỘNG 137

6.1 BIỂU ĐỒ CHUYỂN TRẠNG THÁI 137
6.1.1 - Trạng thái 139
6.1.2 - Quá độ 141
6.1.3 - Trạng thái ẩn 142
6.1.4 - Lớp và biểu đồ trạng thái 144
6.2 BIỂU ĐỒ HOẠT ĐỘNG 144
6.2.1 - Trạng thái hành động và trạng thái hoạt động....145
6.2.2 - Quá độ 145
6.2.3 - Rẽ nhánh....145
6.2.4 - Đường dẫn tương tranh 146
6.2.5 - Đường bơi....147
6.2.6 - Luồng đối tượng 147
6.2.7 - Gửi và nhận tín hiệu 148
6.3 THỰC HÀNH....149
6.3.1 - Sử dụng Rational Rose....149
6.3.2 - Thí dụ: Hệ thống bán hàng (tiếp theo) 152

CHƯƠNG 7 BIỂU ĐỒ KIẾN TRÚC VẬT LÝ VÀ PHÁT SINH MÃ TRÌNH....155

7.1 BIỂU ĐỒ THÀNH PHẦN 155
7.1.1 - Thành phần là gì?....155
7.1.2 - Biểu tượng thành phần trong Rational Rose 156
7.1.3 - Phụ thuộc thành phần....157
7.1.4 - Biểu đồ thành phần....158
7.2 BIỂU ĐỒ TRIỂN KHAI 159
7.2.1 - Phần tử mô hình của biểu đồ....159
7.2.2 - Tiến trình 160
7.3 THỰC HÀNH....160
7.3.1 - Sử dụng Rational Rose....160
7.3.2 - Phát sinh mã trình bằng Rose....164
7.3.3 - Rational Rose và Visual C++....170
7.3.4 - Thí dụ: Hệ thống bán hàng (tiếp theo) 171

CHƯƠNG 8 VÍ DỤ ÁP DỤNG....184

8.1 KHẢO SÁT TIẾN TRÌNH TÁC NGHIỆP....184
8.2 PHÂN TÍCH LĨNH VỰC....190
8.3 PHÂN TÍCH HỆ THỐNG....194
8.3.1 - Xây dựng biểu đồ trường hợp sử dụng (Use Case-UC)....194 8.4
BIỂU ĐỒ TƯƠNG TÁC....204
8.4.1 - Tiến trình đặt trước sách để mượn 204
8.4.2 - Tiến trình mượn sách , tạp chí....206 8.5
BIỂU ĐỒ LỚP. 208 8.6
BIỂU ĐỒ TRIỂN KHAI 209 8.7
THIẾT KẾ GIAO DIỆN....211

CHƯƠNG 9 MÃ TRÌNH PHÁT SINH TRONG ROSE....214

9.1 PHÁT SINH MÃ TRÌNH C++....214
9.1.1 - Các lớp 214
9.1.2 - Quan hệ kết hợp....216

9.1.3 - Quan hệ phụ thuộc tập hợp.....220

9.1.4 - Quan hệ kế thừa.....222 9.2
PHÁT SINH MÃ TRÌNH JAVA.....222
9.2.1 - Các lớp 223
9.2.2 - Quan hệ kết hợp.....224
9.2.3 - Quan hệ phụ thuộc tập hợp.....225
9.2.4 - Quan hệ kế thừa.....226 9.3
PHÁT SINH MÃ TRÌNH VISUAL BASIC.....227
9.3.1 - Các lớp 227
9.3.2 - Quan hệ kết hợp.....229
9.3.3 - Quan hệ kế thừa đơn.....230 9.4
PHÁT SINH MÃ TRÌNH SQL.....231
9.4.1 - Các lớp 231
9.4.2 - Quan hệ kết hợp.....231
9.4.3 - Quan hệ kế thừa.....233

LỜI NÓI ĐẦU

Hệ thống tin học ngày càng phức tạp. Xu thế áp dụng phương pháp hướng đối tượng (phương pháp mới) thay cho phương pháp cấu trúc (phương pháp truyền thống) ngày càng phổ biến khi xây dựng các hệ thống phần mềm lớn và phức tạp. Hơn nữa, từ khi Ngôn ngữ mô hình hóa thống nhất (Unified Modeling Language – UML) được tổ chức OMG (*Object Management Group*) công nhận là chuẩn công nghiệp thì nó đã trở thành công cụ phổ dụng và hữu hiệu cho phương pháp mới này. Mục tiêu của tài liệu này nhằm giới thiệu các khái niệm cơ bản về tiếp cận hướng đối tượng và mô hình hóa hệ thống phần mềm theo phương pháp hướng đối tượng. Các khái niệm mới được mô tả, hướng dẫn thực hành thông qua ngôn ngữ chuẩn UML và phần mềm công cụ mô hình hóa nổi tiếng *Rational Rose* của *Raitonal Software Corporation*.

Phương pháp phân tích thiết kế hướng đối tượng được sử dụng rộng rãi tại các nước phát triển và bắt đầu được sử dụng tại một số đơn vị tin học tại Việt Nam. Tuy nhiên tài liệu bằng tiếng Việt về lĩnh vực này còn rất hiếm hoi, không đáp ứng nhu cầu hiện tại. Hơn nữa, nhận thức được tầm quan trọng của phương pháp mới này, một số trường đại học đã hình thành môn học liên quan đến vấn đề nói trên cho sinh viên, còn một số trường khác đang có kế hoạch đưa chủ đề này vào chương trình đào tạo chính khóa.

Chủ điểm của tài liệu được thể hiện dưới góc nhìn của người phát triển hệ thống phần mềm, không thể hiện dưới góc độ quan sát của nhà phương pháp luận. Lựa chọn này xuất phát từ thực tế là từ phương pháp luận hướng đối tượng dẫn đến việc ứng dụng nó vào xây dựng phần mềm cụ thể còn một khoảng cách xa vời và đầy khó khăn, đặc biệt với trình độ tin học hiện này nói chung còn chưa cao tại Việt Nam. Với quan điểm này, tài liệu được cấu trúc như sau:

Chương mở đầu trình bày khái quát về mô hình và mô hình hóa; các bước xây dựng hệ thống phần mềm và tầm quan trọng của phương pháp hướng đối tượng. Chương tiếp theo giới thiệu ngôn ngữ chuẩn công nghiệp UML, một công cụ hữu hiệu mô hình hóa hệ thống phần mềm. Trong các phần tiếp theo là trình bày kỹ thuật mô hình hóa, từ phân tích yêu cầu đến thiết kế hệ thống, kiến trúc hệ thống và cài đặt bằng ngôn ngữ lập trình. Chương cuối cùng là bài học thực nghiệm các kỹ thuật đã trình bày trong các chương trước vào bài toán cụ thể. Đặc biệt, trong mỗi chương tài liệu đều có phần thực hành trên phần mềm *Rational Rose* để độc giả có thể áp dụng ngày công cụ mới, kỹ thuật mới vào giải quyết vấn đề của riêng họ. Phần phụ lục trình bày một số mã trình trong một vài ngôn ngữ thông dụng tương ứng với các nhóm phần tử trong biểu đồ UML…

Hiện nay phần lớn các bạn sinh viên đại học năm cuối hoặc các kỹ sư tin học mới ra trường đều gặp khó khăn khi nhận nhiệm vụ xây dựng hệ thống phần mềm mới hay nâng cấp phần mềm có sẵn. Các bạn thường không biết bắt đầu từ đâu và làm như thế nào để có được phần mềm và phần mềm tốt, nói cách khác là còn thiếu phương pháp. Do vậy, quyển sách này có thể là tài liệu tham khảo tốt cho các bạn sinh viên và các kỹ sư tin học.

Quyển sách này được hình thành từ nội dung bài giảng của tác giả về chủ đề *Phát triển phần mềm hướng đối tượng băng UML* cho một số lớp sinh viên đại học. Trong quá trình biên soạn tác giả đã nhận được nhiều ý kiến đóng góp quí báu của các chuyên gia trong lĩnh vực này. Trước hết tác giả xin chân thành cảm ơn PGS. TSKH Nguyễn Xuân Huy, CN Ngô Trung Việt, TS Đặng Thành Phu, TS Đoàn Văn Ban, ThS Nguyễn Sơn Hải và các đồng nghiệp khác công tác tại Viện Công nghệ Thông tin, Trung tâm Khoa học Tự nhiên và Công nghệ Quốc gia đã đọc và cho ý kiến sửa chữa bản thảo.

Mặc dù đã rất cố gắng nhưng tài liệu này không tránh khỏi các sai sót. Tác giả xin chân thành cám ơn mọi ý kiến đóng góp của bạn đọc.

Địa chỉ liên lạc: Viện Công nghệ Thông tin, Trung tâm Khoa học Tự nhiên và Công nghệ Quốc gia. Email: dvduc@ioit.ncst.ac.vn

Hà nội, tháng 02 năm 2002

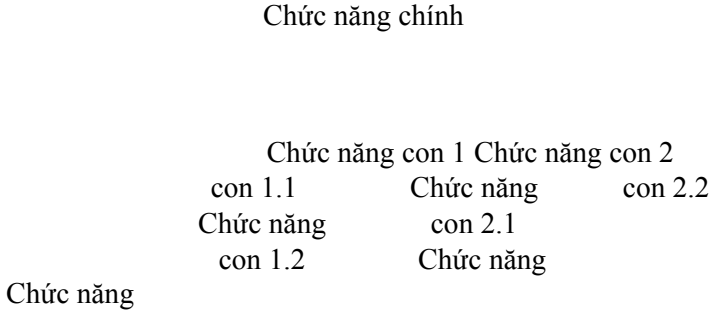
TÁC GIẢ

CHƯƠNG 1

MỞ ĐẦU

Phát triển phần mềm ngày càng trở nên phức tạp. Thay đổi giao diện từ các xâu ký tự sang giao diện đồ họa xu thế sự kiện; kiến trúc hệ thống đa tầng khách/chủ; cơ sở dữ liệu (CSDL) phân tán; Internet … làm tăng độ phức tạp của hệ thống phần mềm. Thách thức trong hai mươi năm tới của xây dựng hệ thống phần mềm không phải là tốc độ thực hiện chương trình, kinh phí hay sức mạnh của nó mà vấn đề sẽ là độ phức tạp (*Sun Microsystem*). Kẻ thù của chúng ta là độ phức tạp, ta phải loại bỏ chúng (*Jan Bean*). Vậy, loại bỏ độ phức tạp bằng cách nào? Các phương pháp tiếp cận hướng cấu trúc, tiệm cận hướng logic, tiếp cận hướng hướng đối tượng và tiếp cận hướng tác tử để có thể giải quyết vấn đề này nhưng ở mức độ khác nhau.

Tổng quát thì việc xây dựng phần mềm phải quan tâm đến tổ chức, các quan hệ và cấu trúc để hình thành được các hành vi phức tạp của hệ thống. Mọi việc khảo sát hệ thống phải được thực hiện với mức độ trừu tượng khác nhau, từ các chi tiết đến tổ chức tổng thể. Do vậy, xây dựng phần mềm là thực hiện dãy tương tác *chia nhỏ và hợp nhất*. Chia nhỏ để hiểu rõ vấn đề và hợp nhất để xây dựng hệ thống. Tiến trình chia nhỏ (tách) đã có truyền thống và tuân thủ các tiêu chí *chức năng*. Các chức năng của hệ thống được nhận diện, sau đó chúng được tách thành các chức năng con. Tiến trình này được thực hiện lặp đi lặp lại cho đến khi có được các thành phần đơn giản đến mức chúng được biểu diễn trực tiếp bằng các hàm hay thủ tục của ngôn ngữ lập trình (hình 1.1). Cách tiếp cận này được gọi là tiếp cận hướng chức năng (hay còn gọi là thủ tục, truyền thống). Người phát triển phần mềm sẽ tập trung vào các nhiệm vụ điều khiển và tách thuật toán lớn thành các thuật toán nhỏ. Khối chính để hình thành phần mềm ở đây là các hàm hay thủ tục.

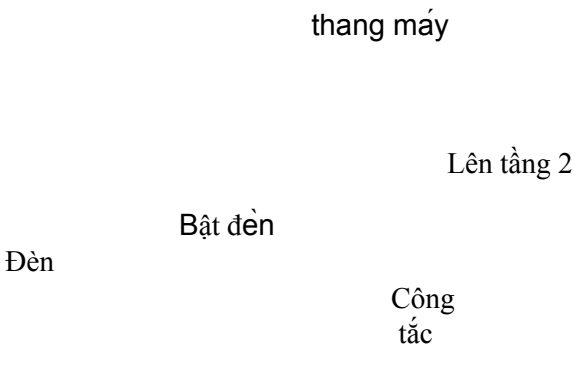


Hình 1.1 Tiếp cận hướng chức năng

Kiến trúc phần mềm được cài đặt theo cách tiếp cận vừa mô tả trên sẽ phản ảnh các chức năng hệ thống. Tiếp cận trên cơ sở chức năng và cơ chế phân cấp chỉ cho lại kết quả mong muốn khi các chức năng được nhận biết đầy đủ và nó không được thay đổi theo thời gian. Thực tế lại không đúng như vậy vì trong rất nhiều trường hợp, phát triển phần mềm không bao giờ kết thúc hoàn toàn, luôn có cái gì đó phải sửa đổi, nâng cấp. Sửa đổi hay mở rộng hệ thống quá nhiều làm cho chương trình khác xa quan niệm ban đầu. Do đó cần phải có phương pháp mới cho khả năng làm chủ được độ phức tạp, giúp quản lý được chất lượng, độ tin cậy phần mềm ngày cả khi cấu trúc bị tách ra hay tiến hóa.

Mở

Cửa Phòng



Hình 1.2 Tiếp cận hướng đối tượng

Quan điểm hướng đối tượng hình thành trên cơ sở tiếp cận hướng hệ thống, nó coi hệ thống như thực thể được tổ chức từ các thành phần mà chỉ được xác định khi nó thừa nhận và có quan hệ với các thành phần khác. Phương pháp tách vấn đề đang giải quyết để hiểu chúng ở đây không chỉ dựa trên cơ sở *cái hệ thống làm* mà còn dựa trên việc tích hợp *hệ thống là cái gì* với *hệ thống làm gì*. Thí dụ trên hình 1.2 mô tả các đối tượng và các quan hệ giữa các đối tượng của hệ thống thang máy. Theo cách tiếp cận này thì các chức năng hệ thống được biểu diễn thông qua cộng tác của các đối tượng; việc thay đổi, tiến hóa chức năng sẽ không ảnh hưởng đến cấu trúc tĩnh của phần mềm. Sức mạnh của tiếp cận hướng đối tượng là việc tách (chia) và nhập (thống nhất) được thực hiện nhờ tập phong phú các cơ chế tích hợp của chúng; khả năng thống nhất cao những cái nó đã được tách ra để xây dựng các thực thể phức tạp từ các thực thể đơn giản.

Tiếp cận hướng đối tượng đã tỏ rõ lợi thế khi lập trình các hệ thống phức tạp. Những người phát triển phần mềm nhận thấy rằng phát triển phần mềm hướng đối tượng sẽ cho lại phần mềm thương mại chất lượng cao: tin cậy, dễ mở rộng và dễ sử dụng lại, chạy trơn tru, phù hợp với yêu cầu người dùng đang mong đợi. Chúng còn cho khả năng hoàn thành phần mềm đúng kỳ hạn và không vượt quá khinh phí dự kiến ban đầu.

Ngoài phương pháp cấu trúc và phương pháp hướng đối tượng vừa đề cập trên đây, người ta còn thấy một số phương pháp khác được các nhà tin học đề xuất cho công nghệ phần mềm như tiếp cận hướng logic, tiếp cận hướng tác tử (agent)… Phương pháp tiếp cận hướng logic mô tả quan hệ "logic" giữa tính chất và thuộc tính của các sự kiện nhờ các cơ sở lập luận và luật suy diễn biểu diễn bởi những gợi ý trước. Tiếp cận này hình thành trên cơ sở ngôn ngữ lập trình *Prolog*. Phương pháp tiếp cận tác tử là kiểu mở rộng của tiếp cận hướng đối tượng. Gần đây nó được giới công nghiệp rất quan tâm. Các đơn vị cơ sở để phát triển hệ thống ở đây là các tác tử, nó là modun phần mềm. Đối tượng được khởi động khi nhận thông điệp từ bên ngoài và tác tử tích cực làm việc hay không phụ thuộc vào môi trường chứa nó. Tuy nhiên, phương pháp hướng đối tượng đang ngày cành được nhiều người sử dụng hơn cả.

1.1 LỊCH SỬ HƯỚNG ĐỐI TƯỢNG

Khái niệm hướng đối tượng hình thành từ ngôn ngữ lập trình *Simula*, nhưng nó chỉ trở nên quen thuộc khi xuất hiện ngôn ngữ *C++* và *SmallTalk* vào cuối những năm 80 của thế kỷ XX. Sơ đồ trong hình 1.3 chỉ ra thời gian xuất hiện và quan hện của các ngôn ngữ lập trình [OES00]. Trong hình vuông với góc tròn là tên các ngôn ngữ lập trình hướng đối tượng.

PL/1

Algol

1960

Fortran LISP

Cobol

~~Smalltalk-72~~ Smalltalk-74

1970

Smalltalk-76 Smalltalk-78

C

Pascal

Simula

Prolog

Loops

Objective C

1980

C++

ObjectPascal

Ada

Smalltalk-80

CLOS

Eiffel

1990

Ada 9 ObjectCobol

Java

Hướng đối tượng Không hướng đối tượng

Hình 1.3 Các ngôn ngữ lập trình

Khi các ngôn ngữ hướng đối tượng được sử dụng rộng rãi thì nhu cầu có phương pháp phát triển phần mềm hướng đối tượng trở nên cấp bách. Vào đầu những năm 90 của thế kỷ XX đã xuất hiện các phương pháp hướng đối tượng sau đây: Phương pháp *Booch*, *OMT* (*Object Modeling Technique*), *OOSE* (*Object Oriented Software Engineering*)/*Objectory*, *Fusion* và *Coad/Yourdon*. Mỗi phương pháp có ký pháp, tiến trình và công cụ hỗ trợ riêng. Chúng đều có ưu điểm và nhược điểm riêng. Người sử dụng rất khó khăn để chọn cho mình một phương pháp phù hợp. Do nhận biết đươc các vấn đề này, vào năm 1994 các tác giả của các phương pháp này đã hợp tác nhằm tạo ra phương pháp mới. Bắt đầu là sự thống nhất phương pháp *Booch* với OMT-2 của *Rumbagh* để

hình thành *Unified Method 0.8* tại *Rational Rose Corporation*. Tháng 6 năm 1995, *Ivar Jacobson* (tác giả của *OOSE/Objectory*) gia nhập với họ. Từ thời điểm này, nhóm phát triển phương pháp hướng đối tượng nói trên cho rằng nhiệm vụ của họ là tạo ra ngôn ngữ mô hình hóa thống nhất cho cộng đồng hướng đối tượng. Do vậy, họ đã đổi tên công việc của mình thành *Unified Modeling Language – UML* (Ngôn ngữ mô hình hóa thống nhất). *Booch*, *Rumbaugh* và *Jacobson* đã đưa ra nhiều phiên bản UML, trong đó phiên bản UML 0.9 xuất hiện năm 1995, UML xuất hiện vào năm 1997. Phần lớn UML được xây dựng trên nền tảng của các phương pháp *Booch*, OMT và OOSE, nhưng UML còn bao gồm cả các khái niệm có nguồn gốc từ các phương pháp khác như *David Harel*, *Gamma – Helm – Johnson – Vlissides* và *Fusion*. UML còn là kết quả của sự đóng góp từ các hãng lớn như Digital Equipment Corporation (DEC), Hewlett –

Packard (HP), I-Logix, Intellicorp, IBM, ICON Computing, MCI Systemhouse, Microsoft, Oracle, Rational Software Corporation, Texas Instrument, Taskon, ObjectTime và Unisys. Phiên bản UML 1.1 đã được đệ trình lên OMG (Object Management tháng 11-1997. Phiên bản UML 1.3 xuất hiện vào năm 1999. Phiên bản UML 1.4 xuất hiện vào tháng 2-2000.

1.2 MỘT SỐ KHÁI NIỆM CƠ BẢN

Phần này trình bày một số khái niệm cơ bản áp dụng trong phân tích và thiết kế hướng đối tượng.

Phương pháp (*method*). Phương pháp (hay phương thức) là cách thức cấu trúc các suy nghĩ và hành động của con người. Nó cho biết chúng ta phải làm cái gì, làm như thế nào, làm khi nào và tại sao phải làm như vậy để hình thành hệ thông phần mềm.

Đối thương (*object*). Theo nghĩa thông thường thì đối tượng là người, vật hay hiện tượng mà con người nhằm vào trong suy nghĩ, trong hành động [PHE96]; là bất kỳ cái gì nhìn thầy và sờ mó được. Trong phương pháp hướng đối tượng thì đối tượng là trừu tượng cái gì đó trong lĩnh vực vấn đề hay trong cài đặt của nó; phản ảnh khả năng hệ thống lưu giữ thông tin về nó và tương tác với nó; gói các giá trị thuộc tính và các dịch vụ (phương thức, phương pháp) [OCAD91].

Lớp (*class*). Theo nghĩa thông thường thì lớp là nhóm của nhiều người hay vật có tính tương tự nhất định hay đặc điểm chung (từ điển Webster's). Trong phương pháp hướng đối tượng thì lớp là mô tả một hay nhiều đối tượng, mô tả tập thống nhất các thuộc tính và phương thức. Nó còn có thể mô tả cách tạo đối tượng mới trong lớp như thế nào [COAD91].

Trừu tượng (*abstract*). Trừu tượng là nguyên lý bỏ qua những khía cạnh của chủ thể (*subject*) không liên quan đến mục đích hiện tại để tập trung đầy đủ hơn vào các khía cạnh còn lại. Như vậy có thể nói rằng *trừu tượng là đơn giản hóa thế giới thực một cách thông minh*. Trừu tượng cho khả năng tổng quát hóa và ý tưởng hóa vấn đề đang xem xét. Chúng loại bỏ đi các chi tiết dư thừa mà chỉ tập chung và các điểm chính, cơ bản [LIBE98].

Mô hình (*model*). Nếu phải xây ngôi nhà thì chắc chắn không làm đơn giản là cứ mua gạch, sắt thép về lắp dần đến khi hình thành nhà ở, mà phải có kế hoạch chi tiết và thiết kế trước. Nói cách khác là phải xây dựng mô hình. Tương tự như vậy, trong lĩnh vực phần mềm, mô hình là kế hoạch chi tiết của hệ thống, nó giúp ta lập kế hoạch trước khi xây dựng hệ thống. Mô hình giúp ta khẳng định tính đúng đắn của thiết kế, phù hợp yêu cầu, hệ thống vẫn giữ vững khi yêu cầu người dùng thay đổi. Phương pháp hướng đối tượng xem một phần của thế giới thực như các đối tượng. Máy điện thoại, xe ô tô, con người … là các đối tượng. Các đối tượng này lại bao gồm nhiều đối tượng khác như xe ô tô có tay lái, bánh xe, … con người có tay, chân, mắt, mũi … Đối tượng thế giới thực có thể có cấu trúc rất phức tạp, làm cho con người khó nhận thức được chúng. Trong cuộc sống hàng ngày ta đơn giản hóa đối tượng khi suy nghĩ, hay nói cách khác ta làm việc với

mô hình. Thí dụ, quả địa cầu là mô hình của trái đất. Mô hình chỉ lựa chọn một vài khía cạnh có ý nghĩa cho việc thực hiện công việc cụ thể nào đó. Chúng cho phép dự đoán, hiểu các khía cạnh của vấn đề đang quan tâm. Thí dụ khi làm việc với đối tượng con người: trong sổ lương có tên, số bảo hiểm xã hội và tiền lương. Nhưng trong sổ điện thoại lại chỉ có tên, số điện thoại và địa chỉ. Vậy, mô hình là bức tranh hay mô tả của vấn đề đang được cố gắng giải quyết hay biểu diễn. Mô hình còn có thể là mô tả chính giải pháp. Trong phát triển phần mềm, thay cho đối tượng thực, ta sẽ làm việc với biểu tượng (hình 1.4). Tiến trình phát triển phần mềm là làm giảm một số đặc trưng của đối tượng để hình thành mô hình: làm giảm độ phức tạp bằng mô hình trừu tượng.



Hình 1.4 Mô hình thế giới thực

Phương pháp luận (*methodology*). Phương pháp luận mô tả cách thức suy nghĩ về phần mềm và phát triển phần mềm. Nó bao gồm ngôn ngữ mô hình hóa *metamodel* (mô hình của mô hình) và tiến trình. Phương pháp luận khác với phương pháp. Phương pháp luận là nghiên cứu phương pháp. *Metamodel* mô tả hình thức các phần tử mô hình, cú pháp và ngữ nghĩa của các ký pháp trong mô hình.

Lĩnh vực vấn đề (*domain problem*). Mục tiêu của tiếp cận hướng đối tượng là mô hình hóa các đặc tính tĩnh và động của môi trường, nơi xác định yêu cầu phần mềm. Môi trường này được gọi là lĩnh vực vấn đề. Vấn đề là câu hỏi đặt ra để giải quyết hoặc xem xét. Lĩnh vực là không gian (vùng) của các hoạt động hoặc ảnh hưởng. Nó là vùng tác nghiệp hay kinh nghiệm của con người trong đó phần mềm được sử dụng. Vậy, lĩnh vực vấn đề là vùng mà ta đang cố gắng xem xét. Thí dụ của lĩnh vực vấn đề có thể là tài chính, giáo dục,…

Phân tích. Phân tích là tách, chia nhỏ tổng thể thành các phần để tìm ra đặc tính, chức năng, quan hệ… của chúng (từ điển Webster's). Khái niệm phân tích trong tiếp cận hướng đối tượng là thực hiện nghiên cứu lĩnh vực vấn đề, dẫn tới đặc tả hành vi quan sát từ ngoài và các thông báo nhất quán, hoàn chỉnh, khả thi của những cái cần [COAD91]. Phân tích hướng đối tượng tập trung vào tìm kiếm, mô tả các đối tượng (khái niệm) trong lĩnh vực vấn đề. Thí dụ hệ thống thư viện có khái niệm Sách, Thư viện …

Thiết kế. Là tập tài liệu kỹ thuật toàn bộ, gồm có bản tính toán, bản vẽ… để có thể theo đó mà xây dựng công trình, sản xuất thiết bị, làm sản phẩm… [PHE96]. Khái niệm phân tích trong tiếp cận hướng đối tượng là thực hiện đặc tả các hành vi bên ngoài, bổ sung chi tiết nếu cần thiết

để cài đặt hệ thống trên máy tính, bao gồm tương tác người - máy, quản lý nhiệm vụ, quản lý dữ liệu [COAD91]. Thiết kế hướng đối tượng tập trung vào xác định đối tượng phần mềm logic sẽ được cài đặt bằng ngôn ngữ hướng đối tượng.

Xây dựng (lập trình) hướng đối tượng: là thiết kết các modun sẽ được cài đặt. Thí dụ lớp Book sẽ được cài đặt bằng C++, Java … như thế nào.

Mô hình hóa (*modeling*). Khái niệm *mô hình hóa* thường được sử dụng đồng nghĩa với phân tích, đó là việc thực hiện tách hệ thống thành các phần tử đơn giản để dễ hiểu. Trong khoa học máy tính, mô hình hóa bắt đầu từ mô tả vấn đề, sau đó là mô tả giải pháp vấn đề. Các hoạt động này còn được gọi là *phân tích và thiết kế*. Khi thu thập yêu cầu cho hệ thông, ta phải tìm ra nhu cầu tác nghiệp của người dùng và ánh xạ chúng thành yêu cầu phần mềm sao cho đội ngũ phát triển phần mềm hiểu và sử dụng được chúng. Tiếp theo là khả năng phát sinh mã trình từ các yêu cầu này, đồng thời đảm bảo rằng các yêu cầu phải phù hợp với mã trình vừa phát sinh và dễ dàng chuyển đổi mã trình ngược lại thành yêu cầu. Tiến trình này được gọi là mô hình hóa.

Mô hình hóa trực quan. Mô hình hóa trực quan là tiến trình lấy thông tin từ mô hình và hiển thị đồ họa bằng tập các phần từ đồ họa chuẩn. Tiêu chuẩn là cốt lõi để thực hiện một trong các lợi thế của mộ hình trực quan, đó là vấn đề giao tiếp. Giao tiếp giữa người dùng, người phát triển, phân tích viên, kiểm tra viên, người quản lý và những người khác tham gia dự án là mục tiêu quan trọng nhất của mô hình hóa trực quan. Tương tác này có thể được thực hiện bằng văn bản. Nhờ mô hình trực quan mà ta có thể chỉ ra các *tầng* mà hệ thống làm việc, bao gồm tương tác giữa người dùng với hệ thống, tương tác giữa các đối tượng trong hệ thống hay giữa các hệ thông với nhau. Sau khi tạo mô hình, ta có thể chỉ ra từng phần quan tâm. Thí dụ, người dùng có thể quan sát tương tác giữa họ với hệ thông từ mô hình, phân tích viên quan sát tương tác các đối tượng từ mô hình, người phát triển sẽ quan sát được đối tượng mà họ sẽ phát triển… Các nhà tin học đã rất cố gắng để hình thành ký pháp mô hình hóa trực quan. Một số ký pháp quen thuộc là của Booch, OMT và UML. Phần mềm công cụ *Rational Rose 2000* trợ giúp các ký pháp này. Tóm lại, lý do cơ bản để mô hình hóa là: xây dựng mô hình để hiểu sâu sắc hơn về hệ thống đang được xây dựng. Chúng ta xây dưng mô hình cho các hệ thống phức tạp vi ta không thể hiểu nó như tổng thể. Nhờ mô hình hóa ta sẽ đạt được các mục tiêu sau:

Mô hình giúp ta hiển thị hệ thống như chính nó hay như cách mà ta muốn nó hiển thị.

Mô hình cho phép ta đặc tả cấu trúc hay hành vi hệ thống.

Mô hình cho ta mẫu để hướng đẫn trong việc xây dựng hệ thống.

Mô hình giúp ta làm tài liệu cho các quyết định khi phân tích thiết kế hệ thống.

1.3 NGUYÊN TẮC QUẢN LÝ ĐỘ PHỨC TẠP

Như đã trình bày trên thì nhiệm vụ quan trọng nhất của người xây dựng hệ thống phần mềm ngày nay là quản lý được độ phức tạp. Theo *Coad* và *Yourdon* [COAD91] thì các nguyên tắc quản lý độ phức tạp của hệ thống trong phân tích và thiết kế hướng đối tượng bao gồm các vấn đề mô tả dưới đây.

Trừu tượng hóa. Sử dụng nguyên tắc trừu tượng hóa có nghĩa là thừa nhận thế giới thực là phức tạp; thay vì cố gắng hiểu biết *toàn bộ* bằng lựa chọn một phần của vấn đề. Trừu tượng bao gồm hai loại chính: trừu tượng thủ tục (*procedural*) và trừu tượng dữ liệu (*data*). Trừu tượng thủ tục thường được đặc trưng bởi trừu tượng *chức năng/chức năng con*. Chia nhỏ tiến trình xử lý thành các bước là phương pháp cơ bản để quản lý độ phức tạp. Tuy nhiên việc chia nhỏ để tổ chức thiết kế là khá tùy tiện và không ổn định. Trừu tượng thủ tục không phải là hình thức trừu tượng

của phương pháp hướng đối tượng. Trừu tượng dữ liệu là cơ chế mạnh, dựa trên cơ sở tổ chức suy nghĩ và đặc tả về các nhiệm vụ của hệ thống. Trừu tượng dữ liệu là nguyên tắc xác định kiểu dữ liệu cho các thao tác áp dụng cho đối tượng, với ràng buộc là các giá trị lưu trữ trong đối tượng chỉ được sửa đổi hay quan sát thông qua các thao tác đó. Người thiết kế áp dụng trừu tượng dữ liệu để xác định thuộc tính và phương thức xử lý thuộc tính xâm nhập thuộc tính thông qua phương thức.

Bao bọc (*encapsulation*). Còn gọi là dấu thông tin. Nguyên tắc này dựa trên nền tảng là mỗi thành phần của chương trình được bao bọc hay dấu quyết định thiết kế đơn lẻ. Giao diện tời mỗi mođun được hình thành sao cho ít nhìn thấy các công việc bên trong. Bao bọc làm tăng tính sử dụng lại khi phát triển hệ thống mới. Bao bọc các nội dung liên quan với nhau làm giảm lưu lượng giữa các phần của hệ thông khi hoạt đông.

Kế thừa (*inheritance*). Cơ chế biểu diễn tính tương tự của các lớp, đơn giản hóa định nghĩa những lớp tương tự từ các lớp khác đã định nghĩa trước. Nó miêu tả tổng quát hóa và đặc biệt hóa, tạo ra các thuộc tính và phương thức chung cho các lớp phân cấp. Nguyên tắc này hình thành nền tảng của kỹ thuật biển diễn những cái chung của các lớp.

Kết hợp (*association*). Là hợp nhất hay liên kết các ý tưởng (từ điển Webster's). Sử dụng kết hợp để gắn một vài phần tử xảy ra vào thời điểm cụ thể hay dưới điều kiện tương tự nhau. Thí dụ, gắn xe cộ vào chủ xe…

Giao tiếp bằng thông điệp. Thông điệp là giao tiếp (viết nói) được trao đổi giữa người với người (từ điển Webster's). Giao tiếp bằng thông điệp liên quan đến tính bao bọc trong đó các chi tiết của hành động sẽ thực hiện được bao bọc trong nơi nhận thông điệp.

Đa hình (*polymorphism*). Đa hình là các thông điệp đồng âm được gửi đến các đối tượng của lớp khác nhau để khởi sự những hành vị khác nhau [OEST00].

Ảnh hưởng của phương pháp tổ chức. Bách khoa toàn thư *Britannica* cho rằng để hiểu thế giới thực, con người sử dụng ba phương pháp tổ chức suy nghĩ như sau: (1) Phân biệt đối tượng cụ thể với các thuộc tính của nó; thí dụ, phân biệt cây với kích thước của nó hoặc quan hệ không gian với các đối tượng khác. (2) Phân biệt giữa toàn bộ đối tượng với thành phần của nó; thí dụ, phân biệt cây với cành cây.(3) Phân biệt giữa các lớp đối tượng với nhau; thí dụ, phân biệt lớp cây với lớp đất đá. Cả ba phương pháp tổ chức này được áp dụng và chúng cho cái nhìn rõ ràng hơn trong lĩnh vực vấn đề và trách nhiệm của hệ thống khi tiếp cận hướng đối tượng.

Quy mô (*scale*). Nguyên tắc áp dụng qui tắc tổng thể-thành phần để quan sát cái gì đó rất lớn được gọi là qui mô.

Phân lớp hành vi. Sau khi đã tìm ra ảnh hưởng tổ chức suy nghĩ, ta phải hiểu rõ các hành vi của đối tượng. Hành vi là những phản ứng, cách cư xử biểu hiện ra ngoài. Phân lớp hành vi bao gồm ba loại sau: (a) trên cơ sở tạo ra kết quả tức thì; (b) sự tương tự của lịch sử tiến hóa (thay đổi theo thời gian) và (c) sự tương tự của các chức năng.

Mẫu (*pattern*). Năm 1977 Christopher Alexander [MULL97] đã đề xuất khái niệm mẫu khi thiết kế hệ thống theo quan điểm hướng đối tượng. Mẫu là tổ hợp đối tượng và lớp. Lợi thế của thiết kế theo mẫu cho phép xử lý các quan niệm về kiến trúc ở mức cao hơn đối tượng vì chúng là cụ thể trong lĩnh vực ứng dụng. Có thể xem mối tương ứng giữa mẫu và các đối tượng như mối tương ứng giữa chương trình con và các dòng lệnh chương trình. Mẫu là đơn vị sử dụng lại trong thiết kế hướng đối tượng.

1.4 NGUYÊN TẮC MÔ HÌNH HÓA

Khả năng của con người là có giới hạn khi khảo sát các vấn đề phức tạp như tổng thể. Thông qua mô hình hóa ta sẽ giới hạn vấn đề nghên cứu bằng cách chỉ tập trung vào một khía cạnh của vấn đề vào một thời điểm. Đó là quan điểm *chia để trị* và *Edsger Dijkstra* đã phát biểu từ vài năm trước đây: *Tấn công vào vấn đề khó bằng cách chia nhỏ nó thành dãy các vấn đề nhỏ hơn mà ta có thể giải quyết được*. Mô hình hóa sẽ làm tăng tri thức của con người. Việc chọn mô hình đúng cho khả năng mô hình làm việc ở mức trừu tượng cao. Mô hình hóa đã có lịch sử lâu đời trong mọi lĩnh vực kỹ nghệ. Người ta đã rút ra bốn nguyên tắc cơ bản sau [BRJ99]:

1. *Việc chọn mô hình nào để tạo lập có ảnh hưởng sâu sắc đến cách giải quyết vấn đề và cách hình thành các giải pháp.*

   Các mô hình đúng sẽ làm sáng tỏ vấn đề phát triển phức tạp nhất, cho cái nhìn thấu đáo vấn đề cần giải quyết. Mặt khác, mô hình tồi sẽ làm ta lạc lối, làm cho ta chỉ tập trung vào các nhiệm vụ không thích hợp. Việc chọn mô hình cho hệ thống phần mềm tác động mạnh đến cách quan sát thế giới. Nếu xây dựng hệ thống theo cách nhìn của người phát triển CSDL thì họ sẽ tập trung vào mô hình quan hệ thực thể, đẩy hành vi vào trigger (khởi sự hành động) và thủ tục lưu trữ. Dưới con mắt của người phân tích cấu trúc, mô hình tập trung vào thuật toán và luồng dữ liệu từ tiến trình này sang tiến trình khác. Dưới con mắt của người phát triển hướng đối tượng, hệ thống có kiến trúc tập trung vào vô số lớp và các

   mẩu thử tương tác giữa các đối tượng lớp. Một cách tiếp cận trên có thể phù hợp cho mỗi lớp ứng dụng hay thói quen phát triển hệ thống. Tuy nhiên kinh nghiệm cho thấy rằng tiếp cận hướng đối tượng là ưu việt nhất cho mọi kiến trúc. Mỗi cách nhìn thế giới sẽ dẫn đến sự khác nhau về giá cả và lợi nhuận của hệ thống được xây dựng.

2. *Mỗi mô hình biểu diễn hệ thống với mức độ chính xác khác nhau.*

   Khi xây dựng nhà cao tầng, đôi khi ta phải đứng xa hàng trăm mét để quan sát tổng thể. Tương tự với các mô hình phát triển phần mềm, đôi khi ta chỉ cần có mô hình nhanh, đơn giản về giao diện người dùng; đôi khi lại phải quan sát sâu hơn đến mức các bit khi giải quyết vấn đề tắc nghẽn đường truyền hệ thống. Trong mọi trường hợp nói trên thì các loại mô hình tốt nhất là mô hình cho phép ta lựa chọn mức độ chi tiết khác nhau, phụ thuộc vào ai sẽ là người quan sát và tại sao họ lại cần quan sát nó. Người phân tích và người sử dụng cuối cùng muốn tập trung vào câu hỏi *cái gì?*, người phát triển sẽ tập trung trả lời câu hỏi *như thế nào?*. Cả hai muốn biểu diễn hệ thống ở mức độ chi tiết khác nhau và vào thời điểm khác nhau.

3. *Mô hình tốt nhất phải là mô hinh phù hợp với thế giới thực.*

   Mô hình càng gần với cách suy nghĩ của ta về thế giới thực thì càng dễ dàng quản lý độ phức tạp. Nếu mô hình vật lý của ngôi nhà mà không đáp ứng cách sử dụng của các vật liệu có trên thị trường thì giá trị của mô hình đó chỉ có hạn. Nếu giả thiết của mô hình toán học của con tàu vũ trụ là điều kiện lý tưởng và công nghệ hoàn hảo thì sẽ loại bỏ các đặc điểm nguy hiểm tiềm tàng của tàu vũ trụ thực. Tốt nhất là phải có mô hình kết nối rõ ràng với thế giới thực. Mọi mô hình đều đơn giản hóa thế giới thực, do vậy phải đảm bảo tiến trình đơn giản hóa sẽ không loại bỏ đi các chi tiết quan trọng. Với phần mềm, trong các kỹ thuật phân tích cấu trúc thì mô hình phân tích và mô hình thiết kế hệ thống sẽ được tách biệt nhau. Thiếu cầu nối giữa hại loại mô hình này, dẫn tới hệ thống sẽ được hiểu và xây dựng khác nhau vào các thời điểm khác nhau. Trong hệ thống hướng đối tượng, có khả năng liên kết mọi quan sát tương đối độc lập vào toàn thể.

4. *Không mô hình nào là đầy dủ. Mỗi hệ thống thường được tiếp cận thông qua tập mô hình*

gần như độc lập nhau.

Khi xây dựng tòa nhà, không có một bản thiết kế nào có thể bộc lộ toàn bộ chi tiết của nó. Ít nhất thì ta phải có thiết kế các tầng, thiết kế cầu thang, thiết kế hệ thông điện, nước, thiết kế hệ thống cứu hỏa,… Khái niệm *gần như độc lập nhau* ở đây có nghĩa rằng các mô hình này được hình thành và nghiên cứu tách biệt nhưng nó vẫn có quan hệ với nhau. Thí dụ, ta có thể thiết kế hệ thông điện, nước một cách độc lập, nhưng trong quá trình thiết kế này thì phải quan tâm đến thiết kế của các tầng nhà cho nó phù hợp. Tương tự trong hệ thống phần mềm hướng đối tượng, để hiểu kiến trúc hệ thống phần mềm ta cần một vài quan sát bổ trợ nhau: quan sát *trường hợp sử dụng* diễn tả yêu cầu hệ thống, quan sát thiết kế (thu thập từ vựng của không gian vấn đề và không gian giải pháp), quan sát tiến trình (mô hình hóa phân bổ tiến trình và luồng của hệ thống), quan sát cài đặt (tập trung vào hiện thực vật lý của hệ thống), quan sát triển khai (tập trung vào các nhiệm vụ kỹ nghệ của hệ thống). Mỗi quan sát đều có khía cạnh kiến trúc hay hành vi riêng. Tập hợp lại các quan sát này có khả năng biểu diễn được kế hoạch chi tiết của phát triển phần mềm.

Phục thuộc vào bản chất của hệ thống mà mỗi mô hình có tầm quan trọng khác nhau. Thí dụ, với hệ thống quản lý nhiều dữ liệu thì các mô hình quan sát thiết kế tĩnh sẽ quan trọng hơn. Nếu hệ thống phải có nhiều giao diện người dùng thì các quan sát *trường hợp sử dụng* tĩnh và động đều rất quan trọng. Hệ thống thời gian thực coi trọng quan sát tiến trình động. Cuối cùng, hệ thống phân tán (các ứng dụng Web) coi mô hình triển khai và cài đặt là quan trọng nhất.

1.5 KHÁI QUÁT VỀ TIẾN TRÌNH PHÁT TRIỂN PHẦN MỀM

Hệ thống là tổ hợp phần cứng, phần mềm cung cấp giải pháp cho vấn đề cần giải quyết. Ngày nay, trong khi hệ thống quá phức tạp mà tri thức lại quá chuyên ngành cho nên một người không thể biết mọi khía cạnh tác nghiệp. Một người không thể hiểu đồng thời mọi vấn đề của hệ thống: từ thiết kế giải pháp, viết mã trình, triển khai trên nền phần cứng đến đảm bảo chắc chắn mọi thành phần phần cứng đều làm việc tốt với nhau. Tiến trình phát triển phần mềm phức tạp phải được nhiều người thực hiện. Trước hết là khách hàng, đó là người đưa ra vấn đề cần giải quyết. Phân tích viên làm tài liệu vấn đề của khách hàng và chuyển nó tới người phát triển, đó là những lập trình viên xây dựng phần mềm để giải quyết, kiểm tra và triển khai nó trên các phần cứng. Phát triển phần mềm có thể được thực hiện bằng nhiều con đường khác nhau. Các dự án có thể tuân thủ một trong các loại tiến trình lặp và tăng dần. Mỗi loại có ưu và nhược điểm riêng.

1.5.1 - Các phương pháp mô hình hóa hệ thống

1.5.1.1 - Mô hình thác nước

Từ đã lâu, hệ thống phần mềm thường được mô hình hóa theo phương pháp *thác nước (waterfall)*. Phương pháp này được *Royce* mô tả từ năm 1970 (hình 1.5). Trong mô hình này phát triển phần mềm là dãy các pha liên tiếp từ phân tích yêu cầu, thiết kế hệ thống, phát triển hệ thống đến thử nghiệm và triển khai hệ thống. Pha sau chỉ được bắt đầu khi pha trước đã hoàn thành.

Phân tích

Thiết kế

Viết chương trình

Thử nghiệm

Hình 1.5 Mô hình thác nước

Để xây dựng được hệ thống phần mềm ta phải mô tả được vấn đề (*problem*) và yêu cầu (*requirement*) của khách hàng bằng trả lời các câu hỏi như *vấn đề của hệ thống là gì?* và *hệ thống cần phải làm gì?*. Pha phân tích của tiến trình tập trung vào việc điều tra vấn đề thay cho việc tìm ra giải pháp. Thí dụ, khi xây dựng hệ thống quản lý thư viện thì phân tích có nghĩa là tìm kiếm tiến trình tác nghiệp nào liên quan đến việc sử dụng nó. Để có tài liệu phân tích đầy đủ và đúng đắn thì phải phân tích *lĩnh vực vấn đề*. Lĩnh vực vấn đề là khu vực tác nghiệp của con người, trong đó phần mềm được xây dựng. Thí dụ, hệ thống phần mềm thư viện trong trường học lĩnh vực là giáo dục, sinh viên… Pha thiết kế tập trung vào giải pháp logic, thí dụ phải trả lời câu hỏi hệ *thống đang xây dựng thực hiện các yêu cầu và các ràng buộc như thế nào?*. Trong hệ thống quản lý thư viện thì pha này thực hiện trả lời câu hỏi *thu thập, ghi chép việc mượn, trả sách hay tạp chí như thế nào?*. Pha cài đặt (viết trương trình) tập trung vào mà hóa trương trình.

Phân tích

Thiết kế

Kiểm tra chức năng

Chương trình ứng dụng

hình 1.6 Mô hình chữ "V"

Kiểm tra tích hợp

Viết chương trình Kiểm tra mô đun

Những người tham gia vào xây dựng hệ thống phần mềm như khách hàng, phân tích viên, lập trình viên… theo phương pháp *thác nước* rất ít khi cùng làm việc với nhau để chia sẽ các hiểu biết sâu sắc vấn đề đang giải quyết. Do vậy họ mất rất nhiều thời gian để xây dựng được hệ thống phần mềm.

Chu kỳ *thác nước* còn được biểu diễn dưới dạng chữ V (hình 1.6), trong đó pha kiểm tra được thực hiện đồng thời với các pha phát triển khác. Thí dụ, kiểm tra chức năng được thực hiện trong quá trình phân tích, kiểm tra tích hợp trong pha thiết kế, kiểm tra mođun trong pha mã hóa chương trình.

1.5.1.2 - Mô hình lặp và tăng dần

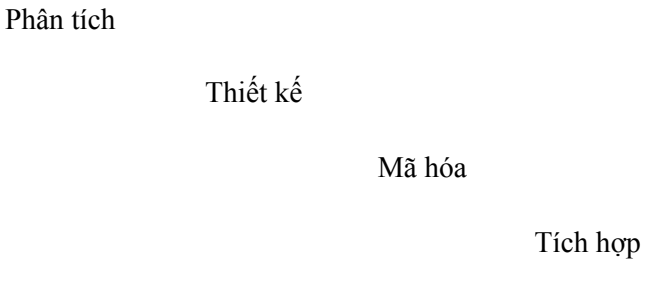
Mô hình *thác nước* không cho ta đi ngược lại chuỗi trình tự phát triển phần mềm như trình bày trên. Bắt đầu dự án, theo mô hình này thì ta phải xác định toàn bộ yêu cầu, nó được thực hiện thông qua bàn bạc với người sử dụng hệ thống và khảo sát chi tiết các tiến trình tác nghiệp. Thực tế thì khi kết thúc công việc, may mắn lắm chỉ 80% nhu cầu hệ thống là được thu thập trong pha

phân tích. Tiếp theo là pha thiết kế, nơi kiến trúc hệ thống sẽ được xác định. Pha này tập trung vào những nhiệm vụ như đặt chương trình ở đâu, cần phần cứng nào… Trong khi thực hiện công việc này, ta có thể tìm ra một số nhiệm vụ mới (nhu cầu mới) của hệ thống. Do đó, xuất hiện nhu cầu đi trở lại người sử dụng để trao đổi, bàn bạc về nó; có nghĩa rằng chúng ta phải trở lại pha phân tích. Sau khi đi lại vài lần như vậy ta mới chuyển đến pha phát triển để bắt đầu lập trình hệ thống. Khi mã hóa chương trình, ta phát hiện ra rằng một vài quyết định khi thiết kế là không thể cài đặt. Vậy ta lại phải trở lại pha thiết kế xem xét lại các nhiệm vụ. Sau khi mã hóa xong, pha kiểm tra bắt đầu. Trong khi kiểm tra ta nhận thấy rằng một vài yêu cầu chưa đủ chi tiết, giải thích nhầm lẫn có thể xảy ra. Vậy ta phải quay trở lại pha phân tích để xem xét lại yêu cầu. Sau vài lần lặp ta mới có được hệ thống hoàn chỉnh và phân phát cho người sử dụng. Tác nghiệp có thể thay đổi theo thời gian khi xây dựng hệ thống. Người sử dụng có thể phàn nàn về sản phẩm ta làm ra không hoàn toàn như họ mong đợi. Nguyên nhân có thể là: vấn đề tác nghiệp (*bussiness*) thay đổi quá nhanh; người sử dụng không truyền đạt đúng cái họ muốn; đội ngũ dự án không tuân thủ tiến trình… Đội ngũ phát triển thường lập ra các biểu đồ và vô số tài liệu, văn bản, nhưng người dùng không phải lúc nào cũng hiểu cái mà đội ngũ phát triển cung cấp cho họ. Giải pháp nào để tránh các vấn đề này? Câu trả lời là mô hình hóa trực quan có thể giúp họ.

Phát triển phần mềm là tiến trình phức tạp. Nếu bỏ qua khả năng quay trở lại các bước thực hiện trước đó thì thiết kế hệ thống có thể sai làm và thiếu sót nhu cầu. Để có thể đi ngược lại các bước phát triển hệ thống phần mềm ta có phương pháp mới, phương pháp phát triển lặp (*iterative*). Phát triển lặp là làm đi làm lại việc gì đó. Trong phương pháp này ta sẽ đi qua các bước phân tích, thiết kế, phát triển, kiểm tra và triển khai phần mềm theo từng bước nhỏ nhiều

lần. Cần nhớ rằng không có khả năng thu thập đầy đủ mọi yêu cầu vào công đoạn đầu tiên của dự án. Các vấn đề có thể nảy sinh, vậy ta phải lập kế hoạch lặp trong dự án. Theo quan niệm này thì dự án được coi là dãy các thác nước nhỏ. Mỗi thác nước được thiết kế sao cho đủ lớn để hoàn thành thiện từng bộ phận quan trọng của dự án và đủ nhỏ để tối thiểu nhu cầu đi trở lại.

Hình 1.7 [MULL97] cho thấy mỗi chu kỳ lặp là một vòng đời thác nước nhỏ. Vòng lặp sau được hình thành trên cơ sở tiến hóa của vòng lặp trước đó. Như vậy, các pha truyền thống được lặp đi lặp lại và tăng dần. Trong phương pháp này, phân tích viên, người thiết kế, người lập trình… hợp tác làm việc để hiểu biết sâu sắc hệ thống, chi sẻ các ý tưởng mới dẫn tời xây dựng được hệ thông mạnh, phức tạp hơn.



Hình 1.7 Mô hình lặp và tăng dần

Có nhiều biết thể của chu kỳ lặp. Các chu kỳ lặp này được hình thành trên cơ sở phạm vi của dự án, độ phức tạp của vấn đề và các lựa chọn kiến trúc. Thí dụ, chu kỳ lặp hình chữ *b* của *Birrel N.D* và *Ould M.A*. (1985) tập trung vào pha bảo trì hệ thống, được áp dụng cho các dự án trung bình; chu kỳ lặp hình chữ *O* của *Boehm B.M* (1988) bao gồm lặp công việc phân tích và thiết kế. Mô hình này cho khả năng các nhóm thực hiện song song dự án.

1.5.2 - Các pha phát triển phần mềm

Không có tiến trình phát triển phần mềm nào là phù hợp cho mọi dự án, mọi lĩnh vực ứng dụng [MULL97]. Phần này mô tả tiến trình phát triển phần mềm tổng quát, mỗi dự án phải thích nghi chúng với các ràng buộc riêng. Phát triển phần mềm được quan sát từ hai góc độ bổ trợ nhau. Đó là, góc độ hỗ trợ bao gồm các khía cạnh tài chính, chiến lược, thị trường và con người và góc độ kỹ thuật bao gồm kỹ nghệ, kiểm tra chất lượng và phương pháp mô hình hóa.

1.5.2.1 - Khía cạnh kỹ thuật trong tiến trình phát triển phần mềm

Góc nhìn kỹ thuật tập trung vào triển khai và tổ chức các hoạt động kỹ thuật để dẫn tời sản sinh các thế hệ phần mềm khác nhau. Như trình bày trên, chu kỳ phát triển được xem như trình tự các lặp, thông qua nó mà phần mềm tiến triển dần. Kết quả của mỗi vòng lặp là chương trình có thể chạy được (khả thực). Nội dung của lặp phải có ích cho tiến trình phát triển và cho người sử dụng. Một số kết quả của lặp chỉ được sử dụng nội bộ tiến trình phát triển, một số khác được sử dụng để thể hiện trạng thái tiến triển. Từ lặp này đến lặp khác, chương trình sẽ được bổ sung các chức năng mới và chất lượng của nó đươc nâng cao cho đến một vòng lặp nào đó sẽ sản sinh ra thế hệ thứ nhất của phần mềm.

Lặp	Kết quả	Pha
Lặp khởi đầu	Makét	Khảo sát khả thi
Lặp kiến trúc	Bản mẫu kiến trúc	
Lặp kiến trúc	Bản mẫu kiến trúc	Chi tiết
Lặp phát triển	Bản mẫu phát triển	
Lặp phát triển	Bản mẫu phát triển	
Lặp phát triển	Bản beta	Xây dựng Chuyển giao
Lặp chuyển giao	Bản beta	
Lặp chuyển giao	Thế hệ phần mềm	

Hình 1.8 Tiến trình phát phần mềm

Các hoạt động truyền thống không được thực hiện trình tự như trong chu kỳ phát triển *thác nước*, nó được phân bổ vào các lặp khác nhau. Mỗi lặp bao gồm lập kế hoạch, phân tích, thiết kế, cài đặt, thử nghiệm và tích hợp. Chương trình khả thực là kết quả của các bước lặp được gọi là bản mẫu (*prototype*). Bản mẫu là tập con của một ứng dụng, nó được sử dụng để hình thành các

yêu cầu hay đánh giá hiệu quả của công nghệ lựa chọn. Bản mẫu ngày càng phong phú thông qua mỗi bước lặp. Thông thường, bản mẫu thứ nhất chỉ có một mục đích là chưng minh quan niệm ứng dụng. Nó được hình thành càng nhanh càng tốt, đôi khi bỏ qua tiêu chuẩn chất lượng thông thường. Bản mẫu này thường được gọi là mô hình hay maket. Các bước lặp áp chót sẽ phát sinh các phiên bản beta, nó được gửi đến nhóm người sử dụng thử nghiệm. Phiên bản chính thức cũng là bản mẫu như các bản mẫu khác (hình 1.8) [MULL97]. Nó được xem như bản mẫu cuối cùng của thế hệ phần mềm đang phát triển và là bản mẫu đầu tiên của thế hệ phần mềm tiếp theo.

1.5.2.2 - Quản lý rủi ro trong tiến trình phát triển lặp

Như trong mọi hoạt động khác của con người, phát triển phần mềm phải tuân thủ luật chia sẻ rủi ro. Phân tích rủi ro bao gồm đánh giá dự án, công nghệ và tài nguyên để xác định, hiểu rõ bản chất của rủi ro và quan hệ giữa chúng phải được mô tả vắn tắt để mọi thành viên trong dự án biết chúng. Rủi ro phải được lượng hóa và phải biết rằng nó có thể hay không có thể loại bỏ. Không được mô tả thiếu rõ ràng trong tài liệu như "Hệ thống cần đủ nhanh" hay "Bộ nhớ cần đủ lớn"…

Rủi ro của dự án được chia thành bốn nhóm như sau đây:

Rùi ro về thương mại: Có thể thu thập đầy đủ thông tin cạnh tranh của sản phẩm trên thị trường?

Rủi ro về tài chính: Nhà đầu tư phát triển phần mềm có đủ kinh phí để hoàn thành dự án?

Rủi ro về kỹ thuật: Nền tảng công nghệ vững chắc và đã được thử thách?

Rủi ro về phát triển: Đội ngũ phát triển có đủ kinh nghiệm? Họ có làm chủ hoàn toàn công nghệ đang sử dụng?

Tuy nhiên, luôn nhớ rằng cái rủi ro lớn nhất là không biết được rủi ro xảy ra ở đâu. Nhiệm vụ quản lý rủi ro là phải lập kế hoạch làm giảm rủi ro, sau đó là thực hiện kế hoạch này.

1.5.2.3 - Khía cạnh hỗ trợ tiến trình phát triển phần mềm

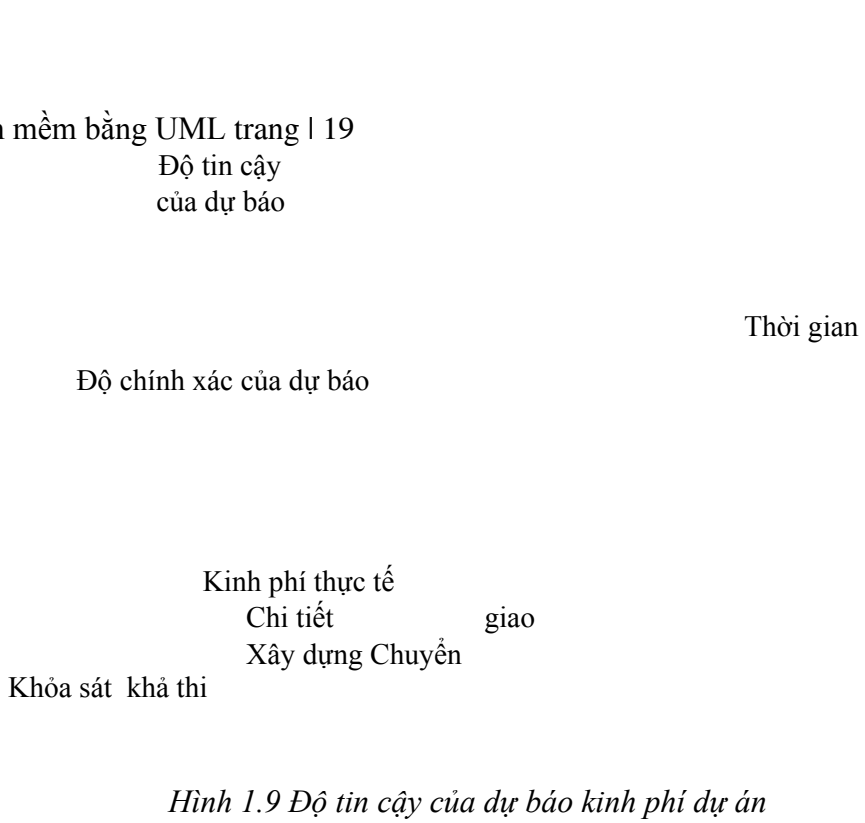
Từ góc nhìn hỗ trợ, phát triển phần mềm được thực hiện trong bố pha: Khảo sát khả thi hay khởi đâu (*inception*), chi tiết (*elaboration*), xây dựng (*construction*) và chuyển giao (*transition*). Chu kỳ phát triển phần mềm từ góc độ này được mô tả trên hình 1.8. Hình này còn cho thấy quan hệ giữa hai góc nhìn khác nhau: góc nhìn hỗ trợ và góc nhìn kỹ thuật. Dưới đây là mô tả các pha phát triển phần mềm từu góc nhìn hỗ trợ.

Pha khởi đầu (hay khảo sát khả thi). Pha này bao gồm khảo sát thị trường, đặc tả sản phẩm cuối cùng, xác định phạm vi dự án. Khởi đầu là bắt đầu dự án, từ khi ai đó cho rằng nếu có hệ thống phần mềm mới để giúp đỡ công việc của họ thì tốt hơn. Tiếp theo là ai đó nghiên cứu ý tưởng. Người quản lý (khách hàng) hỏi bao lâu thì có phần mềm? Kinh phí là phần mềm là bao nhiêu? Tính khả thi của dự án thế nào? Tiến trình tìm ra các câu trả lời cho các câu hỏi này thuộc pha khởi đầu. Khảo sát thị trường không phải là công việc của các kỹ sư phần mềm mà là công việc của chuyên gia khảo sát thị trường và phân tích cạnh tranh. Họ cố gắng đánh giá việc xây dựng hệ thống phần mềm mới hay nâng cấp phần mềm có sẵn có kinh tế không, giúp các công ty xác định các ưu, nhược điểm. Công việc đầu tiên ở đây là hình dung bức tranh tổng quát của sản phẩm để dễ dàng nhận ra và tách các thành phần cho pha chi tiết. Theo E. Morel thì bức tranh này được mô tả như sau:

Bức tranh sản phẩm = Cái gì + Cho ai + Giá bao nhiêu

trong đó, Cái gì muốn đề cập đến các đặc trưng tông thể về sản phẩm; *Cho ai* xác định khách hàng sử dụng sản phẩm và *Giá bao nhiêu* dự báo giá của mỗi sản phẩm mà người sử dụng có thể chấp nhận.

Thông thường phải xây dựng *bản mẫu khái niệm* cho pha khảo sát khả thi để đánh giá tính rủi ro của các chức năng phần mềm, sức mạnh và độ phức tạp của hệ thống, công nghệ mới sẽ áp dụng. Từ đó mà các quyết định về quan niệm sản phẩm được hình thành. Loại bản mẫu này không cần tuân thủ các quy luật phát triển phần mềm thông thường như độ tin cậy cao hay tốc độ xử lý phải nhanh. Đó chỉ là maket hệ thống, mã trình của nó sẽ không đóng vai trò gì trong sản phẩm cuối cùng. Từ đây, tính rủi ro của dự án được nhận biết và pha khảo sát thì sẽ tiếp tục cố gắng đánh giá kinh phí cho dự án và lợi nhuận sẽ đem lại. Dự báo kinh phí luôn là công việc khó khăn. Số liệu của dự báo ban đầu sẽ không bao giờ đúng, chỉ có được số liệu đúng khi kết thúc phát triển hệ thống. Do vậy, việc dự báo này thường được hiệu chỉnh dần dần trong nhiều bước khác.



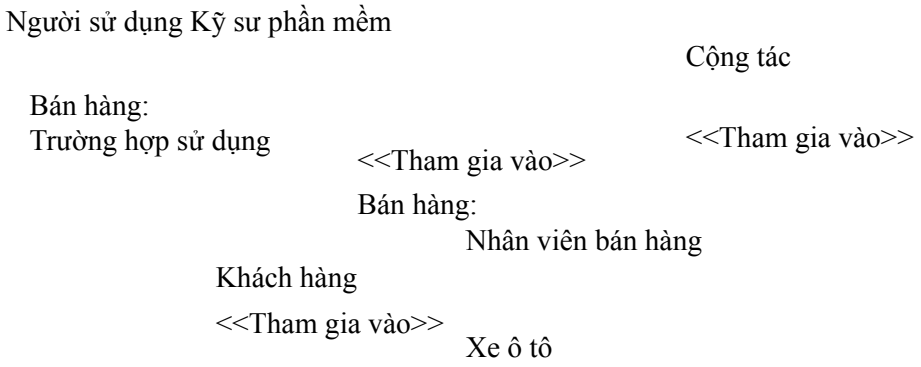
Hình 1.9 Độ tin cậy của dự báo kinh phí dự án

Với các dự án nhỏ, khảo sát khả thi thường chỉ giới hạn bởi danh sách yêu cầu phần mềm. Với các dự án trung bình (thực hiện khoảng một năm) thì pha khảo sát khả thi sẽ thực hiện trong khoảng một tháng, bao gồm cả việc xây dựng maket. Với các dự án lớn, việc xây dựng maket có thể là một dự án con và nó cũng có các bước thực hiện như mô tả trên.

Pha chi tiết. Pha chi tiết bắt đầu bằng phân tích yêu cầu và mô hình hóa lĩnh vực. Nó có

nhiệm vụ lựa chọn kiến trúc, làm giảm mức độ rủi ro của dự án, cuối cùng là xác định được kế hoạch đầy đủ cho các nhiệm vụ phát triển hệ thống phần mềm. Tham gia vào pha này bao gồm kiến trúc sư hệ thống, chuyên gia lĩnh vực, người sử dụng, đại diện của nhóm kiểm tra chất lượng và thử nghiệm, người viết tài liệu, chuyên gia về các công cụ phát triển phần mềm.

Phân tích yêu cầu được thực hiện trên cơ sở khảo sát các trường hợp sử dụng. Các trường hợp sử dụng được mô tả theo khái niệm khách hàng, có thể khác xa với hình thức mô tả của kỹ nghệ phần mềm. Các phân tích viên có nhiệm vụ chuyển đổi chúng sang hình thức gần máy tính hơn, thí dụ, chuyển đổi trường hợp sử dụng sang cộng tác của các đối tượng trong lĩnh vực ứng dụng. Đó là các đối tượng trong thế giới thực, công tác với nhau để hình thành các chức năng trong trường hợp sử dụng. Người sử dụng vẫn có thể hiểu rõ trên hình thức biểu diễn này. Hình 1.10 là thí dụ về cài đặt trường hợp sử dụng Bán hàng bằng ba đối tượng hợp tác là khách hàng, người bán hàng và xe ô tô.



Hình 1.10 Cài đặt trường hợp sử dụng

Pha chi tiết cho lại các sản phẩm sau đây:

Mô tả hành vi hệ thống dưới hình thức *trường hợp sử dụng* (use case), ngữ cảnh hệ thống, các tác nhân (người sử dụng hệ thống), các kịch bản ứng dụng và mô hình các lớp ứng dụng. Với dự án trung bình thì có thể bao gồm hàng chục trường hợp sử dụng, khoảng 100 kịch bản chính, vài trăm kịch bản phụ và khoảng từ 50 đến 100 lớp lĩnh vực.

Kiến trúc, tài liệu mô tả kiến trúc, mô tả rủi ro.

Kế hoạch đầy đủ cho phát triển trong dự án.

Kế hoạch chi tiết cho các vòng lặp, tiêu chí đánh giá, danh sách yêu cầu cho mỗi vòng lặp và kết quả cảu kiểm tra chất lượng.

Có thể bao gồm các bản thảo của tài liệu hướng dẫn sử dụng.

Thời gian thực hiện pha chi tiết rất khác nhau trong từng dự án, nó phụ thuộc vào loại ứng dụng và cơ sở hạ tầng lựa chọn cho nó. Thí dụ, nếu dự án thực hiện trong một năm thì pha này sẽ kéo dài từ hai đến bốn tháng. Không nên đánh giá quá nhiều thời gian để có được phân tích hoàn hảo. Nên chuyển dần sang giai đoạn tiếp theo khi đã khảo sát khoảng 80% kịch bản chính. Không nên đi quá chi tiết khi khảo sát kịch bản để tránh việc mất đi tính trừu tượng trong giải pháp.

Pha xây dựng. Pha xây dựng đề cập đến tiến trình phát triển và kiểm tra phần mềm. Pha xây

dựng tương ứng với triển khai các vòng lặp. Mỗi vòng lặp ở đây cho lại một mẫu khả thực ổn định, đầy đủ. Đó là những phiên bản đầu tiên của phần mềm. Việc cài đặt lặp bao gồm các hoạt động như sau: Nhận ra các kịch bản hoàn chỉnh hay sử dụng lại trong vòng lặp trên cơ sở khảo sát các rủi ro và kết quả của vòng lặp trước đó; giao nhiệm vụ cụ thể cho đội ngũ phát triển để hoàn thiện vòng lặp; xác định tiêu chí đánh giá, vị trí kiểm tra và hạn định; tập hợp lại tài liệu người sử dụng và tài liệu triển khai.

Pha xây dựng kết thúc khi hoàn thiện phần mềm và được kiểm nghiệm. Phải đảm bảo rằng phần mềm đồng bộ với mô hình. Với dự án nhỏ, pha xây dựng kéo dài từ hai đến ba tháng. Trong các nước phát triển, dự án hướng đối tượng trung bình được thực hiện trong khoảng một năm với năm hay sáu nhân viên thì pha xây dựng chiếm từ sáu đến chín tháng. Với dự án lớn thì mỗi tiểu hệ được xem như dự án con và chúng cũng có các vòng lặp riêng.

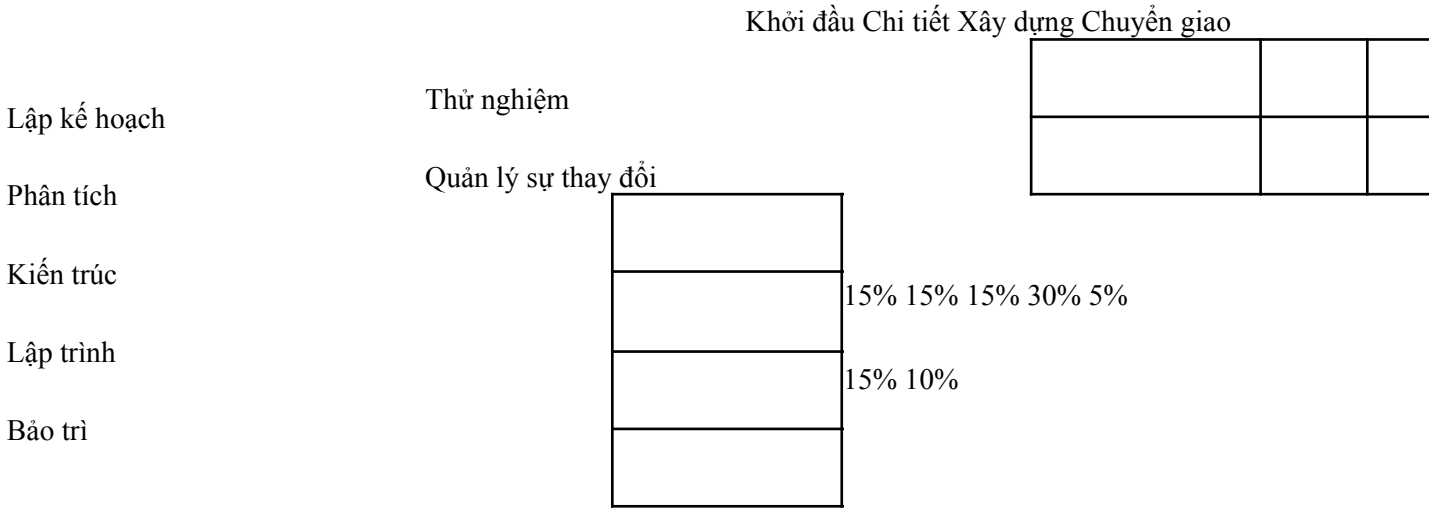
Pha chuyển giao. Pha chuyển giao bắt đầu khi sản phẩm phần mềm hoàn thiện được chuyển đến cộng đồng người sử dụng. Mức độ phức tạp của pha này phụ thuộc vào các ứng dụng cụ thể. Pha chuyển giao bao gồm sản xuất hàng loại, vận chuyển, cài đặt, huấn luyện, hỗ trợ kỹ thuật và bảo trì. Tham gia vào pha này bao gồm một vài người từ đội ngũ phát triển và thử nghiệm chương trình, kiến trúc sư hệ thống làm việc bán thời gian và người có trách nhiệm cập nhật tài liệu. Nhân viên hỗ trợ kỹ thuật, nhân viên bán hàng, quảng cáo sẽ thực hiện việc kiểm tra.

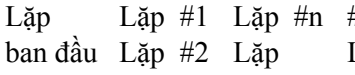
Trong pha chuyển giao, công việc phát triển phần mềm hầu như đã hoàn thành. Mọi sản phẩm đều đạt tời mức chín muồi để cung cấp rộng rãi đến người quản lý dự án và người sử dụng. Người sử dụng sẽ được cung cấp: phiên bản beta và phiên bản cuối cùng của chương trình khả thực; tài liệu hướng dẫn sử dụng, tài liệu triển khai và cài đặt. Người quản lý dự án được cung cấp: có mô hình hệ thống cuối cùng; tiêu chí đánh giá cho mỗi vòng lặp; mô tả việc phân phối sản phẩm; kết quả của kiểm tra chất lượng và các kinh nghiệm rút rra từ thực hiện dự án.

1.5.2.4 - Phân bổ hoạt động trong các pha phát triển phần mềm

Mỗi hoạt động phát triển phần mềm là khác nhau trong các pha phát triển. Không có sự tương ứng một – một giữa các pha dưới góc nhìn hỗ trợ với các pha cổ điển của chu kỳ *thác nước*. Các hoạt động như phân tích, triển khai trải dài trên nhiều pha từ góc nhìn hỗ trợ. Các hoạt động dược

phân bổ giữa các chu kỳ, điểm bắt đầu và kết thúc của chúng không tương ứng với các giới hạn các pha từ góc nhìn hỗ trợ (hình 1.11).





Hình 1.11 Sơ đồ các pha phát triển phần mềm

Công sức Thời gian

5% 20% 65% 10% 10% 30% 50% 10%

Sơ đồ trên hình 1.11 cho ta thấy:

Kế hoạch thực hiện trong các pha phát triển.

Công việc phân tích được thực hiện chủ yếu trong pha chi tiết, một phần của nó được thực hiện trong pha khảo sát khả thi và pha xây dựng.

Phần lớn kiến trúc được xác định trong pha chi tiết.

Việc viết mã trình (bao gồm cả thử nghiệm các modun) bắt đầu từ pha chi tiết và được thực hiện trong toàn bộ pha cài đặt.

Việc bảo trì được xác định ngay từ phiên bản đầu tiên của phân mềm, thông thường nó bắt đầu trong pha chi tiết.

Thử nghiệm và kiểm tra chất lượng trả dài suốt toàn bộ các pha và áp dụng cho mọi bản mẫu chương trình.

Quản lý sự thay đổi (phiên bản và cấu hình) lưu trữ lịch sử toàn bộ dự án.

Dãy số bên phải hình 1.11 thể hiện phân bổ công sức thực hiện các hoạt động của một dự án phát triển phần mềm hướng đối tượng do năm nhân viên thực hiện trong khoảng một năm. Các hoạt động đó bao gồm:

Lập kế hoạch: bao gồm các hoạt động thí điểm dự án, kế hoạch phát triển quản lý các tài nguyên dự án.

Phân tích: Bao gồm phát triển mô hình đối tượng và mô hình người sử dụng, đặc tả tổng thể về dự án, mô tả các tiêu chí đánh giá.

Kiến trúc: bao gồm viết mã trình cơ sở, thiết kế tổng quan về ứng dụng và xây dựng cơ chế chung

Cài đặt: Nhóm toàn bộ thiết kế chi tiết, viết mã trình và kiểm tra các mođun chương trình.

Thử nghiệm và đánh giá: Bao gồm các hoạt động chứng minh rằng phần mềm phù hợp với mục tiêu ban đầu và các tiều chí đánh giá.

Quản lý thay đổi: Lưu trữ trạng thái kết quả của các giai đoạn phát triển.

Các dãy số phía dưới hình 1.11 chỉ ra phân bố công sức và thời gian cho các pha phát triển phần mềm của một dự án cỡ trung bình.

Chính bản thân *Ngôn ngữ mô hình hóa trực quan UML* không định nghĩa tiến trình phát triển phần mềm, nhưng UML và phần mềm công cụ *Rational Rose* (còn gọi tắt là *Rose*) được sử dụng có hiệu quả trong một số công đoạn của tiến trình phát triển phần mềm. Khi bắt đầu dự án, trong pha khảo sát khả thi, *Rose* được sử dụng để phát sinh mô hình trường hợp sử dụng. Trong pha chi tiết, *Rose* được sử dụng để xây dựng biểu đồ trình tự và biểu đồ cộng tác, chỉ ra các đối tượng tương tác với nhau như thế nào. Biểu đồ lớp được xây dựng trong *Rose* để thấy sự phụ thuộc vào nhau của các đối tượng. Trong giai đoạn đầu của pha xây dựng, biểu đổ thành phần được hình thành nhờ *Rose* để chỉ ra sự phụ thuộc của các thành phần trong hệ thống và cho ta khả năng phát

sinh mã trình cơ bản. Trong suốt pha xây dựng, *Rose* cho ta khả năng chuyển đổi ngược mã trình thành mô hình để tích hợp mọi sự thay đổi trong quá trình phát triển. Trong pha chuyển giao, *Rose* được sử dụng để cập nhật các mô hình được tạo lập ra cho dự án.

CHƯƠNG 2

KHÁI QUÁT VỀ UML

UML là ngôn ngữ mô hình hóa, trước hết nó là mô tả ký pháp thống nhất, ngữ nghĩa và các định nghĩa về *metamodel* (mô tả định nghĩa chính ngôn ngữ mô hình hóa), nó không mô tả về phương pháp phát triển. UML được sử dụng để hiển thị, đặc tả, xây dựng và làm tài liệu các vật phẩm (*artifacts*) của phân tích quá trình xây dựng hệ thống phần mềm theo hướng đối tượng. UML được sử dụng cho mọi tiến trình phát triển phần mềm, xuyên suốt vòng đời phát triển và độc lập với các công nghệ cài đặt hệ thống.

2.1 GIỚI THIỆU UML

UML là ngôn ngữ chuẩn để viết kế hoạch chi tiết phần mềm. Nó phù hợp cho việc mô hình hóa các hệ thống như hệ thông tin doanh nghiệp, các ứng dụng phân tán trên nền Web, hệ thông nhúng thời gian thực… Các khung nhìn của ngôn ngữ được quan sát từ góc độ phát triển và triển

khai hệ thống, nó không khó hiểu và dễ sử dụng. UML là ngôn ngữ mô hình được cả con người và máy sử dụng. Nhớ lại rằng phương pháp là cách cấu trúc rõ ràng suy nghĩ và hành động của ai đó. Phương pháp cho người sử dụng biết làm cái gì, làm như thế nào, khi nào làm việc đó và tại sao lại làm như vậy. Phương pháp chứa mô hình và các mô hình này đươc sử dụng để mô tả cái gì đó. Sự khác nhau chủ yếu của phương pháp và ngôn ngữ mô hình hóa là ngôn ngữ mô hình hóa thiếu tiến trình cho biết làm cái gì, làm như thế nào, khi nào làm việc đó và tại sao lại làm nhu vậy. Như mọi ngôn ngữ mô hình hóa khác, UML có ký pháp (các biểu tượng sử dụng trong mô hình) và tập các luật sử dụng nó. Các luật bao gồm cú pháp, ngữ nghĩa và *pragmatic* (luật hình thành câu có nghĩa).

Cần chú ý rằng ngay cả khi nắm chắc ngôn ngữ UML, không có gì đảm bảo sẽ có mô hình tốt. Tương tự nhà văn viết tiểu thuyết bằng ngôn ngữ tự nhiên, ngôn ngữ chỉ là công cụ, còn tiểu thuyết có hay hay không là phục thuộc hoàn toàn vào nhà văn.

Để sử dụng UML có hiệu quả, đòi hỏi phải hiểu được ba vấn đề chính sau [BRJ99]

Các phần tử cơ bản của mô hình UML

Các qui định liên kết các phần tử mô hình

Một số cơ chế chung áp dụng cho ngôn ngữ này.

UML là ngôn ngữ và nó chỉ là một phần của tiến trình phát triển phần mềm, độc lập với tiến trình. Tuy nhiên UML rất phù hợp với các tiến trình hướng *trường hợp sử dụng* (*Use case – UC*), lấy kiến trúc làm trung tâm, tương tác tăng dần.

2.1.1.1 - UML là ngôn ngữ

Ngôn ngữ phải có từ vựng và qui tắc tổ hợp các từ trong từ vựng để giao tiếp. Ngôn ngức mô hình là ngôn ngữ có từ vựng và qui tắc tập trung vào biểu diễn về mặt vật lý và khái niệm của hệ thống. UML là ngôn ngữ chuẩn công nghiệp để lập kế hoạch chi tiết phần mềm. Như đã trình bày trong chương trước, không có mô hình nào là thỏa mãn cho toàn bộ hệ thống. Thông thường ta phải xây dựng nhiều mô hình cho một hệ thống, do vậy ngôn ngữ phải cho phép biểu diễn nhiều khung nhìn (*views*) khác nhau của một kiến trúc hệ thống trong suốt quá trình phát triển phần mềm. Từ vựng và qui tắc ngôn ngữ UML cho ta cách thức xây dựng mô hình và đọc mô hình, nhưng không cho biết mô hình nào cần phải được lập và khi nào lập chúng. Nhiệm vụ đó được xác định nhờ qui trình phát triển phần mềm. Qui trình phát triển phần mềm sẽ giúp chúng ta đưa

ra quyết định hình thành vật phẩm nào, hoạt động nào và nhân viên nào sẽ tạo ra, sử dụng và quản lý chúng. Đồng thời chúng được sử dụng như thế nào vào việc quản lý toàn bộ dự án.

2.1.1.2 - UML là ngôn ngữ để hiển thị

Với nhiều lập trình viên, không có khoảng cách từ ý tưởng đến cài đặt mã trình. Họ suy nghĩ vấn đề và viết ngay mã trình cho nó. Thực tế là có thể trực tiếp viết mã trình cho một số việc, trực tiếp mô tả thuật toán và biểu thức bằng văn bản. Tuy nhiên, việc giao tiếp giữa mô hình khái niệm với những cái khác trong vòng đời phát triển phần mềm sẽ khó khăn khi mọi người không sử dụng chung một ngôn ngữ cho dự án. Nếu dự án hay tổ chức nào đó đưa ra ngôn ngữ riêng của họ thì người mới tham gia dự án sẽ gặp rất nhiều khó khăn. Hơn nữa, một số vấn đề của hệ thống phần mềm sẽ được hiểu rõ ràng hơn thông qua mô hình thay cho ngôn ngữ lập trình văn bản. Thí dụ, có thể suy luận ra ý nghĩa phân cấp lớp nhưng không thể hiểu thấu đáo nếu bắt đầu trực tiếp bằng mã trình của lớp trong phân cấp. Tương tự, ta không thể hiểu rõ hệ thống trên nền Web nếu không có mô hình. UML giúp xây dựng mô hình để dễ dàng giao tiếp. Một số công việc phù hợp

với mô hình hóa băng văn bản, một số công việc khác lại phù hợp hơn với mô hình hóa bằng đồ họa. UML là ngôn ngữ đồ họa. Với nhiều hệ thống, mô hình trong ngôn ngữ đồ họa dễ hiểu hơn so với ngôn ngữ lập trình. Sau mỗi biểu tượng đồ họa của ngôn ngữ UML là ngữ nghĩa. Vậy, khi xây dựng mô hình trong UML thì người phát triển khác hay các công cụ hỗ trợ mô hình hóa có thể hiểu mô hình một cách rỗ ràng.

2.1.1.3 - UML làn ngôn ngữ đặc tả

Đặc tả là mô tả rõ ràng những điểm mấu chốt của vấn đề. UML cho phép mô tả mô hình chính xác, không nhập nhằng và hoàn thiện. UML hướng tới đặc tả thiết kế, phân tích và quyết dịnh cái đặt trong quá trình phát triển và triển khai hệ thống phần mềm.

2.1.1.4 - UML là ngôn ngữ để xây dựng

UML là không phải là ngôn ngữ lập trình trực quan, nhưng mô hình của nó có thể kết nối trực tiếp tới các ngôn ngữ lập trình khác nhau. Có nghĩa rằng có thể ánh xạ mô hình trong UML tới các ngôn ngữ lập trình khác nhau như Java, C++ hay các bảng CSDL quan hệ, CSDL hướng đối tượng. Ánh xạ này cho khả năng biến đổi thuận từ mô hình UML sang ngôn ngữ lập trình. Đồng thời, cho khả năng biến đổi ngược lại từ cài đặt về mô hình UML; có nghĩa rằng nó cho khả năng làm việc với văn bản hay đồ họa một cách nhất quán.

2.1.1.5 - UML là ngôn ngữ tài liêu

UML hướng tới làm tài liệu kiến trúc hệ thống và các chi tiết của nó. UML cho khả năng biểu diễn yêu cầu, thử nghiệm, mô hình hóa các hoạt động lập kế hoạch và quản lý sản phẩm.

UML cho biết giới hạn của hệ thống và các chức năng chính của nó thông qua UC và tác nhân.

Trong UML, các UC được mô tả bằng biểu đồ logic

Biểu diễn cấu trúc tĩnh của hệ thống nhờ biểu đồ lớp

Mô hình hóa các hành vi đối tượng bằng biểu đồ chuyển trạng thái

Phản ảnh kiến trúc cài đặt vật lý bằng biểu đồ thành phần và biểu đồ triển khai

Mở rộng các chức năng bằng *stereotypes*

2.2 MÔ HÌNH KHÁI NIỆM CỦA UML

Để hiểu được UML ta phải hình dung được mô hình khái niệm của ngôn ngữ. Nó đòi hỏi nắm được ba vấn đề chính, bao gồm các phần tử cơ bản để xây dựng mô hình, quy tắc liên kết các phần tử mô hình và một số cơ chế chung sử dụng cho ngôn ngữ. Khi nắm vững được các vấn đề này thì ta có thể đọc được mô hình UML và tạo ra một vài mô hình cơ bản.

2.2.1 - Phần tử mô hình trong UML

Các khối hình thành mô hình UML gồm ba loại như sau: phần từ, quan hệ và biểu đồ. Phần tử là trừu tượng căn bản trong mô hình, các quan hệ gắn các phần tử này lại với nhau; còn biểu đò nhóm tập hợp các phần tử. Trong UML có bốn loại phần tử mô hình, đó là cấu trúc, hành vi, nhóm và chú thích. Các phần tử này là các khối để xây dựng hướng đối tượng cơ bản của UML.

2.2.1.1 - Phần tử cấu trúc

Phần từ cấu trúc là các danh từ trong mô hình UML. Chúng là bộ phận tĩnh của mô hình để biểu diễn các thành phần khái niệm hay vật lý. Có bảy loại phần tử cấu trúc như mô tả sau đây:

Lớp. Lớp mô tả tập các đối tượng cùng chung thuộc tinh, thao tác, quan hệ và ngữ nghĩa. Một lớp cài đặt một hay nhiều ghép nối. Hình 2.1 biểu diễn đồ họa lớp bằng hình chữ nhật, thông thường chú có tên, thuộc tính và thao tác.

Giao diện. Giao diện là tập hợp các thao tác làm dịch vụ của lớp hay thành phần. Giao diện mô tả hành vi thấy được từ ngoài của thành phần. Giao diện biểu diễn toàn bộ hay một phần hành vi của lớp. Giao diện định nghĩa tập đặc tả thao tác chứ không định nghĩa cài đặt của chúng. Ký pháp đồ họa của nó được mô tả trên hình 2.2. Giao diện thường không đứng một mình mà được gắn vào lớp hay thành phần thực hiện giao diện.



Hình 2.1 Lớp Hình 2.2 Giao diện

Phần tử cộng tác. Phần tử cộng tác là mô tả ngữ cảnh của tương tác. Ký pháp đồ họa của nó là hình elíp với đường vẽ nét đứt, kèm theo tên như trên hình 2.3. Phần tử cộng tác thể hiện một giải pháp thi hành bên trong hệ thống, bao gồm các lớp, quan hệ và tương tác giữa chúng để đạt được một chức năng mong đợi của UC.

Hình 2.3 Cộng tác Hình 2.4 Use case

Trường hợp sử dụng (*Use case*). Mô tả chình tự các hành động mà hệ thống sẽ thực hiện để đạt được một kết quả cho tác nhân nào đó. Tác nhân là những gì bên ngoài tương tác với hệ thống. Tập hợp các UC của hệ thống sẽ hình thành các trường hợp mà hệ thống được sử dụng. Sử dụng UC để cấu trúc các phần tử có tính hành vi trong mô hình. Nó được hiện thực hóa bởi phần tử cộng tác như vừa mô tả trên. Hình 2.4 là ký pháp đồ họa của UC.

Lớp tích cực (*active class*). Lớp tích cực là lớp có đối tượng làm chủ một hay nhiều lớp tiến trình hay luồng. Lớp tích cực được xem như lớp thông thường nhưng đối tượng của nó biểu diễn các thành phần có hành vi đang tương tranh với các thành phần khác. Ký pháp đồ họa của nó

tương tự lớp thông thường nhưng biên chữ nhật được tô đậm. Thông thường chúng cũng có tên, thuộc tính và thao tác.

Thành phần. Thành phần biểu diễn vật lý mã nguồn, các tệp nhị phân trong quá trình phát triển hệ thống. Ký pháp đồ họa của nó đươc biểu diễn trên hình 2.5

Nút (node). Nút là thể hiện thành phần vật lý, tồn tại khi chương trình chạy và biểu diễn các tài nguyên tính toán. Có thể đặt tập các thành phần trên nút chuyển từ nút này sang nút khác. Nút có thể là máy tính, thiết bị phần cứng. Ký pháp đồ họa của chúng được mô tả trên hình 2.6.

Máy chủ

Myfile.cpp

Hình 2.5 Cộng tác Hình 2.6 Use case

2.2.1.2 - Phần tử hành vi

Phần tử hành vi là bộ phận động của mô hình UML. Chúng là các động từ của mô hình, biểu diễn hành vi theo thời gian và không gian. Có hai loại chính là tương tác và trạng thái.

Tương tác. Tương tác là hành vi bao gồm tập các thông điệp trao đổi giữa các đối tượng trong ngữ cảnh cụ thể để thực hiện mục đích cụ thể. Hành vi của nhóm đối tượng hay của mỗi thao tác có thể được chỉ ra bằng tương tác. Biểu diễ đồ họa của thông điệp được thể hiện như trên hình 2.7, bao gồm mũi tên và tên thao tác của nó.

Hiển thịChờ

Hình 2.7 Thông điệp Hình 2.8 Trạng thái

Máy trạng thái. Máy trạng thái là hành vi chỉ ra trật tự các trạng thái mà đối tượng hay tương tác sẽ đi qua để đáp ứng sự kiện. Hành vi của lớp hay cộng tác của lớp có thể được xác định bằng

máy trạng thái. Máy trạng thái kích hoạt nhiều phần tử, bao gồm trạng thái, chuyển tiếp (từ trạng thái này sang trạng thái khác), sự kiện và các hoạt động (đáp ứng sự kiện). Ký pháp đồ họa của trạng thái được thể hiện trên hình 2.8, nó chứa tên và trạng thái con nếu có.

2.2.1.3 - Phần tử nhóm

Phần tử nhóm là bộ phận tổ chức của mô hình UML. Chỉ có một phần tử thuộc nhóm này có tên là gói (*package*). Gói là cơ chế đa năng để tổ chức các phần tử vào nhóm. Các phần tử cấu trúc, hành vi và ngay cả phần tử nhóm có thể cho vào gói. Không giống thành phần (*component*), phần tử nhóm hoàn toàn là khái niệm, có nghĩa rằng chúng chỉ tồn tại vào thời điểm phát triển hệ thống chứ không tồn tại vào thời gian chạy chương trình. Ký pháp đồ họa của nhóm được biểu diễn trên hình 2.9. gói giúp ta quan sát hệ thống ở mức tổng quan hơn.

Chú thích (*annotaitonal*)

Phần tử chú thích là bộ phận chú giải của mô hình UML. Đó là lời giải thích áp dụng để mô tả các phần tử khác trong mô hình. Phần tử chú thích được gọi là ghi chú (*note*). Ký pháp đồ họa của chúng thể hiện trên hình 2.9

Các luật
thương mại

Đây là
chú thích

Hình 2.9 Nhóm và lời ghi chú

2.2.2 - Các quan hệ trong UML

Có bốn loại quan hệ trong UML, bao gồm quan hệ phụ thuộc, kết hợp, khai quát và hiện thực hóa. Chúng là cơ sở để xây dựng mọi quan hệ trong UML.

Phụ thuộc (*dependency*). Phục thuộc là quan hệ ngữ nghĩa hai phần tử trong đó thay đổi phần tử độc lập sẽ tác động đến ngữ nghĩa của phần tử phục thuộc. Ký pháp đồ họa của nó được thể hiện trên hình 2.10.

0..1 .
Ông chủ Nhân viên

Hình 2.10 Cộng tác Hình 2.11 Use case

Kết hợp (*association*). Kết hợp là quan hệ cấu trúc để mô tả tập liên kết (một liên kết là kết nối giữa các đối tượng). Khi đối tượng của lớp này gửi/nhận thông điệp đến/từ đối tượng của lớp kia thì ta gọi chúng là có quan hệ kết hợp. Ký pháp đồ họa của kết hợp được mô tả trên hình 2.11 chúng có thể chứa tên nhiệm vụ và tính nhiều (*multiplicity*). Tụ hợp (*aggregation*) là dạng đặc biệt của kết hợp, nó biểu diễn quan hệ cấu trúc giữa toàn thể và bộ phận. Ký pháp đồ họa của nó trên hình 2.12. một dạng dặc biệt của tập hợp là quan hệ hợp thành (*composition*), trong đó nếu như đối tượng toàn thể bị hủy bỏ thì các đối tượng bộ phận của nó cũng bị hủy bỏ theo. Biểu diễn đồ họa của tập hợp như trên hình 2.13.

Hình 2.12 Tụ hợp Hình 2.13 Use case

Khái quát hóa (*generalization*). Khái quát hóa là quan hệ đặc biệt hóa/ khái quát hóa mà trong đó đối tượng cụ thể sẽ kế thừa các thuộc tính và phương pháp của đối tượng tổng quát. Ký pháp đồ họa của khái quát hóa được mô tả trên hình 2.14.

Hình 2.14 Khái quát hóa Hình 2.4 Hiện thực

Hiện thực hóa (*realization*). Hiện thực hóa là quan hệ ngữ nghĩa giữa giao diện và lớp (hay thành phần) hiện thực lớp; giữa UC và hợp tác hiện thực UC. Biểu diễn đồ họa của nó dược mô tả trên hình 2.15.

2.2.3 - Kiểu dữ liệu

Kiểu dữ liệu không phải là phần tử mô hình trong UML. Kiểu dữ liệu cơ sở là kiểu dữ liệu không có cấu trúc. UML có các kiểu dữ liệu sau:

Boolean: là kiểu đếm với hai giá trị True và False

Biểu thức (Expression): là xâu ký tự có cú pháp

Tính nhiều (Multiplicity): là tập không rỗng của các số nguyên dương và ký tự * (để biểu thị tính nhiều vô hạn).

Tên (Name): là xâu ký tự cho khả năng đặc tả phần tử.

Số nguyên (Integer): là kiểu cơ bản và là phần tử của tập vô hạn các sô nguyên âm và dương.

Xâu (String): là trật tự cảu các ký tự, được sử dụng là tên.

Thời gian (Time): xâu ký tự biểu dirn giá trị tuyệt đối hay khoảng tương tương đối.

Không lý giải (Uninterpreted): là 'cái gì đó' mà ý nghĩa của nó phụ thuộc và lĩnh vực.

2.2.4 - Biểu đồ UML

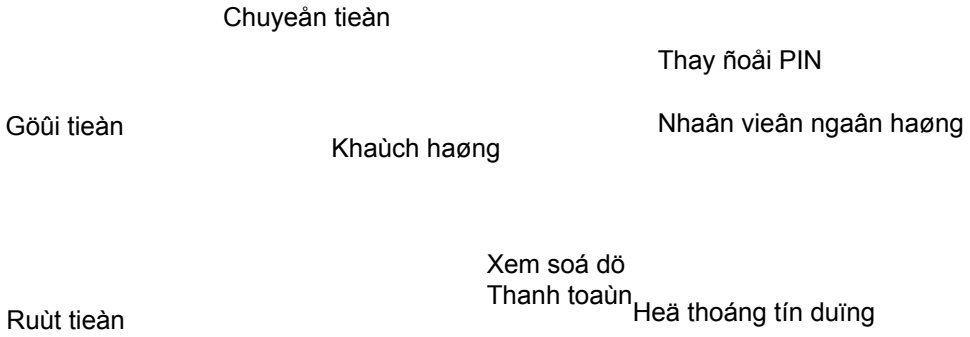
Biểu đồ là biểu diễn đồ họa tập hợp các phần tử mô hình. Vẽ biểu đồ để biểu diễn hệ thống đang xây dựng dưới các góc độ quan sát khác nhau. Có thể hiểu biểu đồ là ánh xạ của hệ thống. Một phần tử có thể xuất hiện trong một hay nhiều biểu đồ. Về lý thuyết thì biểu đồ có thể bao gồm tổ hợp vô số phần từ đồ họa và quan hệ vừa mô tả trên. UML cho khả năng xây dựng một vài kiểu biểu đồ trực quan để biểu diễn các khía cạnh khác nhau của hệ thống, bao gồm biểu đồ trường hợp sử dụng (*use case diagram*), biểu đồ trình tự (*sequence diagram*), biểu đồ cộng tác (*collaboration diagram*), biểu đồ lớp (*class diagram*), biểu đồ biến đổi trạng thái (*state transition diagram*), biểu đồ thành phần (*component diagram*) và biểu đồ triển khai (*deployment diagram*). Các loại biểu đồ nay sẽ được giới thiệu tóm tắt dưới đây thông qua ví dụ về xây dựng phần mềm cho hệ thống máy rút tiền tự động (*Automated Teller Machine – ATM*). Giả sử rằng ta có các máy rút tiền tự động ATM đặt tại các đường phố khác nhau trong thành phố. Chúng được nối với máy tính trung tâm tại trụ sở ngân hàng thông qua hệ thống mạng máy tính. Máy tính trung tâm lưu trữ và quản trị CSDL khách hàng, xử lý một số công việc chuyên ngành và yêu cầu ATM trả tiền. Máy rút tiền tự động bao gồm máy đọc thẻ từ, màn hình và bàn phím để tương tác với người sử dụng.

2.2.4.1 - Biểu đồ trường hợp sử dụng (Use case – UC)

Biểu đồ này chỉ ra tương tác giữa các UC và tác nhân. UC biểu diễn các chức năng hệ thống. Tác nhân là con người hay hệ thống khác cung cấp hay thu nhận thông tin từ hệ thống đang được xây dựng. Biểu đồ UC tập trung vào quan sát trạng thái tĩnh của các UC trong hệ thống. Vì UC biểu diễn yêu cầu hệ thống từ góc độ người dùng, cho nên UC là chức năng mà hệ thống phải có. Biểu đồ loại này chi ra tác nhân nào khởi động UC và khi nào tác nhân nhận thông tin từ hệ thống.

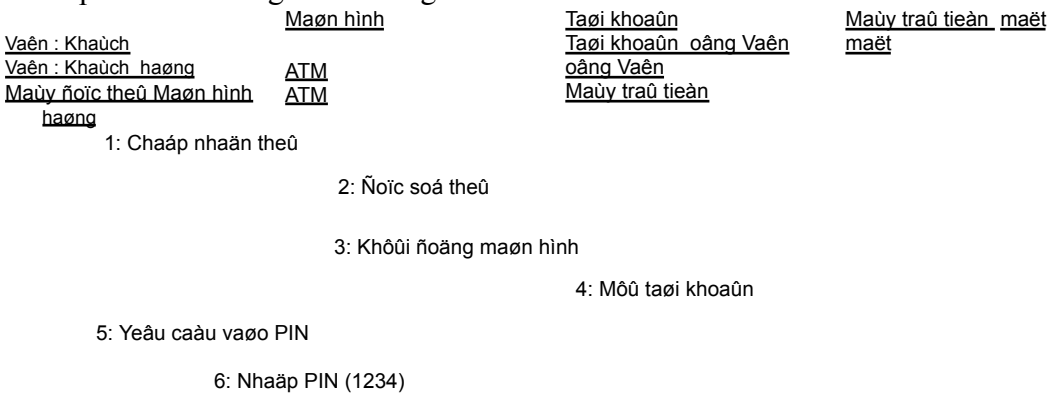
Biểu đồ trên hình 2.16 chỉ ra tương tác giữa các UC và tác nhân của hệ thống rút tiền tự động ATM. *Khách hàng* (là tác nhân) có khả năng khởi động một số UC như *Rút tiền*, *Gửi tiền*, *Chuyển*

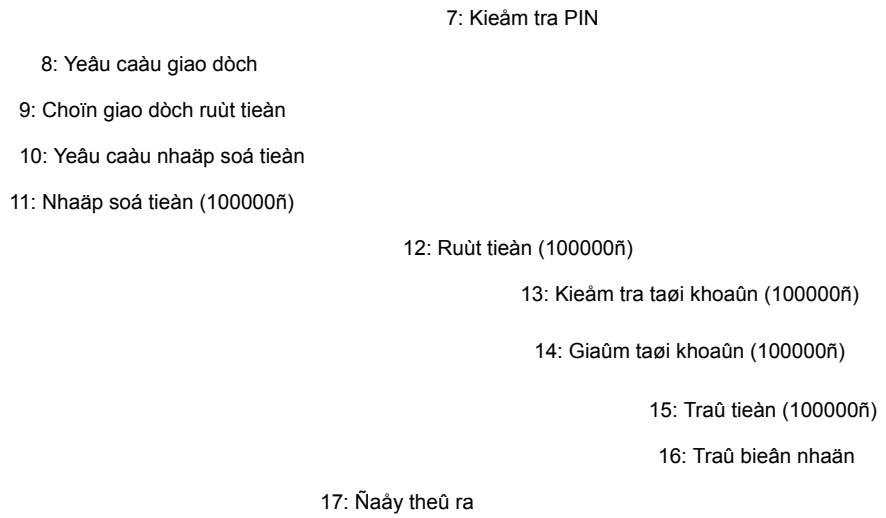
tiền, *Xem số dư tài khoản*, *Thay đổi số căn cước cá nhân (PIN)* và *Thanh toán*. *Nhân viên ngân hàng* (là tác nhân khác) có khả năng khởi động UC với *Thay đổi số căn cước cá nhân*. Trường hợp sử dụng *Thanh toán* có mũi tên đi đến *Hệ thống tín dụng*. Hệ thống ngoài có thể là tác nhân, thí dụ *Hệ thống tín dụng* là hệ thống ở bên ngoài hệ thống ATM, do vậy nó là tác nhân. Mũi tên đi từ UC đến tác nhân cho biết UC phát sinh thông tin để tác nhân sử dụng (UC trả lại thông tin cho tác nhân).



Hình 2.16 Biểu đồ UC của ATM

Biểu đồ UC chỉ ra chức năng tổng thể của hệ thống đang phát triển. Khách hàng, quản lý dự án, phân tích viên, lập trình viên, kỹ sư kiểm tra chất lượng và những ai quan tâm tổng thể đến hệ thống đều có thể biết được hệ thống sẽ hỗ trợ gì thông qua biểu đồ này.





Hình 2.17 Biểu đồ trình tự của hệ thông ATM

2.2.4.2 - Biểu đồ trình tự (sequence)

Biểu đồ trình tự chỉ ra luồng chức năng xuyên qua các UC, nó là biểu đồ mô tả tương tác giữa các đối tượng và tập trung vào mô tả trật tự các thông điệp theo thời gian. Thí dụ trường hợp sử dụng *Rút tiền* có thể có nhiều trình tự như khách hàng yêu cầu rút tiền khi đã hết tiền trong tài khoản, khi khách hàng nhập nhầm PIN hay trường hợp hoạt động bình thường. Hình 2.17 là thí dụ kịch bản bình thường (nhập dúng PIN, số tiền trong tài khoản đủ số lượng yêu cầu rút). Tác nhân tác động UC này được hiển thị trên đỉnh biểu đồ, thí dụ tác nhân khách hàng trên hình 2.16.

Các đối tượng mà hệ thống cần để thực hiện UC *Rút tiền* cũng được đặt trên đỉnh biểu đồ. Mỗi mũi tên trong biểu đồ thể hiện thông điệp truyền giữa tác nhân và đối tượng hay đối tượng với đối tượng để thực hiện chức năng cụ thể. Chú ý rằng biểu đồ trình tự hiển thị phần tử mô hình đối tượng chứ không phải lớp. Thí dụ, biểu đồ này hiển thị tên khách hàng cụ thể (ông Văn) thay cho hiển thị lớp *Khách hàng*.

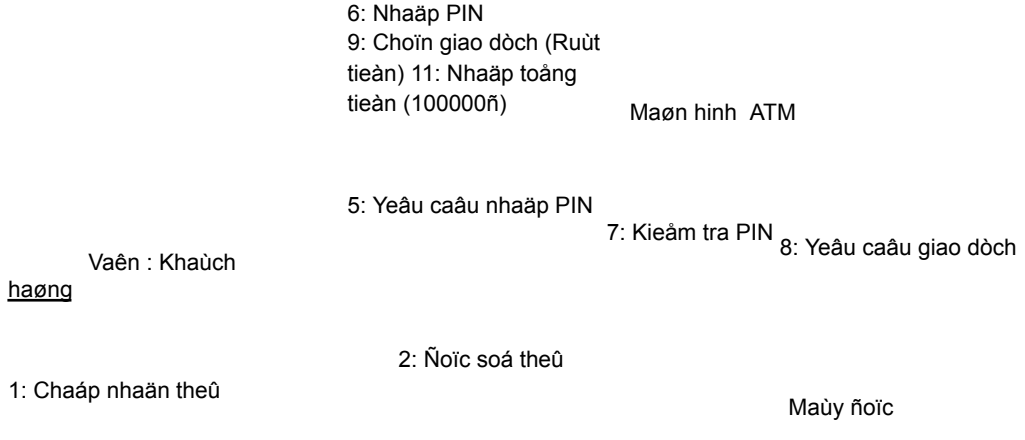
UC rút tiền bắt đầu khi khách hàng bỏ thẻ tín dụng vào máy đọc thẻ. Máy đọc thẻ là đối tượng được chỉ ra trong hình chữ nhật trên đỉnh biểu đồ. Sau đó máy đọc thẻ sẽ đọc số thẻ tín dụng, mở đối tượng tài khoản của ông Văn và khởi động màn hình của hệ thống ATM. Màn hình hiển thị dấu nhắc để nhập số hiệu khách hàng (PIN). Thí dụ, khách hàng nhập số PIN 1234. Màn hình kiểm tra PIN với đối tượng tài khoản và thấy phù hợp. Màn hình hiển thị các chức năng để ông Văn lựa chọn. Ông ta chọn chức năng *Rút tiền*. Màn hình hiển thị dấu nhắc để ông Văn nhập số tiền sẽ rút. Ông ta nhập số tiền rút 100000đ. Màn hình thực hiện rút tiền từ tài khoản có đủ 100000đ? Giảm số dư trong tài khoản đi 100000đ. Yêu cầu máy trả tiền chi trả 100000đ tiền mặt và in ra biên nhân. Cuối cùng yêu cầu máy đọc thẻ trả lại thẻ tín dụng cho ông Văn.

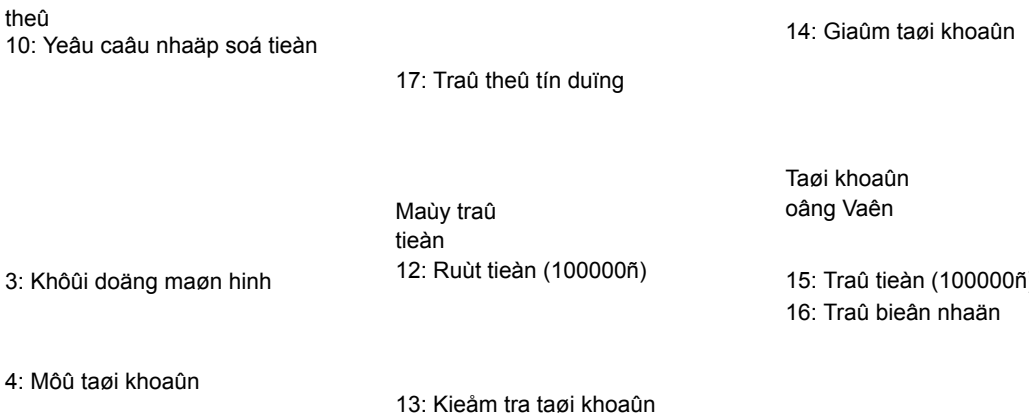
Biểu đồ trình tự trên đây mô tả toàn bộ luồng sử lý cho UC rút tiền thông qua thí dụ trường hợp ông Văn rút 100000đ. Khách hàng có thể thấy được tiến trình tác nghiệp cụ thể của họ thông qua biểu đồ. Phân tích viên thấy được luồng tiến trình, người phát triển thấy được các đối tượng

cần xây dựng và các thao tác cho các đối tượng này, kỹ sư kiểm tra chất lượng có thể thấy chi tiết của tiến trình để xây dựng qui trình thử nghiệm, kiểm tra. Biểu đồ trình tự có ích cho mọi người tham gia dự án.

2.2.4.3 - Biểu đồ cộng tác (Collabaration)

Biểu đồ cộng tác chỉ ra các thông tin như biểu đồ trình tự theo cách khác, nó tập trung vào tổ chức cấu trúc của các đối tượng gửi và nhận thông điệp. Hình 2.18 là thí dụ biểu đồ cộng tác của biểu đồ trình tự tương ứng mô tả trên hình 2.17. Biểu đồ cộng tác và biểu đồ trình tự thuộc loại biểu đồ tương tác và chúng có thể biến đổi qua lại. Trong biểu đồ cộng tác, đối tượng đặt trong chữ nhật, tác nhân là người hình cây như trong biểu đồ trình tự. Trong khi biểu đồ trình tự biểu diễn tương tác đối tượng và tác nhân theo thời gian thì biểu đồ cộng tác lại không quan tâm đến thời gian. Thí dụ trên hình 2.18 cho thấy máy đọc thẻ ra lệnh mở tài khoản ông Văn, đối tượng tài khoản của ông Văn ra lệnh máy đọc trả lại thẻ tín dụng. Các đối tượng giao tiếp trực tiếp với nhau được thể hiện bằng đường nối.



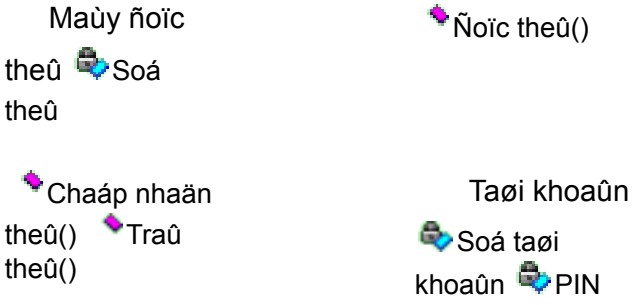


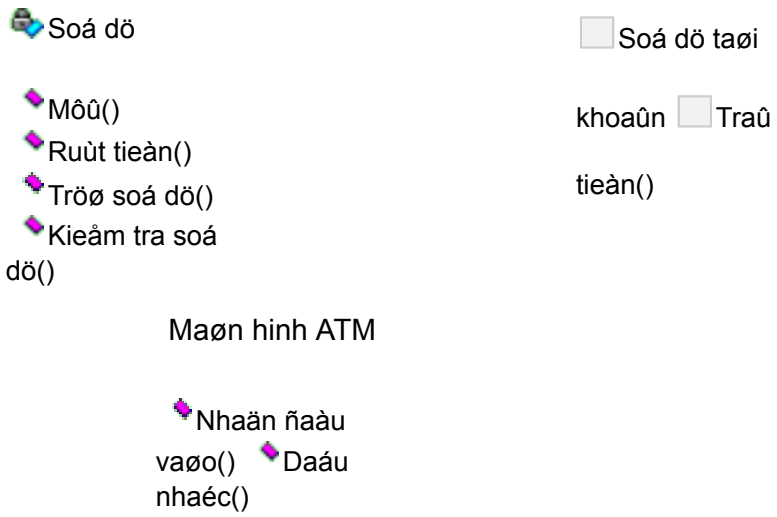
Hình 2.18 Sơ đồ cộng tác ông Văn rút 100000đ

Tuy rằng biểu đồ cộng tác cùng chỉ ra các thông tin như biểu đồ trình tự, nhưng biểu đồ cộng tác được sử dụng vì lý do khác. Kỹ sư kiểm tra chất lượng và kiến trúc sư hệ thống thấy được việc phân bổ tiến trình giữa các đối tượng thông qua biểu đồ loại này. Thí dụ, nếu biểu đồ cộng tác có hình dạng như ngôi sao, với nhiều đối tượng giao tiếp với đối tượng trung tâm thì kiến trúc sư có thể kết luận rằng hệ thống quá phục thuộc vào một đối tượng và họ sẽ đi thiết kế lại phân bổ tiến trình để nâng cao hiệu suất hệ thống.

2.2.4.4 - Biểu đồ lớp (class)

Biểu đồ lớp chỉ ra tương tác giữa các lớp trong hệ thống. Các lớp được xem như kế hoạch chi tiết của các đối tượng. Tài khoản ông Văn là đối tượng; *Tài khoản* là lớp. Lớp *Tài khoản* chứa *PIN* khách hàng và hành vi *Kiểm tra PIN*. Mỗi lớp trong biểu đồ lớp được tạo ra cho mỗi loại đối tượng trong biểu đồ trình tự và cộng tác. Thí dụ biểu đồ lớp cho UC *Rút tiền* được mô tả trên hình 2.19





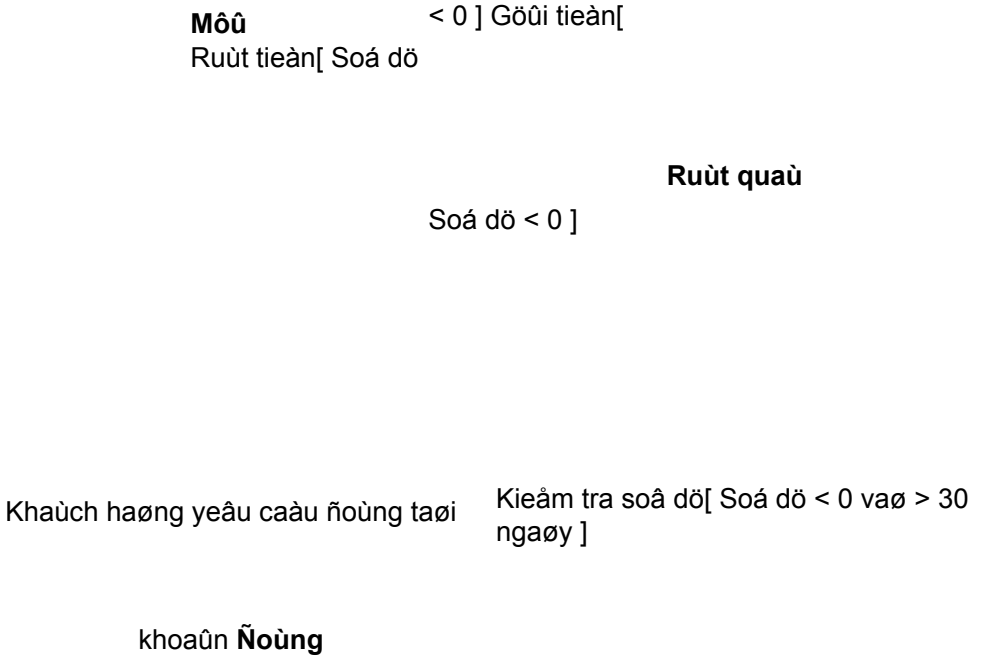
Hình 2.19 Biểu đồ lớp của UC Rút tiền

Biểu đồ lớp trên hình 2.19 chỉ ra quan hệ giữa các lớp hình thành nên UC *Rút tiền*. Biểu đồ này bao gồm bốn lớp, đó là *Máy đọc thẻ, Tài khoản, Màn hình ATM*, và *Máy trả tiền*. Mỗi lớp trong biểu đồ được biểu diễn bằng hình chữ nhật chia làm ba phần: tên lớp (thí dụ tên lớp *Tài khoản*), thuộc tính (thí dụ lớp *Tài khoản* chứa ba thuộc tính: *Số tài khoản, Số căn cước cá nhân – PIN* và *Cân đối tài khoản*) và thao tác (thí dụ lớp *Account* có bốn thao tác: *Mở tài khoản, Rút tiền, Trừ tiền trong tài khoản* và *Kiểm tra số tiền trong tài khoản*. Đường nối giữa các phần tử biểu đồ lớp là quan hệ giao tiếp giữa chúng. Phía trái của một số thuộc tính và thao tác có gắn biểu tượng khóa; có nghĩa rằng đó là các thuộc tính và thao tác riêng.

Người phát triển sử dụng biểu đồ lớp để xây dựng các lớp. Các công cụ phần mềm như *Rose* phát sinh mã trình xương sống cho các lớp, sau đó người phát triển phải chi tiết hóa nó bằng ngôn ngữ lập trình. Kiến trúc sư quan sát thiết kế hệ thống thông qua biểu đồ lớp. Nếu trên biểu đồ thấy một lớp chứa quá nhiều chức năng thì phải chia chúng ra nhiều lớp khác nhau. Kiến trúc sư cũng dễ phát hiện ra nếu giữa các lớp thiếu giao tiếp.

2.2.4.5 - Biểu đồ chuyển trạng thái (state transition)

Biểu đồ chuyển trạng thái mô tả vòng đời của đối tượng, từ khi nó được sinh ra đến khi bi phá hủy. Biểu đồ chuyển trạng thái cung cấp cách thức mô hình hóa các trạng thái khác nhau của đối tượng. Trong khi biểu đồ lớp cung cấp bức tranh tĩnh về các lớp và quan hệ của chúng thì biểu đồ chuyển trạng thái được sử dụng để mô hình hóa các hành vi động của hệ thống.



Hình 2.20 Biểu đồ biến đổi trạng thái của lớp tài khoản

Biểu đồ chuyển trạng thái chỉ ra hành vi của đối tượng. Thí dụ, đối tượng *Tài khoản* trong ngân hàng có thể ở trong một vài trạng thái như mở, đóng hay rút quá mức. Tài khoản sẽ ứng xử khác nhau với mỗi trạng thái khác nhau. Hình 2.20 là thí dụ biểu đồ chuyển trạng thái của lớp Tài khoản.

Biểu đồ trên cho thấy các trạng thái và quá trình chuyển trạng thái của Tài khoản. Thí dụ, khi *Tài khoản* đang mở và *Khách hàng* yêu cầu đóng *Tài khoản* thì nó chuyển sang trạng thái đóng. Yêu cầu của *Khách hàng* được gọi là sự kiện. Sự kiện là cái gây ra biến đổi từ trạng thái này sang trạng thái khác. Nếu *Tài khoản* mở và khách hàng rút tiền thì có thể dẫn tới trạng thái rút quá. Trạng thái này xảy ra khi khách hàng con nợ ngân hàng hay *Tài khoản* < 0. Điều kiện này có thể được nêu trên biểu đồ trong đấu ngoặc vuông và được gọi là *điều kiện giác*. Điều kiện gác điều khiển việc xảy ra hay không xảy ra biến đổi trạng thái. Biểu đồ biến đổi trạng thái có hai trạng thái đặc biệt, đó là *Khởi đầu (Start)* và *Dừng (Stop)*. Trên biểu đồ trạng thái chỉ có duy nhất một trạng thái *Start* và có thể có nhiều hay không có trạng thái *Dừng*. Các tiến trình xảy ra khi đối

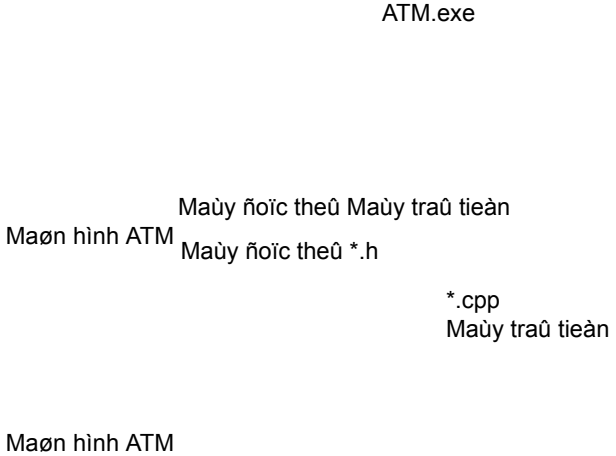
tượng đang trong trạng thái nào đó thì được gọi là *hành động (action)*. Thí dụ, khi *Tài khoản* bị rút quá thì thông báo được gửi tới khách hàng; việc gửi thông báo là hành động.

Thông thường, không tạo lập biểu đồ chuyển trạng thái cho mọi lớp mà chỉ sử dụng cho các lớp phức tạp. Nếu đối tượng của lớp tồn tại trong nhiều trạng thái và có ứng xử khác nhau trong mỗi trạng thái thì nên xây dựng biểu đồ chuyển trạng thái cho chúng. Nhiều dự án không quan tâm đến loại biểu đồ này. Biểu đồ chuyển trạng thái chỉ dành cho việc làm tài liệu *Rose* không phát sinh mã trình từ biểu đồ này.

2.2.4.6 - Biểu đồ thành phần (component)

Biểu đồ thành phần cho ta cái nhìn vật lý của mô hình. Biểu đồ thành phần cho ta thấy được các thành phần phần mềm trong hệ thống và quan hệ giữa chúng. Hai loại thành phần trong biểu đồ, đó là thành phần khả thực và thành phần thư viện. Trong *Rose*, mỗi lớp mô hình được ánh xạ đến một thành phần mã nguồn.

Hình 2.21 mô tả biểu đồ thành phần của hệ thông ATM. Biểu đồ trên hình 2.21 là các thành phần phía máy trạm trong hệ thống ATM. Nếu quyết định cài đặt hệ thống bằng ngôn ngữ C++ thì trong mỗi lớp sẽ có tiệp .cpp và tiệp .h riêng biệt, vậy mỗi lớp được ánh xạ đến hai thành phần riêng trong biểu đồ. Thí dụ, lớp Màn hình ATM ánh xạ đến hai thành phần để thể hiện tiệp *.h (header)* và tiệp *.cpp (thân lớp)* của lớp. Thành phần tô đen là đặc tả cảu gói (*package specification*); biểu diễn tệp *cpp* của lớp *Màn hình ATM*. Thành phần không tô đen cũng được gọi là đặc tả gói, biểu diễn tiệp *.h* của lớp. Thành phân *ATM.exe* trên biểu dồ là đặc tả nhiệm vụ và biểu diễn luồng (*thread*) xử lý. Các thành phần nối với nhau bằng đường gạch – gạch để cho biết chúng có quan hệ phụ thuộc. Thí dụ, lớp *Máy đọc thẻ* phụ thuộc vào lớp *Màn hình ATM*. Có nghĩa rằng phải có lớp *Màn hình ATM* thì lớp *Máy đọc thẻ* mới dịch được. Khi toàn bộ các lớp dịch được thì ta mới có thành phần *ATMClient.exe*.



Hình 2.21 Biểu đồ thành phần của ATM client

Thí dụ máy rút tiền tự động ATM có hai luồng xử lý cho nên chúng có hai trình khả thực. Một trình là *Máy trạm ATM*, bao gồm các thành phần *Máy trả tiền, Máy đọc thẻ* và *Màn hình ATM*. Trình thứ hai là *Máy chủ ATM*, nó chỉ có thành phần *Tài khoản*. Biểu đồ thành phần của *Máy chủ ATM* trên hình 2.22.

Taøi khoaûn Taøi khoaûn

Hình 2.22 Biểu đồ thành phần của máy chủ ATM

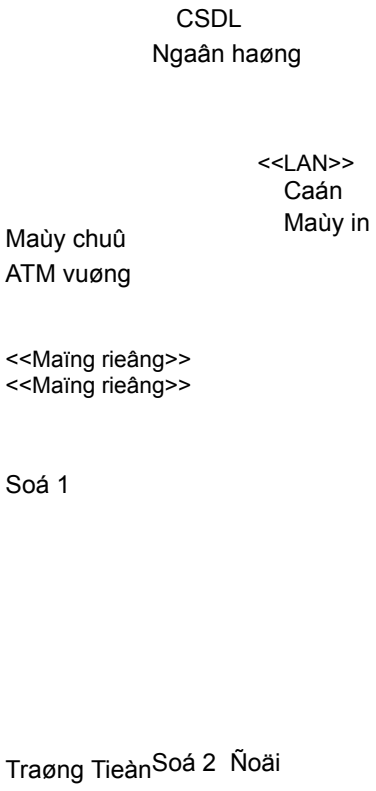
Có thể có nhiều biểu đồ thành phần cho một hệ thống, số lượng này phụ thuộc vào các hệ thống con của chúng. Mỗi hệ thống con là gói thành phần. Tổng quát, gói là tập hợp các đối tượng. Thí dụ này có hai gói, đó là gói *Máy trạm ATM* và gói *Máy chủ ATM*.

Bất kỳ ai có trách nhiệm dịch chương trình đều quan tâm đến biểu đồ loại này. Biểu đồ cho ta thấy trình tự dịch của các mođun trong hệ thống. Đồng thời nó cũng cho biết rõ thành phần nào được tạo ra khi chạy chương trình. Biểu đồ thành phần chỉ ra ánh xạ của lớp và các thành phần cài đặt.

2.2.4.7 - Biểu đồ triển khai (deployment)

Biểu đồ triển khai chỉ ra bố trí vật lý của mạng và các thành phần hệ thống sẽ đặt ở đâu. Trong thí dụ hệ thống ATM thì ATM bao gồm nhiều hệ thống con chạy tách biệt trên các thiết bị vật lý khác nhau hay còn gọi là nút. Biểu đồ triển khai của hệ ATM được thể hiện trên hình 2.23. Biểu đồ trên hình này cho thấy *Máy trạm ATM* sẽ chạy trên nhiều địa điểm khác nhau. Chúng

giao tiếp với *Máy chủ ATM* qua mạng riêng. *Máy chủ ATM* sẽ giao tiếp với các *Máy chủ CSDL* thông qua mạng LAN. Như vậy, hệ thống ATM này có kiến trúc ba tầng: một tầng là CSDL, một tầng là máy chủ, tầng còn lại là máy trạm.



Hình 2.23 Biểu đồ triển khai của hệ thống ATM

Thông qua biểu đồ triển khai mà người quản lý dự án, người sử dụng, kiến trúc sư và đội ngũ triển khai hiểu phân bổ vật lý của hệ thống và các hệ thống con sẽ được đặt ở đâu.

2.3 KIẾN TRÚC HỆ THỐNG

Kiến trúc là trừu tượng hóa các khía cạnh quan trọng nhất của hệ thống. Nó cung cấp khung trong đó thiết kế sẽ được xây dựng. Nó mô tả tầm cỡ, sức mạnh của hệ thống, thu thập các UC quan trọng nhất và các yêu cầu ứng dụng. Nó thể hiện phần mềm sẽ được tổ chức như thế nào và cung cấp các giao thức trao đổi dữ liệu và giao tiếp giữa các mođun. Việc hiển thị, đặc tả, xây dựng và làm tài liệu hệ thống phần mềm đòi hỏi hệ thống phải được xem xét từ nhiều khía cạnh khác nhau. Người sử dụng khác nhau (người sử dụng cuối cùng, nhà phân tích, người phát triển, người tích hợp hệ thống, kiểm tra viên, người quản lý dự án …) có cách quan sát hệ thống khác

nhau vào các thời điểm khác nhau. Kiến trúc hệ thống là *vật phẩm* quan trọng nhất, được sử dụng để quản lý các điểm nhìn khác nhau để điều khiển phát triển hệ thống tăng dần và lặp trong suốt chu kỳ sống. Kiến trúc là tập các quyết định về:

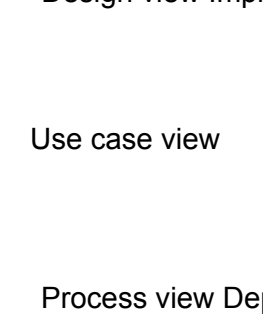
Tổ chức của hệ thống phần mềm

Lựa chọn các phần tử cấu trúc và giao diện cho hệ thống

Hành vi của chúng thể hiện trong hợp tác giữa các phần tử

Tổ hợp các phần tử cấu trúc và hành vị vaò hệ thống con lớn hơn

Kiến trúc phần mềm không chỉ liên quan đến cấu trúc và hành vi mà cả chức năng, tính sử dụng lại, dễ hiểu, ràng buộc công nghệ …



Hình 2.24 Mô hình hóa kiến trúc hệ thống

Kiến trúc hệ thống phần mềm được mô tả bằng các khung nhìn. Các khung nhìn ánh xạ vào tổ chức và cấu trúc hệ thống, mỗi khung nhìn tập trung vào khía cạnh cụ thể của hệ thống (hình 2.24).

Theo biểu đồ của *Philippe Krutchen* [LIBE98] ta có năm khung nhìn như sau: khung nhìn trường hợp sử dụng (*Use case view*), khung nhìn logic (*Logical view*), khung nhìn cài đặt (*Implementation view*), khung nhìn triển khai (*Deployment view*) và khung nhìn tiến trình (*Process view*). Mỗi khung nhìn phục vụ mục đích riêng.

2.3.1 - Khung nhìn UC

Khung nhìn này đứng trước mọi khung nhìn khác (hình 2.24). Nó được hình thành từ giai đoạn phân tích yêu cầu và được sử dụng để điều khiển và thúc đẩy phần việc còn lại của thiết kế. Nó mô tả các hành vi hệ thống theo cách nhìn của khách hàng, phân tích viên và kỹ sư kiểm tra, thử nghiệm. Khung nhìn UC chứa các tác nhân, UC, biểu đồ UC (khía cạnh tĩnh của khung nhìn) trong hệ thống. Chúng cũng có thể bao gồm vài biểu đồ trình tự, biểu đồ cộng tác (khía cạnh động của khung nhìn) và gói. Khung nhìn UC tập trung vào mức độ cao của cái hệ thông sẽ làm, không quan tâm đến hệ thống làm như thế nào. Chúng không xác định tổ chức của hệ thống.

Khi dự án bắt đầu, khách hàng, phân tích viên và người quản lý dự án làm việc với UC, biểu đồ UC và tài liệu UC để thống nhất về hệ thống. Một khi khách hàng đã nhất trí về các UC và tác nhân thì họ sẽ nhất trí về phạm vi hệ thống. Hệ thống được tiếp tục phát triển băng khung nhìn logic.

2.3.2 - Khung nhìn thiết kế

Rose gọi khung nhìn này là khung nhìn logic (*logical view*). Khung nhìn logic biểu diễn tổ chức của các lớp có ý nghĩa nhất và các quan hệ của chúng với nhau. Khung nhìn logic tập trung vào hệ thống cài đặt hành vi trong UC như thế nào. Nó bao gồm các lớp, biểu đồ lớp, biểu đồ đối tượng (khía cạnh tĩnh của khung nhìn), biểu đồ tương tác, biểu đồ biến đổi trạng thái (khía cạnh động của khung nhìn) và các gói. Hầu hết mọi người trong dự án đều quan tâm đến khung nhìn logic.

Thông thường đội ngũ phát triển phần mềm tiệm cận khung nhìn logic theo hai bước. Bước thứ nhất là nhận ra các lớp phân tích (*analysis class*). Các lớp này độc lập với ngôn ngữ. Trong UML các lớp này được biểu diễn bằng các biểu tượng sau:

Lớp phân tích có thể xuất hiện cả ở trong biểu đồ tương tác của khung nhìn UC. Một khi đã nhận ra các lớp phân tích thì đội ngũ phát triển phần mềm chuyển chúng sang lớp thiết kế (*design class*). Đó là những lớp phụ thuộc ngôn ngữ.

Khung nhìn logic tập trung vào cấu trúc logic của hệ thống. Trong khung nhìn này ta sẽ nhận ra các *bộ phận* hệ thống, khảo sát thông tin và hành vi, khảo sát quan hệ giữa các bộ phận. Cần cẩn thận khi gán thông tin và hành vi cho lớp, nhóm các lớp, khảo sát quan hệ giữa các lớp và gói để đảm bảo khả năng sử dụng lại.

2.3.3 - Khung nhìn cài đặt

Rose gọi khung nhìn này là khung nhìn thành phần (*component view*). Thành phần là mođun vật lý hay tiệp mã trình để lắp ráp thành hệ thống vật lý. Khung nhìn thành phần bao gồm: thành phần, biểu đồ thành phần và gói.

Người quan tâm nhất đến khung nhìn thành phần là người có trách nhiệm quản lý mã trình, dích chương trình và triển khai ứng dụng. Một vài thành phần là thư viện, một số khác là mã trình khả thực (*exe*) và thư viện (*dll*).

2.3.4 - Khung nhìn triển khai

Khung nhìn này tập trung vào phân bổ vật lý của tài nguyên và phân bổ nhiệm vụ giữa các tài nguyên. Khung nhìn triển khai liên quan đến triển khai vật lý của hệ thống, khác với kiến trục logic. Thí dụ, hệ thống có kiến trúc ba tầng logic, bao gồm giao diện, logic và tác nghiệp và logic CSDL tách biệt nhau. Nhưng triển khai có thể chỉ có hai tầng, trong đó logic tác nghiệp và logic CSDL trên cùng một máy. Khung nhìn triển khai bao gồm tiến trình (luông thực hiện trong vùng nhớ riêng), bộ xử lý và thiết bị.

Khung nhìn triển khai chỉ ra các tiến trình và thiết bị trên mạng và các kết nối vật lý giữa chúng. Biểu đồ triển khai cũng hiển thị tiến trình và chỉ ra tiến trình nào chạy trên máy nào.

2.3.5 - Khung nhìn tiến trình

Khung nhìn tiến trình biểu diễn phân tách các luồng thực hiện chương trình (tiến trình – *process*, luồng – *thread*, nhiệm vụ – *task*,…), đồng bộ giữa các luồng, phân bổ các đối tượng và

lớp cho các luồng thực hiện khác nhau. Khung nhìn tiến trình tập trung vào các nhiệm vụ tương tranh tương tác với nhau như thế nào trong hệ thông đa nhiệm. Trong biểu đồ cảu phần mềm công cụ *Rose 2000* không có khung này.

2.3.6 - Cần bao nhiêu khung nhìn

Không phải tất cả các hệ thống đều đòi hỏi đầy đủ các khung nhìn mô tả trên. Hệ thống trên máy riêng lẻ có thể bỏ khung nhìn triển khai, nếu hệ đơn xử lý thì bỏ khung nhìn tiến trình, nếu chương trình nhỏ thì bỏ khung nhìn cài đặt.

2.4 RATIONAL ROSE LÀ GÌ?

Rational Rose là phần mềm công cụ mạnh hỗ trợ phân tích, thiết kế hệ thống phần mềm theo hướng đối tượng. Nó giúp ta mô hình hóa hệ thống trước khi viết mã trình, nó đảm bảo tính đúng đắn, hợp lý của kiến trúc hệ thống từ khi khởi đầu dự án. Mô hình *Rose* là bức tranh hệ thống, nó bao gồm toàn bộ biểu đồ UML, tác nhân, trường hợp sử dụng, đối tượng, lớp, thành phần và các nút triển khai trong hệ thống. Nó mô tả chi tiết hệ thống bao gồm cài gì và chúng làm việc ra sao để người phát triển hệ thống có thể sử dụng mô hình như kế hoạch chi tiết cho việc xây dựng hệ thống. Rose hỗ trợ giải quyết vấn đề muôn thủa là đội ngũ dự án giao tiếp với khách hàng và làm tài liệu yêu cầu.

Theo phong cách lập trình truyền thống thì sau khi đã xác định yêu cầu hệ thống, người phát triển sẽ lấy một vài yêu cầu, quyết định thiết kế và viết mã trình. Một số người phát triển khác cũng làm như vậy với yêu cầu khác, thiết kế khác. Cách làm này dẫn tời nhiều khó khăn cho ai muốn hiểu và quản trị toàn bộ hệ thống, họ khó thấy được quyết định thiết kế đã được làm trước đó. Nếu không có tài liệu thiết kế thì khó đảm bảo rằng hệ thống được xây dựng đúng là hệ thống mà người sử dụng nghĩ tới. Tuy rằng các yêu cầu được làm tài liệu đầy đủ, nhưng thiết kế chỉ tồn tại trong đầu người phát triển nào đó, người khác sẽ không có ý tưởng gì về cấu trúc hệ thống. Nếu người phát triển chuyển đi nơi khác thì dự án sẽ gặp khó khăn. Phong cách khác phát triển hệ thống là sau khi xác định yêu cầu, các thiết kế phải được làm tài liệu chi tiết. Mọi người tham gia phát triển cùng trao đổi quyết định thiết kế trước khi viết mã trình. Do vậy, dự án không còn phải lo lắng khi ai đó rời bỏ dự án. Ngoài người phát triển hệ thông quan tâm đến mô hình, ta còn thấy mọi thành viên khác của dự án đều có thể thu nhận các thông tin cần thiết từ mô hình.

Khách hàng và quản lý dự án sử dụng các biểu đồ UC để có cài nhìn bao quát về hệ thống và thống nhất với nhau về phạm vi dự án.

Quản lý dự án sử dụng biểu đồ UC và tài liệu để chia nhỏ dự án thành tiểu dự án có thể quản lý được.

Thông qua tài liệu UC, các phân tích viên và khách hàng thấy được các chức năng hệ thống sẽ cung cấp

Các phân tích viên và người phát triển, thông qua các biểu đồ trình tự và biểu đồ công tác, thấy được logic mà hệ thống tuân thủ, các đối tượng trong hệ thống và các thông điệp giữa các đối tượng.

Đội ngũ kiểm tra chất lượng thu thập thông tin thông qua tài liệu UC và các biểu đồ tương tác để viết mô tả kiểm tra hệ thống.

Người phát triển sử dụng biểu đồ lớp, biểu đồ biến đổi trạng thái để có cái nhìn chi tiết về các phần hệ thống và chúng có quan hệ với nhau như thế nào.

Đội ngũ triển khai sử dụng các biểu đồ thành phần và biểu đồ triển khai để thấy được các tệp khả thực (*exe*) nào, tệp *DLL* nào và các thành phần khác cần được tạo lập; các thành phần này được triển khai trên mạng như thế nào.

Toàn bộ đội ngũ dự án sử dụng mô hình để đảm bảo rằng các yêu cầu có thể được chuyển sang mã trình và ngược lại, mã trình có thể được chuyển trở lại yêu cầu hệ thống.

Hơn nữa, *Rational Rose* còn hỗ trợ phát sinh mã khung chương trình trong nhiều ngôn ngữ khác nhau như *C++, Java, Visual Basic, Oracle8 ...*

2.5 KHẢ NĂNG SỬ DỤNG UML

Mục tiêu của UML là để mô tả bất kỳ loại hệ thống nào bằng biểu đồ hướng đối tượng. Đặc tính của hệ thống đó được tóm tắt như sau:

Các hệ thống thông tin: Lưu trữ, truy vấn, biểu diễn thông tin. Quản lý số lượng thông tin lớn có quan hệ phức tạp trong CSDL quan hệ hay CSDL hướng đối tượng.

Các hệ thống kỹ thuật: Quản lý và điều khiển các thiết bị kỹ thuật như truyền tin, hệ thống quân sự, dây chuyền công nghiệp. Thông thường phải quản lý các giao diện người sử dụng đặc biệt, không có phần mềm chuẩn. Các hệ thống này phần lớn là hệ thống thời gian thực.

Các hệ thống nhúng thời gian thực: Thực hiện trên phần cứng đơn giản nhúng trong các thiết bị khác như điện thoại di động, xe ô tô, các dụng cụ gia đình… Các hệ thống này thường thiếu thiết bị như màn hình, ổ đĩa…

Các hệ thông phân tán: Phân tán trên nhiều máy. Đòi hỏi cơ chế giao tiếp đồng bộ để đảm bảo toàn vẹ dự liệu. Thường được xây dựng trên cơ chế đối tượng như CORBA, COM/DCOM…

Các phần mềm hệ thống: Xác dịnh hạ tầng kỹ thuật để phần mềm khác sử dụng như hệ điều hành, CSDL…

Các hệ thống thương mại: Mô tả mục tiêu, tài nguyên (con người, máy tính…) và các quy luật (chiến lược thương mại, luật …) và qui trình thương mại.

Tuy nhiên ngày nay một hệ thống thường thuộc nhiều loại hoặc tổ hợp của các loại hệ thống kể trên, nhưng ta vẫn có khả năng sử dụng UML để mô hinh hóa chúng.

2.6 THỰC HÀNH

Phần mềm *Rose* có nhiều phiên bản khác nhau. Độc giả có thể sử dụng phần mềm *Rose 98, Rose 2000, Rose v2002* hay các phiên bản thử nghiệm để thực hành các bài tập trong tài liệu này. Độc giả có thể tìm kiếm các phiên bản thử nghiệm hay phiên bản hạn chế (dành cho huấn luyện, đào tạo) của Rose trên trang Web sau:

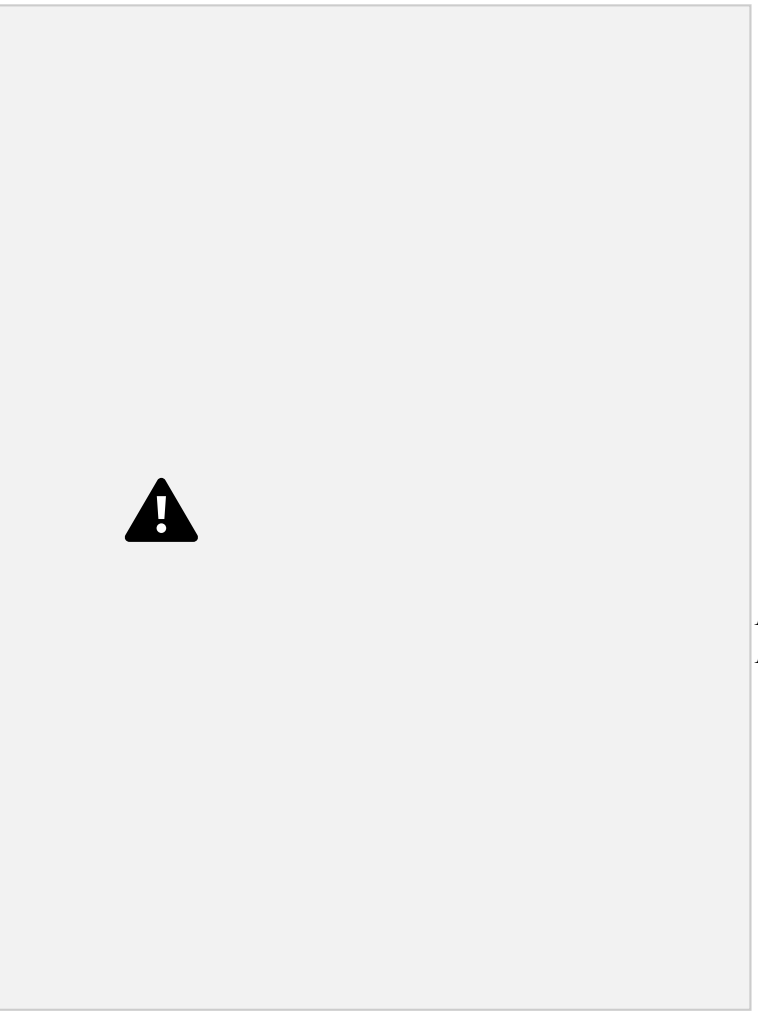
http://www.rational.com/tryit/rose

Mục tiêu của bài thực hành này là làm quen với các màn hình của phần mềm công cụ *Rose 2000*, một phiên bản của phần mềm công cụ *Rational Rose*. Hãy thự hiện theo các bước như mô tả dưới đầy để làm quen với Rose:

1. Khởi động *Rose 2000*. Trên màn hình có các thành phần như hình 2.25.

2. Có thể tắt/mở các cửa sổ duyệt (*Brower Window*) và cửa sổ tài liệu (*Document Window*) bằng cách chọn thực đơn *View>Brower*.

Nơi đặt lại thuộc tính của Rose



Hình 2.25 Giao diện của Rational Rose

Cửa sổ biểu đồ

3. Cửa sổ duyệt chứa danh sách toàn bộ phần tử mô hình trong mô hình hiện hành. Browser có thể trôi nổi hay bám dính (*docked*) bằng cách nhấp đúp chuột trên biên cửa sổ. Các phần tử mô hình hiển thì trong *Browser* dưới dạng cây. Các thông tin nén đươc thể hiện bằng dấu +. Nếu nhấn chuột trên dấu + ta sẽ có thông tín không nén.

   Thự hành: tạo lập tệp mô hình mới bằng cách chọn thực đơn *File>New*, *File>Save as* với tên *tutorial.mdl*

4. Cửa sổ tài liệu là nơi tạo lập, sửa đổi văn bản để gắn vào phần tử mô hình (tác nhân, UC, quan hệ, thuộc tính, thao tác, thành phần và nút). Để tạo tài liệu cho phần tử mô hình ta

làm như sau: chọn phần tử (nhấn chuột lên phần tử), nhập tài liệu vào cửa sổ tài liệu. Cửa sổ tài liệu cũng có thể tắt/mở, trôi nổi hay bám dính như cửa sổ Browse.

Thự hành: Nhập tài liệu cho Use case View, Logical View và Component View.

Use Case View chứa thông tin về tác nhân và UC cho hệ thống đang phát triển.

Logical View chứa thông tin về lớp và quan hệ giữa chúng

Component View chứa thông tin về phần mềm và các phần tử khả thực và thư viện.

5. Lưu trữ mô hình vào đĩa từ

6. Cửa sổ biểu đồ là nơi cho phép ta tạo lập và sử đổi khung nhìn đồ họa mô hình hiện hành. Mỗi biểu tượng trong biểu đồ biểu diễn một thành phần mô hình hóa. Mỗi phần tử mô hình có thể hiển thị trong nhiều biểu đồ mô hình khác nhau. Cửa sổ biểu đồ xuất hiện khi nhấn đúp chuột trên cửa sổ biểu đồ trong cửa sổ duyệt.

CHƯƠNG 3

MÔ HÌNH HÓA

TRƯỜNG HỢP SỬ DỤNG

Phân tích và thiết kế bao gồm mô hình hóa vấn đề và giải pháp từ các góc nhìn khác nhau. Trong pha phân tích ta thường quan tâm đến ba loại mô hình,đó là mô hình trường hợp sử dụng, mô hình lĩnh vự và mô hình giao diện người sử dụng. Mô hình trường hợp sử dụng mô tả hệ thống sẽ được sử dụng như thế nào. Mô hình lĩnh vực sẽ mở rộng mô hình trường hợp sử dụng bằng cách đặt hệ thống vào ngữ cảnh. Mô hình giao diện người sử dụng mô tả người sử dụng tương tác với hệ thống như thế nào. Chương này mô tả cách phân tích tìm kiếm trường hợp sử dụng *(Use case – UC)* , tác nhân *(actor)* và quan hệ giữa chúng. Trường hợp sử dụng và tác nhân xác định phạm vi hệ thống. UC là những gì bên trong hệ thống còn tác nhân là những gì bên ngoài hệ thống. Tiếp theo là trình bày về cách tạo lập biểu đồ UC. Cuối chương là phần giới thiệu về công cụ phần mềm *Rational Rose 2000* và ứng dụng nó vào các công việc nói trên.

3.1 PHÂN TÍCH TRƯỜNG HỢP SỬ DỤNG (USE CASE – UC)

3.1.1 - UC là gì?

Khái niệm UC được *Ivar Jacobson* đề xuất vào năm 1994 khi làm việc cho hãng *Eriction*. UC mô tả ai đó sử dụng hệ thống như thế nào, mô tả tương tác giữa người sử dụng với hệ thống phần mềm để thực hiện các thao tác giải quyết công việc cụ thể nào đó. UC không cho biết hệ thống làm việc bên trong như thế nào. Nó không phải là thiết kế, cũng không phải là kế hoạch cài đặt, nó là một phần của vấn đề cần giải quyết. tiến trình của hệ thống được chia nhỏ thành các UC để có thể nhận ra từng bộ phận của nó một cách rõ ràng và để nhiều người có thể cùng xử lý.

UC là nền tảng của phân tích hệ thống. Việc tìm ra đầy đủ các UC đảm bảo rằng hệ thống sẽ xây dựng đáp ứng mọi nhu cầu của người sử dụng. Mỗi UC là tập hành động. mỗi hành động là cái gì đó mà hệ thống làm, nó là hạt nhân được hệ thống thực hiện hoàn toàn hay không được thực hiện phần nào.

3.1.2 - Xây dựng UC để làm gì?

Mục tiêu xây dựng UC trong tiến trình phát triển hệ thống phần mềm được tóm tắt như sau:

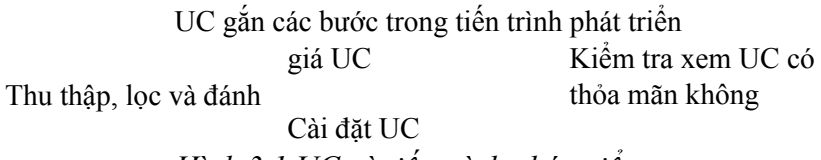
Hình thành quyết định và mô tả yêu cầu chức năng hệ thống. Là kết quả của thỏa thuận giữa khách hàng và người phát triển hệ thống phần mềm.

Cho phép mô tả rõ ràng và nhất quán cái hệ thống sẽ làm, sao cho mô hình có khả năng được sử dụng xuyên suốt quá trình phát triển.

Cung cấp cơ sở để kiểm tra, thử nghiệm hệ thống.

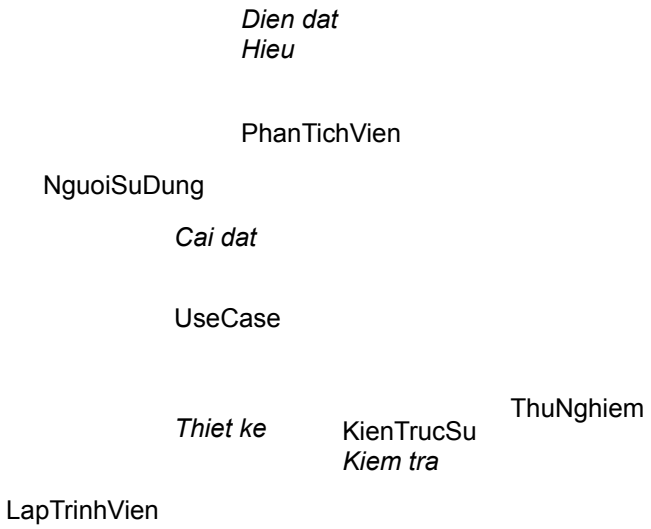
Cho khả năng dễ thay đổi hay mở rộng yêu cầu hệ thống.

Hình 3.1 thể hiện tiến trình phân tích phần mềm theo quan điểm hướng đối tượng được hình thành trên cơ sở UC. Các hành động trong các pha phân tích, thiết kế và cài đặt, kiểm tra và thử nghiệm chương trình đều liên quan đến UC.



Hình 3.1 UC và tiến trình phát triển

Từ mô tả trên ta thấy UC cung cấp cách thức để người phát triển, người sử dụng cuối cùng, các chuyên gia lĩnh vực hiểu nhau. Hình 3.2 cho chúng ta thấy ai sẽ cần UC và cần để làm gì. Người sử dụng diễn đạt UC, kiến trúc sư có nhiệm vụ tiết kế UC, phân tích viên hiểu UC, lập trình viên hiểu UC và nhân viên kiểm tra chất lượng kiểm tra UC



Hình 3.2 Ai quan tâm đến UC?

3.1.3 - Tìm kiếm UC như thế nào ?

Thông thường thì việc xác định và hiểu rõ các yêu cầu hệ thống là rất khó vì khối lượng thông tin liên quan khổng lồ. Do vậy, các yêu cầu sẽ được mô tả một cách lộn xộn không cấu trúc, mâu thuẫn với nhau, thiếu nhiều vấn đề quan trọng, không chính xác… Kết quả của nó có được là tập yêu cầu mờ, rất khó thay đổi. khái niệm UC được đưa ra để tập trung vào biểu thị các yêu cầu từ phía người ứng dụng, xuất phát từ quan điểm đơn giản là hệ thống được xây dựng trước hết là cho người sử dụng chúng. Phân hoạch tập yêu cầu là giảm thiểu đáng kể độ phức tạp của việc xác định yêu cầu (hình 3.3)

Người dùng B

Tập các yêu cầu hệ thống

Người dùng A Người dùng C

Hình 3.3 Phân hoạch yêu cầu bằng UC

Trong phương pháp hướng đối tượng, câu hỏi đặt ra khi bắt đầu thực hiện dự án thường là: *Tìm kiếm UC như thế nào?* Không thể thiếu sự tham gia của khách hàng vào xây dựng UC. Khách hàng hiểu rõ hệ thống sẽ được sử dụng như thế nào. Cách tốt nhất để tìm kiếm UC là phỏng vấn người sử dụng và khảo sát tài liệu của họ. Việc phỏng vấn người sử dụng phải bằng khái niệm, ngôn từ của lĩnh vực vấn đề và của chính người sử dụng. Khi phân tích lĩnh vực ứng dụng, phân tích viên còn phải hợp tác chặt chẽ với với các chuyên gia lĩnh vực và người quản lý dự án. Các chuyên gia lĩnh vực có nhiều kinh nghiệm sử dụng và thiết kế các sản phẩm tương tự, họ có thể giúp chúng ta hiểu chi tiết về người sử dụng tương tác với hệ thống như thế nào. Chất lượng phân tích bị ảnh hưởng nhiều từ các chuyên gia lĩnh vực, do vậy, phải ưu tiên việc giao tiếp, trao đổi kỹ lưỡng với họ. Việc trao đổi với người sử dụng và các chuyên gia lĩnh vực thường được thực hiện độc lập để có thể so sánh cách nhận thức và cách diễn đạt khác nhau của họ. nếu có sự khác biệt thì phải có cuộc họp thảo luận chung để đạt được sự nhất trí hay làm rõ các vấn đề còn mâu thuẫn. kết quả ban đầu của hoạt động này là tác nhân và UC mức cao (mô tả yêu cầu chức năng hệ thống) được bộc lộ. trong thực tế, không phải người sử dụng nào cũng có khả năng mô tả rõ ràng cách mà họ muốn sử dụng hệ thống. những cái họ biết thường nhiều hơn cái họ đang diễn giải. chính UC và biểu đồ UC là công cụ tốt nhất để giúp người sử dụng nói về hệ thống từ góc nhìn của họ.

Thông thường tiến trình tìm kiếm UC sau khi tiềm kiếm tác nhân. tác nhân là thực thể bên ngoài tương tác với hệ thống. Chúng có thể là con người nhưng cũng có thế là hệ thống hay thiết bị phần cứng khác cần tương tác và thời gian. Tương tác là sự trao đổi thông tin. tác nhân con người (người sử dụng) là tác nhân điển hình của mỗi hệ thống. thí dụ trong hệ thống ATM thì khách hàng và người bảo trì hệ thống là tác nhân, trong quản lý thư viện thì độc giả và thủ thư là tác nhân. Tác nhân biểu diễn nhiệm vụ (nó là lớp) chứ không phải là người dùng cụ thể của hệ thống. Thí dụ, độc giả là tác nhân, nhưng *Anh Văn*, *Chị Đào* không phải là tác nhân. Ngân hàng có hệ thống tín dụng để quản trị các thông tin về tài khoản tín dụng của mỗi khách hàng. Hệ thống rút tiền tự động của ATM phải có khả năng giao tiếp với hệ thống tín dụng, vậy hệ thống tín dụng trở thành tác nhân. Thời gian là tác nhân khi nó xác định thời điểm xảy ra sự kiện trong hệ thống.

Một trong các kỹ thuật tìm kiếm tác nhân là trả lời các câu hỏi sau đây:

- Ai sẽ sử dụng các chức năng chính của hệ thống?
- Ai giúp hệ thống làm việc hằng ngày?
- Ai quản trị, bảo dưỡng để hệ thống làm việc liên tục?
- Hệ thống quản lý thiết bị phần cứng nào?
- Hệ thống đang xây dựng tương tác với hệ thống khác nào?

• Ai hay cái gì quan tâm đến kết quả hệ thống cho lại?

Sau khi có tác nhân, hãy trả lời các câu hỏi sau đây để tìm ra các UC:

• tác nhân yêu cầu hệ thống thực hiện chức năng gì?

• Tác nhân cần đọc, tạo lập, bãi bỏ, lưu trữ, sửa đổi các thông tin nào trong hệ thống?

• Có cần thông báo cho tác nhân về sự kiện xảy ra trong hệ thống? có cần tác nhân thông báo cái gì đó cho hệ thống?

• Hệ thống cần vào / ra nào? Vào / ra đi đến đâu hay từ đâu?

Sau khi tìm được UC thì phải gán tên cho chúng. Đặt tên UC theo khái niệm tác nghiệp (không sử dụng khái niệm chuyên môn, kỹ thuật), sử dụng động từ, câu ngắn. UC độc lập với cài đặt (độc lập với ngôn ngữ lập trình), nó có thể được mô tả bằng ngôn ngữ lập trình Java, C++, văn bản trên giấy hay bằng công cụ phần mềm nào đó (*Rational Rose*, *Microsoft Modeler*). Mô tả UC bao gồm các phần tử sau đây:

• Khởi đầu UC - sự kiện khởi động UC. Nó phải được mô tả rõ ràng, thí dụ, "*UC bắt đầu khi X xảy ra*".

• Kết thúc UC – sự kiện dừng UC. Nó phải được nhận diện và làm tài liệu rõ ràng, thí dụ, "*Khi Y xảy ra thì UC kết thúc*".

• Tương tác giữa UC và tác nhân – mô tả rõ ràng cái bên trong hệ thống và cái bên ngoài hệ thống.

• Trao đổi thông tin – tương ứng với các tham số của tương tác giữa hệ thống và tác nhân. nên mô tả theo phong cách không phải phần mềm, thí dụ, "*Người sử dụng làm việc với hệ thống và nhập tên, mật khẩu*".

• Niên đại và nguồn gốc của thông tin – mô tả khi nào hệ thống đòi hỏi thông tin bên trong và bên ngoài, khi nào hệ thống lưu trữ chúng.

• Lặp hành vi trong UC – có thể được mô tả bằng mã giả (*pseudo – code*). •

Tình thế phụ - phải được biểu diễn theo cách thống nhất giữa các UC.

Mỗi một hệ thống thường có khoảng từ 20-50 UC. Mỗi UC cần phải biểu diễn trọn vẹn một giao dịch giữa người sử dụng và hệ thống để cho lại kết quả nào đó. Sau khi xác định xong UC cho hệ thống thì phải kiểm tra xem liệu chúng đã được phát hiện hết chưa. Công việc này được thực hiện bằng cách trả lời các câu hỏi sau đây:

• Mỗi yêu cầu chức năng ở trong ít nhất một UC? Nếu yêu cầu chức năng không ở trong UC nào thì nó sẽ không được cài đặt sau này.

• Đã xem xét mọi tác nhân tương tác với hệ thống?

• Tác nhân cung cấp cho hệ thống thông tin nào?

• Tác nhân nhận thông tin nào từ hệ thống?

• Đã nhận biết mọi hệ thống bên ngoài mà hệ thống này tương tác?

• Thông tin nào hệ thống bên ngoài nhận và gửi cho hệ thống này?

• Các UC vừa nhận ra từ hệ thống có đặc trưng chính như sau:

• UC luôn được tác nhân kích hoạt, tác nhân ra lệnh trực tiếp hay gián tiếp hệ thống thực hiện UC. UC nối với tác nhân bằng quan hệ kết hợp giao tiếp.

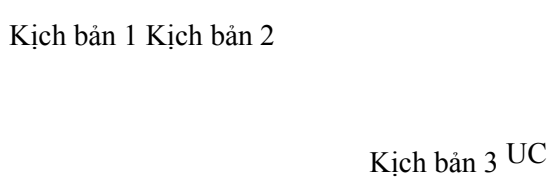
• UC cung cấp giá trị trở lại cho tác nhân; giá trị là cái gì đó mà tác nhân muốn nhận được từ UC.

• UC là hoàn chỉnh, nó được mô tả đầy đủ, không chia UC thành các UC nhỏ hơn.

3.1.4 - Luồng sự kiện trong UC

Như trình bày trên đây, UC bắt đầu mô tả cái hệ thống sẽ làm (không chỉ ra nó sẽ làm như thế nào). Nhưng khi xây dựng hệ thống ta cần mô nó tả chi tiết hơn, các chi tiết này (hành vi của UC) được viết trong tài liệu văn bản luồng sự kiện (*flow of events*) để dễ hiểu. mục đích của luồng sự kiện là làm tài liệu luồng logic đi qua các UC. Tài liệu luồng sự kiện mô tả chi tiết người sử dụng sẽ làm gì và hệ thống sẽ làm gì. Tuy là chi tiết nhưng luồng sự kiện vẫn độc lập ngôn ngữ. Thí dụ, ở đây không đề cập đến loại ngôn ngữ lập trình cụ thể nào sẽ sử dụng để cài đặt. Mỗi UC có nhiều luồng sự kiện (luồng chính, luồng phụ); không thể thể hiện mọi chi tiết của UC trong một luồng sự kiện.

Kịch bản (*scenario*) chỉ ra luồng sự kiện trong một thể hiện (*instance*) cụ thể của UC (hình 3.4). nó là trình tự hành động cụ thể để mô tả hành vi. Kịch bản đi xuyên suốt UC theo nhánh chính hay các nhánh phụ hoặc nhánh đặc biệt của đường đi. Kịch bản là một thể hiện của UC, tương tự đối tượng là thể hiện của lớp, nó cho biết cách sử dụng của hệ thống. khách hàng hiểu hệ thống phức tạp thông qua tập kịch bản mô tả hành vi của nó. Thí dụ, trong một hệ thống có UC tuyển nhân viên. Chức năng tác nghiệp này có nhiều biến thể: tuyển nhân viên mới, thuyên chuyển nhân viên từ cơ quan khác, tuyển nhân viên nước ngoài. Mỗi biến thế biểu diễn trình tự khác nhau, mỗi trình tự đó là kịch bản.



Hình 3.4 Kịch bản trong UC

3.1.4.1 - Làm tài liệu luồng sự kiện

Thông thường tài liệu luồng sự kiện bao gồm: mô tả vắn tắt UC; tiền điều kiện; luồng sự kiện chínhl; luồng sự kiện rẽ nhánh và hậu điều kiện UC. Chúng sẽ được xem xét kỹ lưỡng hơn như dưới đây.

Mô tả. Mỗi UC cần có mô tả ngắn gọn là nó sẽ làm gì. Thí dụ, UC chuyển tiền của hệ thống ATM sẽ có mô tả như sau: UC chuyển tiền cho phép khách hàng hay nhân viên nhà băng chuyển số tiền từ tài khoản này sang tài khoản khác.

Mô tả UC phải ngắn gọn nhưng phải đầy đủ về loại người sử dụng sẽ sử dụng UC và kết quả

cuối cùng mà UC cho lại. các định nghĩa UC này sẽ giúp toàn bộ đội ngũ của dự án thấy rõ tại sao dự án lại có UC này và UC này định làm cái gì.

Tiền điều kiện (*Pe –conditions*). Tiền điều kiện của một UC liệt kê các điều kiện cần được thực hiện trước khi UC khởi động. không phải UC nào cũng có tiền điều kiện. Thí dụ, tiền điều

kiện có thể là UC nào đó được thực hiện trước UC này hay tiền điều kiện là người sử dụng phải có quyền xâm nhập để thực hiện UC này.

Luồng sự kiện chính và luồng rẽ nhánh. Mọi chi tiết của UC được mô tả trong các luồng sự kiện chính và luồng rẽ nhánh. Nó mô tả từng bước cái gì sẽ xảy ra để thực hiện các chức năng của UC. Luồng sự kiện tập trung vào cái hệ thống sẽ làm chứ không tập trung vào nó làm như thế nào, và được mô tả từ góc nhìn của người sử dụng. Luồng sự kiện chính và luồng rẽ nhánh bao gồm:

- UC khởi động như thế nào
- Các đường đi xuyên qua các UC
- Luồng chính thông qua UC
- Luồng rẽ nhánh thông qua UC
- Các luồng lỗi
- UC kết thúc thế nào.

Khi làm tài liệu luồng sự kiện có thể sử dụng các hình thức khác nhau như mô tả bằng văn bản, biểu đồ luồng (*flow chart*)… bằng cách nào đi nữa thì luồng sự kiện phải phù hợp với yêu cầu hệ thống. khách hàng có tài liệu này để khẳng định tính chính xác cái mà họ mong đợi.

Sau đây là thí dụ làm tài liệu luồng sự kiện của UC rút tiền trong hệ thống rút tiền tự động.

Luồng chính

1. UC bắt đầu khi khách hàng đặt thẻ vào máy ATM
2. ATM hiển thị thông báo và chờ khách hàng số căn cước cá nhân (*PIN*)
3. Khách hàng nhập PIN
4. ATM khẳng định PIN có hợp lệ? Nếu PIN không hợp lệ thì thực hiện luồng A1
5. ATM hiển thị các lựa chọn
    a. Gửi tiền vào tài khoản
    b. Rút tiền
    c. Chuyển tiền sang tài khoản khác
6. Khách hàng chọn *Rút tiền*
7. ATM hiển thị câu hỏi số tiền sẽ rút
8. Khách hàng nhập số tiền muốn rút lần này.
9. ATM xác định số dư còn đủ? Nếu không đủ thực hiện luồng A2. Nếu phát hiện lỗi khi kiểm tra số dư thì thực hiện luồng E1
10. ATM trừ số tiền trong tài khoản của khách hàng
11. ATM chuyển tiền cho khách hàng

12. ATM in biên nhận

13. ATM trả lại thẻ tín dụng

14. UC kết thúc.

Luồng nhánh A1: số căn cước cá nhân *(PIN) không hợp lệ*

1. ATM thông báo căn cước cá nhân (*PIN*) không hợp lệ

2. ATM trả lại thẻ tín dụng

3. Kết thúc UC.

Luồng nhánh A2: không đủ tiền trong tài khoản để rút

1. ATM thông báo không đủ tiền

2. ATM trả lại thẻ tín dụng

3. Kết thúc UC.

Luồng E1: lỗi xảy ra khi kiểm tra số dư không đủ

1. ATM thông báo lỗi

2. ATM ghi thông báo này vào tệp chứa lỗi (*error log*)

3. ATM trả lại thẻ tín dụng

4. Kết thúc UC

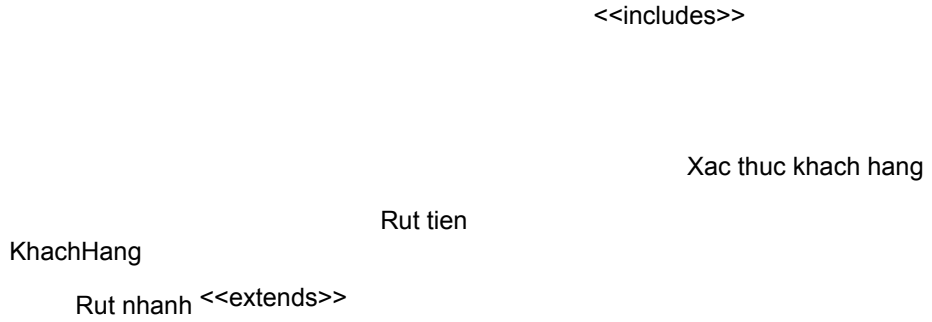
Hậu điều kiện (post–conditions). Hậu điều kiện là điều kiện được thực hiện ngay sau khi kết thúc UC. Nó mô tả trạng thái hệ thống hay tác nhân sau khi hoàn thành UC. Thí dụ, phải đặt cờ báo hiệu UC đã hoàn thành. Tương tự tiền điều kiện, hậu điều kiện có thể chứa thông tin ra lệnh UC khác chạy. không phải mọi UC trong hệ thống đều có hậu điều kiện.

3.2 BIỂU ĐỒ TRƯỜNG HỢP SỬ DỤNG

Mô hình UC trong UML được mô tả bằng một hay nhiều biểu đồ UC. Các biểu đồ UC là công cụ mạnh để thu thập yêu cầu hệ thống. chúng hiển thị các UC, làm dễ dàng các giao tiếp giữa các phân tích viên hệ thống, người sử dụng và giữa phân tích viên hệ thống vời khách hàng. Biểu đồ UC chỉ ra quan hệ giữa các UC và tác nhân. thông thường phải tạo ra vài biểu đồ UC cho một hệ thống. biểu đồ mức cao, *Rational Rose* gọi nó là biểu đồ chính (*Main*), chỉ ra gói hay nhóm UC. Các biểu đồ khác chỉ ra tập các UC và tác nhân. số lượng biểu đồ UC trong một dự án là tùy ý (tốt nhất là khoảng 50 cho dự án tương đối lớn). số lượng biểu đồ đảm bảo đầy đủ thông tin, nhưng cũng không quá nhiều vì sẽ dẫn tới rối loạn. mục đích chính của biểu đồ UC là làm tài liệu tác nhân (mọi thứ bên ngoài phạm vi hệ thống) UC mọi thứ bên trong phạm vi hệ thống) và các quan hệ giữa chúng. Một vài điều cần chú ý khi tạo lập UC như sau:

- Không nên mô hình hóa giao tiếp tác nhân – tác nhân. theo như định nghĩa thì tác nhân nằm ngoài hệ thống, do vậy giao tiếp giữa các tác nhân cũng nằm ngoài phạm vi hệ thống đang xây dựng. Chỉ nên sử dụng biểu đồ luồng công việc (*workflow*) để khảo sát quan hệ này.
- Không hình thành quan hệ trực tiếp giữa hai UC, trừ trường hợp chúng có quan hệ sử dụng (*Uses*) hay quan hệ mở rộng (*extend*s). Biểu đồ UC cho biết có những UC nào trong hệ thống nhưng không cho biết trật tự thực hiện các UC. Biểu đồ hoạt động sẽ cho biết trật tự này.

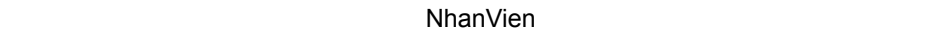
- Mỗi UC phải được tác nhân khởi động, trừ trường hợp đặt biệt khi UC có quan hệ sử dụng hay mở rộng
- CSDL được xem như một lớp dưới toàn bộ biểu đồ UC. Có thể nhập dữ liệu vào CSDL bằng một UC và sử dụng UC khác để xâm nhập chúng. Không hình thành luồng thông tin giữa các UC.



Hình 3.5 Các UC trừu tượng

Một UC không được tác nhân khởi động thì gọi là UC trừu tượng. UC trừu tượng cung cấp một số chức năng cho UC khác sử dụng. Chúng là những UC tham gia vào quan hệ *Uses* hay *extends*. Hình 3.5 là thí dụ phần biểu đồ có sử dụng UC trừu tượng.

Tác nhân mà không có hiện thực (*isntance*) thì gọi là tác nhân trừu tượng. Thí dụ của tác nhân trừu tượng như sau: chia nhân viên làm việc trong một cơ quan thành ba nhóm. Bao gồm nhân viên hưởng lương tháng, nhân viên hưởng lương theo giờ và nhân viên hợp đồng. Ba loại nhân viên này được xem như các tác nhân trong hệ thống quản lý nhân sự. như vậy, trong cơ quan không có ai được gọi là *Nhân viên* mà phải là *Nhân viên hưởng lương tháng*, *Nhân viên hưởng lương giờ* hay *Nhân viên hợp đồng*. Tuy nhiên ta vẫn có thể tạo ra tác nhân *Nhân viên* mang các tính chất chung của ba loại nhân viên nói trên. Nhưng không có hiện thực nào cho tác nhân nhân viên, vậy nó là tác nhân trừu tượng. hình 3.6 mô tả tác nhân trừu tượng và các quan hệ của nó với các tác nhân



hình 3.6 Tác nhân trừu tượng

Các kiểu quan hệ giữa UC và tác nhân trong UML bao gồm quan hệ giao tiếp, quan hệ sử dụng (*Uses*) quan hệ mở rộng (*extends*) và quan hệ tổng quát hóa (*generallization*) tác nhân.

Quan hệ giao tiếp. trong UML. Quan hệ được thể hiện bằng mũi tên. Thí dụ hình 3.7 mô tả tác nhân *Khách hàng* có quan hệ với hệ thống để chạy chức năng *Rút tiền*. Hình này còn cho thấy UC *Rút tiền* kích hoạt tác nhân *Hệ thống thanh toán.*

Quan hệ sử dụng (*Uses*). Trong phiên bản UML 1.3, quan hệ *Uses* được gọi là quan hệ gộp (*include*). Quan hệ *Uses* cho phép một UC sử dụng chức năng của UC khác. Quan hệ này thường được sử dụng để mô hình hóa một vài chức năng sử dụng lại, dùng chung cho hai hay nhiều UC. Trong thí dụ máy rút tiền tự động ATM, hai UC *Rút tiền* và *Gửi tiền vào ngân hàng* cần xác định khách hàng và *số căn cước cá nhân* (PIN) trước khi thực hiện giao dịch. Do vậy chức năng xác nhận này có thể đặt trong một UC riêng, gọi là *Xác nhận khách hàng*. Vào bất kỳ thời điểm nào, UC nào cần xác định khách hàng thì sử dụng UC này.

Hình 3.7 Quan hệ giao tiếp

Trong UML quan hệ *Uses* được mô tả bằng mũi tên và từ <<*includes*>> như trên hình 3.8. trên hình vẽ này, hai UC *Rút tiền* và *Gửi tiền vào* sử dụng chức năng trong UC *Xác nhận khách hàng*. UC *Xác nhận khách hàng* là UC trừu tượng . Còn các UC *Rút tiền* và *Gửi tiền vào* là các UC cụ thể. Các hình chữ nhật gấp góc là các chú thích của biểu đồ



Hình 3.8 Quan hệ include

Quan hệ mở rộng (*extends*). Quan hệ mở rộng cho phép UC mở rộng tùy ý chức năng do UC khác cấp. nó chỉ ra rằng trong một điều kiện nào đó một UC được mở rộng bằng UC khác (mở rộng là gộp vài hành vi của UC tổng quát hơn để sử dụng lại). nó tương tự như quan hệ *Uses* ở chỗ cả hai quan hệ đều tách phần chức năng chung ra một UC mới, đó là UC trừu tượng. Trong UML, quan hệ mở rộng được vẽ bằng mũi tên và có từ <<*extends*>> như trên hình 3.9.

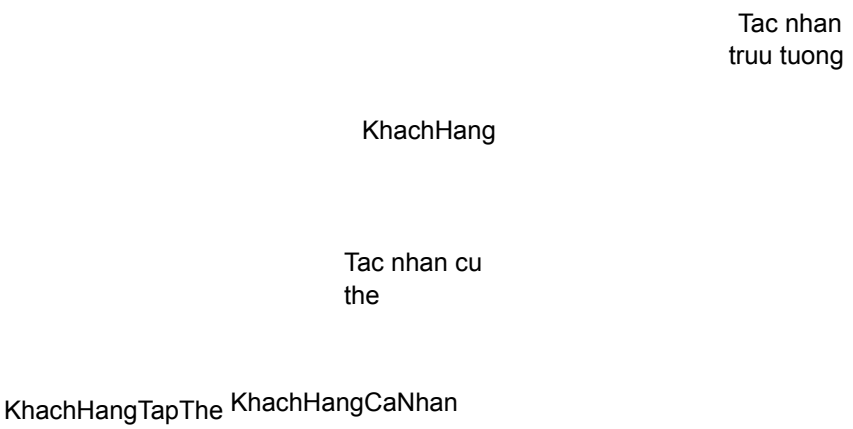
Thí dụ trên hình 3.9 mô tả UC *Rút tiền* đôi khi sử dụng chức năng trong UC *Rút tiền nhanh*. Chỉ và chỉ khi khách hàng chọn thuộc tính rút nhanh dành cho rút số tiền không lớn (thí dụ không rút quá 100.000đ) thì UC này mới chạy. Nó có thể bỏ qua một số thao tác ít quan trọng. UC *Rút nhanh* cung cấp chức năng mở rộng. Nó là UC trừu tượng còn *Rút tiền* là UC cụ thể.

Quan hệ tổng quát hóa (*generalization*) tác nhân. Quan hệ tổng quát hóa tác nhân



Hình 3.9 Quan hệ mở rộng

được sử dụng để chỉ ra một vài tác nhân có một số cái chung. Thí dụ hệ thống có hai loại khách hàng, khách hàng cá nhân và khách hàng tập thể. Biểu đồ trên hình 3.10 mô tả hai loại khách hàng này. Khách hàng cá nhân và tập thể là tác nhân cụ thể. ta có thể định nghĩa loại tác nhân tổng quát (*Khách hàng*) và đặc biệt hóa chúng nhờ quan hệ tổng quát hóa (để có tác nhân *Khách hàng cá nhân* và *Khách hàng tập thể*). Tác nhân *Khách hàng* là tác nhân trừu tượng. ta có thể tiếp tục chia nhỏ, thí dụ chia *Khách hàng tập thể* thành các *Công ty tư nhân* và *Tổ chức chính phủ*… để xây dựng quan hệ tổng quát hóa.



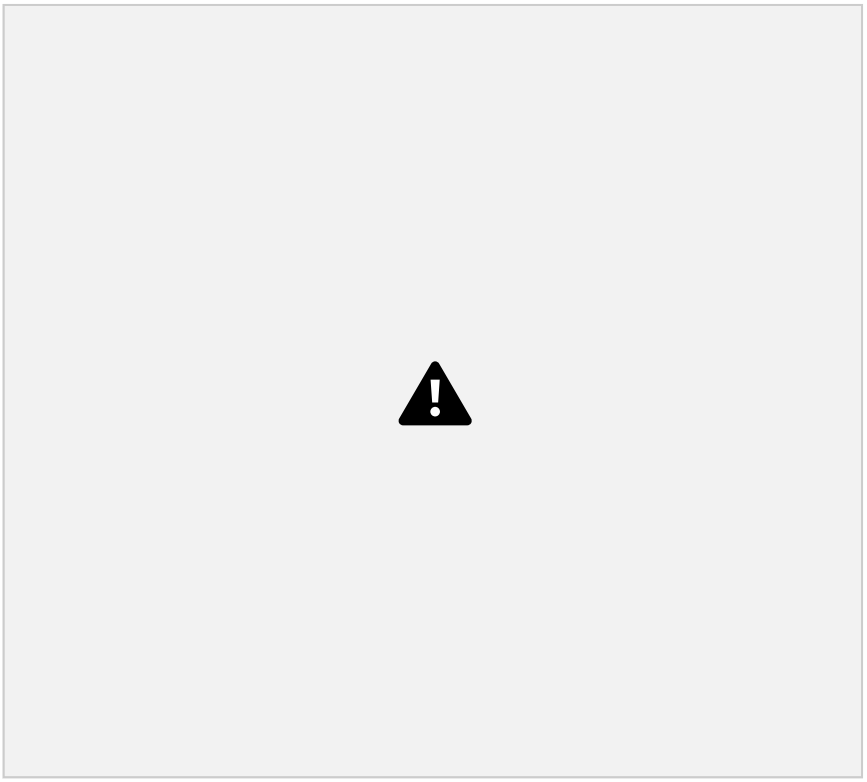
Hình 3.10 Quan hệ tổng quát hóa tác nhân

Không nhất thiết lúc nào cũng tạo ra những kiểu quan hệ như trình bày trên đây. Nếu khách hàng là tập thể tác động vài UC mà khách hàng cá nhân không tác động thì nên có tổng quát hóa tác nhân. Nếu cả hai sử dụng cùng UC thì không cần tổng quát hóa tác nhân. thí dụ biểu đồ UC của hệ thống ATM đã được mô tả trên hình 2.16 của chương 2.

3.3 THỰC HÀNH

3.3.1 - Sử dụng Rational Rose.

Khởi động phần mềm *Rational Rose 2000*.



Hình 3.11 Bổ sung UC mới

3.3.1.1 - Tạo lập biểu đồ trường hợp sử dụng (UC)

Trong Rose các biểu đồ UC được tạo lập trong khung nhìn *USE case*. Rose cung cấp biểu đồ UC mặc định có tên *Main*. Số lượng biểu đồ UC có thể được tạo là tùy ý. Để xâm nhập vào biểu đồ *Main USE case* ta làm như sau (hình 3.11):

1. Nhấn chuột vào dấu + cạnh *USE Case View* trong Browser để mở chúng.
2. Nhìn thấy được biểu đồ *Main USE Case*.
3. Nhấp đúp trong *Main diagram* để mở chúng

Tạo lập biểu đồ USE case mới:

1. Nhấn phím phải trong gói *USE Case View* trong Browser
2. Chọn *New > USE Case Diagram* từ thực đơn
3. Đặt tên cho biểu đồ mới.
4. Nhấp đúp trên tên của biểu đồ mới để mở chúng.

Sử dụng các phím trên thanh công cụ để bổ sung các phần tử biểu đồ vào biểu đồ UC mới.

3.3.1.2 - Bãi bỏ biểu đồ UC

Có thể bãi bỏ biểu đồ UC trong Browser, một khi biểu đồ UC bị xóa thì không thể lấy lại nó. Trình tự xóa biểu đồ UC như sau:

1. Nhấn phím phải trên biểu đồ UC trong Browser
2. Chọn *Delete* từ thực đơn.

3.3.1.3 - Bổ sung UC

Có hai cách bổ sung USE Case vào mô hình: bổ sung UC vào biểu đồ USE case tích cực hay bổ sung trực tiếp USE case mới vào Browser. Bổ sung USE case mới vào biểu đồ USE case được thực hiện như sau

1. Chọn phím *USE case* trên thanh công cụ.
2. Nhấn bất kỳ đâu trên biểu đồ Use case. Use case mới có tên là *NewUSECase*.
3. Nhập tên cho USE case mới.
4. Use case mới tự động cập nhật vào cửa sổ BROWSER, dưới khung nhìn UC.

Bổ sung USE case tồn tại trong mô hính vào biểu đồ USE case:

1. Di USE case từ BROWSER vào biểu đồ USE case đang mở

Bổ sung USE case vào BROWSER được thực hiện như sau:

Nhấn phím chuột phải trên gói khung nhìn Use case trong Browser

2. Chọn *New> USE Case*
3. Use case mới sẽ xuất hiện trong BROWSER
4. Đặt tên cho USE case mới
5. Để gắn USE case vào biểu đồ, di USE case mới từ BROWSER vào biểu đồ.

3.3.1.4 - Bãi bỏ USE case

Có khả năng bãi bỏ UC trong một biểu đồ hay trong toàn bộ các biểu đồ của mô hình. Việc bỏ UC khỏi biểu đồ UC như sau:

1. Chọn USE case trong biểu đồ
2. Nhấn phím *Delete*
3. Use case đã chọn sẽ biến mất khỏi biểu đồ USE case nhưng nó còn trong BROWSER và trong các biểu đồ khác

Việc bãi bỏ USE case trong toàn bộ mô hình được thực hiện như sau:

1. Chọn USE case trong biểu đồ
2. Chọn *Edit> Delete from Model* hay nhấn các phím *Ctrl*+D
3. Use case vừa chọn bị loại bỏ trong toàn bộ mô hình và BROWSER.

3.3.1.5 - Đặc tả USE case

Rose cho khả năng đặc tả chi tiết từng USE case, cho khả năng làm tài liệu thuộc tính như tên, mức ưu tiên, *stereotype* của USE case.

Mở đặc tả Use case theo các bước sau:

1. Nhấn phím chuột phải trên Use case biểu đồ Use case
2. Chọn thực đơn *Open Specification*.

Hoặc

1. Nhấn phím chuột phải trên Use case trong Browser
2. Chọn thực đơn *Open Specification*.

3.3.1.6 - Đặt tên USE case

Mỗi Use case trong mô hình cần có tên duy nhất, độc lập với cài đặt. Thông thường sử dụng động từ hay các câu ngắn làm tên UC, đặt tên Use case như sau:

1. Chọn Use case trong Browser hay trong biểu đồ Use case
2. Nhập tên Use case.

Bổ sung tài liệu cho Use case:

1. Chọn Use case trong Browser
2. Nhập mô tả Use case trong cửa sổ tài liệu.

3.3.1.7 - Quan sát thành viên của USE case

Rose cho khả năng liệt kê toàn bộ các lớp và thao tác trong một UC cụ thể. Chức năng này cho ta dễ theo dõi sự ảnh hưởng đến các lớp khi thay đổi yêu cầu.

Để xem các lớp và thao tác tham gia trong Use case, ta làm như sau:

1. Chọn Use case trong biểu đồ Use case
2. Chọn *Report> Show Participants* trong UC
3. Cửa sổ *Participants* sẽ hiển thị

3.3.1.8 - Gán Stereotype cho Use case

Trong UML, sử dụng *stereotype* để phân nhóm các thành phần mô hình. Thí dụ, ta có hai *Use case A* và *Use case B*. Ta có thể tạo ra hai *stereotype* cho *Use case A* và *B*. *Stereotype* không được sử dụng thường xuyên cho Use case, nó thường sử dụng cho lớp hay cho quan hệ

Gán *stereotype* cho Use case như sau:

1. Nhấn phím chuột phải trên Use case trong Browser hay trong biểu đồ UC
2. Chọn *Open Specification*
3. Nhập *stereotype* trong vùng *stereotype.*

3.3.1.9 - Gán mức ưu tiên cho Use case

Có thể gán mức ưu tiên cho Use case để biết trật tự làm việc vơi các Use case. Gán mức ưu tiên thông qua *Open Specification* tương tự như trường hợp trên.

3.3.1.10 - Tạo lập USE case trừu tượng

Use case trừu tượng là Use case không được kích hoạt trực tiếp từ tác nhân. Nó cung cấp thêm một số chức năng để Use case khác sử dụng. tạo Use case trừu tượng như sau:

1. Tạo Use case trong Browser hay biểu đồ Use case
2. Nhấn phím chuột phải trên biểu đồ Use case hay trong Browser
3. Chọn *Open Specification*
4. Đánh dấu hộp *Abstract*

3.3.1.11 - Gắn tệp vào UC

Sau khi chọn thực đơn đặc tả UC (*Open Specification*), ta có thể quan sát các biểu đồ định nghĩa trong UC, các quan hệ giữa các UC và giữa UC với tác nhân. đồng thời, bảng *Files* trong cửa sổ này còn cho khả năng gán tệp vào UC. Nội dung tệp có thể là tài liệu về luồng sự kiện cho UC, đặc tả các yêu cầu, các kịch bản… Gán tệp vào UC theo các bước sau:

1. Nhấn phím chuột phải vào vùng trắng của *Files tab*
2. Chọn thực đơn *Insert File* để xen tệp
3. Sử dụng hộp thoại *Open* để tìm tên tệp sẽ gán
4. Chọn *Open* để gắn tệp vào UC.

3.3.1.12 - Bổ sung tác nhân

Tương tự như Use case, có thể bổ sung tác nhân vào biểu đồ UC hay trực tiếp vào browser. Một tác nhân trong browser có thể được bổ sung vào một hay nhiều biểu đồ UC.

Trình tự bổ sung tác nhân vào biểu đồ UC như sau:

1. Chọn phím *Actor* trên thanh công cụ
2. Nhấn bất kỳ đâu trong biểu đồ UC. Tác nhân mới sẽ được gán tên mặc định là *NewClass*
3. Nhập tên cho tác nhân mới, nó được tự động gán vào browser.

3.3.1.13 - Hủy bỏ tác nhân

Tương tự như Use case, nếu muốn hủy bỏ tác nhân khỏi biểu đồ UC thì chọn chúng rồi nhấn phím *Delete*. Nếu muốn loại bỏ tác nhân khỏi mô hình thì nhấn phím *Ctrl+D* sau khi đã chọn nó.

3.3.1.14 - Đặc tả tác nhân

Tương tự như Use case, tác nhân cũng có cửa sổ đặc tả để đặt tên, *stereotype*, số yếu tố (*cardinality*) và các chi tiết khác

Sau khi nhấn phím phải chuột trên tác nhân trong biểu đồ UC hay browser ta có thể chọn thực đơn *Open Specification* để gán tên, *stereotype*… cho tác nhân cách làm này tương tự như gán đặc tả cho UC vừa mô tả trên.

3.3.1.15 - Đặt số yếu tố (cardinality) tác nhân

Số yếu tố chỉ ra tổng số hiện thực (*instance*) của tác nhân. thí dụ số yếu tố được sử dụng để thể hiện nhiều người là khách hàng, một người làm quản trị

Số yếu tố	*Ý nghĩa*
n (mặc định)	Nhiều
0..0	zero
0..1	0 hay 1
0..nhiều	0 hay nhiều
1..1	chỉ một
1..n	một hay nhiều

Nhập số yếu tố trong *Detail tab* sau khi chọn thực đơn *Open Specification*.

3.3.1.16 - Tạo lập tác nhân trừu tượng

Tạo tác nhân trừu tượng là tác nhân không có hiện thực (instance), số yếu tố của nó bằng 0. tác nhân trừu tượng được tạo lập như sau:

1. Tạo tác nhân trong Browser hay biểu đồ UC
2. Nhấn phím phải chuột trên tác nhân
3. Chọn thực đơn *Open Specification*
4. Chọn *Detail tab*
5. Đánh dấu hộp thoại *Abstract*

3.3.1.17 - Các quan hệ trong biểu đồ

Bổ sung quan hệ uses vào biểu đồ UC theo các bước sau

1. Chọn phím *Generalization* trên thanh công cụ
2. Nối UC cụ thể để UC trừu tượng

3. Nhấn phím phải chuột trên đường quan hệ, chọn thực đơn *Open Specification*

4. Trong cửa sổ *Stereotype*: nhập uses

5. Mở cửa sổ *UC specification* của UC trừu tượng

6. Đánh dấu vào hộp thoại *Abstract*

Bãi bỏ quan hệ uses trong biểu đồ UC theo các bước sau:

1. Chọn quan hệ trên biểu đồ UC

2. Chọn *Edit> Delete from ModelI* hay nhấn phím *Ctrl+D.*

Việc bổ sung và bãi bỏ quan hệ *extends* trong biểu đồ UC được thực hiện tương tự như bổ sung hay bãi bỏ quan hệ *uses* như vừa mô tả trên đây.

Bổ sung quan hệ khái quát hóa tác nhân (*actor generalization*) như sau:

1. Chọn quan hệ trên biểu đồ UC

2. Chọn phím *Generalization* trên thanh công cụ

3. Vẽ từ tác nhân cụ thể sang tác nhân trừu tượng

4. Mở cửa sổ đặc tả tác nhân của tác nhân trừu tượng

5. Chọn *Detail tab*

6. Đánh dấu *Abstract*.

Quan hệ khái quát hóa tác nhân được hủy bỏ bằng cách nhấn phím *Ctrl+D* sau khi đã chọn quan hệ trên biểu đồ UC

3.3.1.18 - Làm việc với ghi chú (notes)

Khi tạo biểu đồ, nên gắn ghi chú vào UC hay tác nhân. thí dụ ghi chú làm rõ tại sao tác nhân lại tương tác với UC, tại sao UC lại có quan hệ *uses* hay *extends*, tại sao UC hay tác nhân này lại kế thừa từ UC hay tác nhân khác. *Rational Rose* có công cụ (phím trên thanh công cụ) để làm việc này. Bổ sung ghi chú vào biểu đồ như sau:

1. Chọn phím *Note* trên thanh công cụ

2. Nhấn chuột vào nơi muốn đặt ghi chú trên biểu đồ

3. Nhập văn bản cho nó.

Gắn ghi chú vào phần tử mô hình sau:

1. Chọn phím *Anchor Note to Item* trên thanh công cụ

2. Di từ ghi chú đến tác nhân liên quan. *Rose* sẽ vẽ đường nét đứt giữa chúng

Bãi bỏ ghi chú bằng cách thực hiện các bước sau:

1. Chọn ghi chú hay hộp văn bản trong biểu đồ

2. Nhấn phím *Delete* trên bàn phím.

3.3.1.19 - Làm việc với gói (packages)

Khi hiển thị, đặc tả, xây dựng và làm tài liệu hệ thống lớn thì phải quản lý số lượng rất lớn

các phần tử mô hình. Do vậy phải có cơ chế tổ chức chúng lại thành gói. Trong UML, gói là cơ chế đa năng để tổ chức các phần tử mô hình vào nhóm. Các tác nhân, UC, lớp, thành phần đều có thể nhóm trong gói. Trong khung nhìn UC ta có thể nhóm UC và tác nhân vào gói. Các phần tử có ngữ nghĩa gần nhau thì được nhóm vào cùng gói.

Bổ sung gói vào mô hình trong *Rose 2000* như sau:

1. Nhấn phím phải chuột trên khung nhìn UC trong browser
2. Chọn *New> Package*

3. Nhập tên của gói mới

Gói trong biểu đồ UC bị hủy bỏ bằng cách nhấn các phím *Ctrl+D* sau khi nó đã được chọn, hoặc nhấn phím phải chuột trên gói định hủy bỏ trong browser rồi chọn thực đơn *Delete.*

3.3.2 - Thí dụ: hệ thống bán hàng

3.3.2.1 - Vấn đề cần giải quyết

Thí dụ này liên quan tới xây dựng hệ thống theo dõi bán hàng. Mô tả công việc của người sử dụng hệ thống như sau (hình 3.13):

Khi nhận được điện thoại đặt hàng, họ sẽ điền vào mẫu yêu cầu mua hàng (*form*). Sau đó gửi yêu cầu này tới thủ kho. Thủ kho sẽ điền các mặt hàng vào hóa đơn mua hàng (*order*) và chuẩn bị chuyển hàng đến khách hàng. Một bản sao đơn hàng sẽ được gửi cho kế toán. Kế toán sẽ nhập vào hệ thống kế toán để trong báo giá.

Từ mô tả công việc bán hàng như trên, ta có thể xác định rằng hệ thống cần cho khả năng nhập các đơn đặt hàng mới (*Enter New Order*), sửa đổi đơn đăt hàng có sẵn (*Modify Existing Order*), điền đơn hàng (*Fill Order*), trong báo cáo kiểm kê kho (*Print Inventory Report*) và nhập hàng vào kho (*Restock Inventory*). Khi bổ sung đơn hàng mới, hệ thống cần thông báo cho hệ thống kế toán (*Accounting System*) để trong báo giá. Nếu hết mặt hàng nào đó trong kho, hệ thống sẽ từ chối (*Backorder*) bán mặt hàng này. Hình 3.14 là biểu đồ kết quả, nó được xây dựng bằng phần mềm công cụ *Rational Rose 2000* theo những bước mô tả dưới đây.

Đặt mua hàng

Mặt hàng

Chuyển hàng

Hình 3.13 Tiến trình mua bán hàng

3.3.2.2 - Xây dựng biểu đồ Use case

Tạo lập biểu đồ UC cho hệ thống xủ lý đơn hàng:

1. Nhấn đúp chuột trên biểu đồ *Main Use Case* trong browser để vẽ nó
2. Sử dụng phím UC để vẽ UC mới
3. Nhập tên cho UC: *Enter New Order*
4. Lặp lại để vẽ UC khác như sau

    a. Modify Existing Order

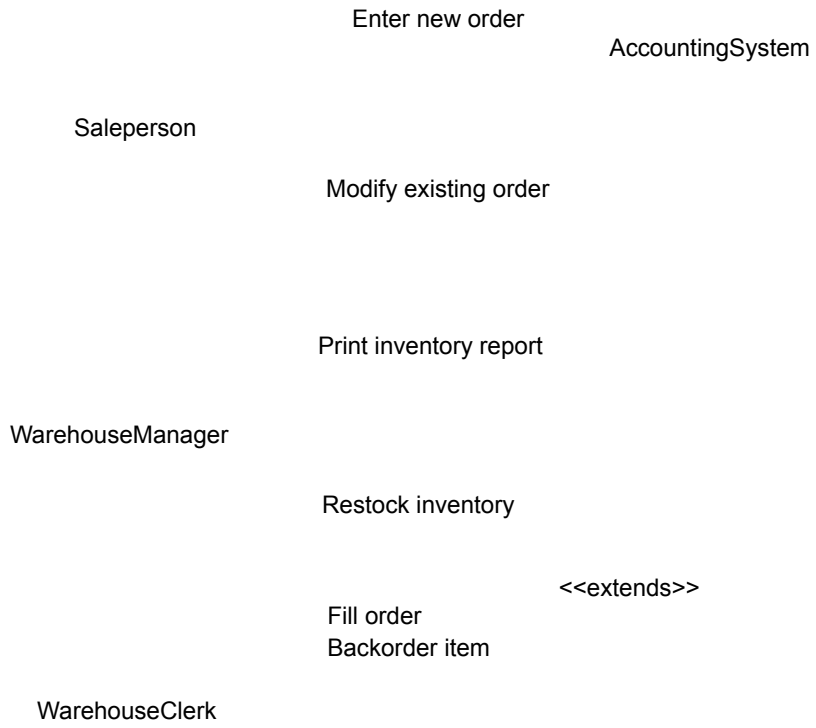
b. Print Inventory Report

c. Restock Inventory

d. Fill Order

e. Backorder Item

5. Sử dụng phím *Actor* để vẽ tác nhân
6. Đặt tên tác nhân mới là *Salesperson*.



Hình 3.14 Biểu đồ UC của hệ thống bán hàng

7. Lặp lại để vẽ các tác nhân sau
    a. Warehourse Manager
    b. Warehourse Clerk
    c. Accounnting System

Đánh dấu UC trừu tượng

8. Nhấn phím phải chuột trên UC Backorder Item trong biểu đồ UC
9. Chọn thực đơn *Open Specification*
10. Đánh dấu hộp thoại *Abstract*

Bổ sung quan hệ

1. Sử dụng phím *Unidirectional Associaton* trên thanh công cụ để vẽ quan hệ giữa *Salesperson* với *Enter New Order*

2. Lặp lại cho các quan hệ còn lại

Bổ sung quan hệ extends

1. Sử dụng phím *Generalization* trên thanh công cụ để vẽ vào biểu đồ
2. Nhấn phím phải chuột trên quan hệ vừa vẽ
3. Chọn thực đơn *Open Specification*
4. Nhập từ ẽ trong hộp danh sách Stereotype

Bổ sung mô tả UC

1. Chọn UC *Enter New Order* trong browser
2. Nhập mô tả vào cửa sổ tài liệu. thí dụ, mô tả có thể là: trường hợp sử dụng này cho người sử dụng khả năng bổ sung đơn hàng mới vào hệ thống
3. Tương tự, nhập mô tả vào các cửa sổ tài liệu cho các UC khác

Bổ sung mô tả tác nhân

1. Chọn tác nhân *Salesperson*
2. Nhập mô tả vào cửa sổ tài liệu. thí dụ, mô tả có thể là: *Nhân viên bán hàng là người thực hiện nhiệm vụ buôn bán sản phẩm*.
3. Tương tự, bổ sung mô tả cho các tác nhân khác.

Gắn tệp vào UC

1. Gắn tệp mô tả luồng sự kiện vào UC *Enter New Order*. Trước khi gắn phải tạo tệp văn bản mô tả luồng sự kiện. Thí dụ, tạo tệp *OrderFlow.doc* với mô tả như sau:

    a. Nhân viên bán hàng chọn thực đơn *Create New Order* để thực hiện

    b. Hệ thống hiển thị mẫu khai *Order Detail*

    c. Nhân viên bán hàng nhập số đơn hàng, tên khách hàng và các mặt hàng mà khách yêu cầu

Nhân viên bán hàng lưu lại đơn hàng

    d. Hệ thống tạo ra đơn hàng mới và lưu nó vào cơ sở dữ liệu

2. Nhấn phím phải chuột trên UC *Enter New Order*
3. Chọn thực đơn *Open Specification*
4. Chọn bảng *Files*
5. Chọn tên tệp để gắn vào UC (nhấn phím phải vào vùng *Filename*)

CHƯƠNG 4

MÔ HÌNH HÓA TƯƠNG TÁC ĐỐI TƯỢNG

Chương này trình bày về mô hình hóa quan hệ giữa các đối tượng trong hệ thống. Hai loại biểu đồ sẽ được xem xét ở đây, đó là biểu đồ trình tự và biểu đồ cộng tác. Hai loại biểu đồ này được gọi một tên chung, đó là biểu đồ tương tác.

4.1 ĐỐI TƯỢNG VÀ TÌM KIẾM ĐỐI TƯỢNG

Đối tượng là cái để gói thông tin và hành vi. Thí dụ, hệ thống ATM có các đối tượng như màn hình ATM, máy đọc thẻ, giấy biên nhận, tài khoản của khách hàng… Mỗi đối tượng gói một vài thông tin và một vài hành vi. Thí dụ, đối tượng *Tài khoản của ông A* chứa thông tin như số tài khoản (*1234*), tên chủ tài khoản (*ông A*), số dư tài khoản (*20 triệu đồng*)… đối tượng này còn chứa các hành vi như cộng tiền vào tài khoản khi ông A gửi vào, trừ tiền trong tài khoản khi ông A rút tiền ra hay thông báo khi ông A rút quá số tiền trong tài khoản… Các vùng lưu giữ thông tin trong đối tượng được gọi là thuộc tính (*attribute*). Giá trị của thuộc tính thay đổi được (thí dụ số tiền trong tài khoản của ông A có thể tăng trong tuần tới) nhưng thuộc tính thì không. Hành vi đối tượng được gọi là thao tác (*operation*). Các thao tác trong thí dụ này thực hiện hiệu chỉnh cân đối tài khoản khi gửi hay rút tiền.

Cách tốt nhất để tìm ra đối tượng là khảo sát *danh từ* trong luồng sự kiện, hay tìm trong tài liệu kịch bản. Kịch bản là phiên bản cụ thể của luồng sự kiện. Thí dụ, một luồng sự kiện của UC *Rút tiền*. Một kịch bản của nó có thể là ông A rút 100.000đ. Kịch bản khác có thể là chị B rút 100.000đ nhưng chị ta nhập nhầm mật mã (*PIN*). Kịch bản khác có thể là ông C rút 100.000đ nhưng số dư tài khoản của ông ta chỉ là 90.000đ. Thông thường có nhiều kịch bản cho một luồng sự kiện. Mỗi biểu đồ trình tự hay biểu đồ cộng tác sẽ mô tả một trong các kịch bản này. Nếu tìm ra danh từ trong kịch bản thì một trong số đó có thể là tác nhân và một số khác có thể là đối tượng hay thuộc tính của đối tượng. Khi xây dựng biểu đồ tương tác thì danh từ cho biết cái nào là đối tượng. Nếu còn do dự rằng đó là đối tượng hay thuộc tính thì hãy hỏi xem nó có hành vi hay không. Nếu nó chỉ có thuộc tính thì đó có thể là thuộc tính, nếu có hành vi thì nó phải là đối tượng. Không nhất thiết mọi đối tượng là ở trong luồng sự kiện. Thí dụ mẫu báo cáo (*form*s) không trong luồng sự kiện nhưng nó xuất hiện trong biểu đồ để tác nhân có thể nhập hay xem dữ liệu. Các đối tượng điều khiển cũng không ở trong luồng sự kiện. Đối tượng điều khiển là đối tượng điều khiển trình tự luồng xuyên qua UC.

Hơn nữa có rất nhiều khái niệm trong khi phân tích sẽ được chuyển sang đối tượng khi thiết

kế. Một số đối tượng trong thiết kế được ủy quyền từ các tác nhân hay khái niệm trừu tượng trong lĩnh vực vấn đề. Thí dụ, trong hệ thống quản lý thư viện, ta phải tách khái niệm tên sách và chính quyển sách. Tên sách là trừu tượng không tương ứng với đối tượng nào trong thế giới thực.

4.2 BIỂU ĐỒ TƯƠNG TÁC

Biểu đồ tương tác (*interaction diagram)* được sử dụng trong UML để mô hình hóa khía cạnh động của hệ thống (hệ thống đang chạy). Biểu đồ tương tác chỉ ra một tương tác, bao gồm tập đối tượng, quan hệ và các thông điệp trao đổi giữa chúng. Biểu đồ này chỉ ra từng bước của một luồng điều khiển cụ thể trong UC. UC *Rút tiền* trong hệ thống rút tiền tự động ATM có nhiều luồng sự kiện. Do vậy ta sẽ có nhiều biểu đồ tương tác cho UC này. UML có hai loại biểu đồ

tương tác, đó là biểu đồ trình tự và biểu đồ cộng tác. Biểu đồ trình tự theo trật tự thời gian, biểu đồ cộng tác chỉ ra cùng loại thông tin nhưng có cách tổ chức khác. Trong khi biểu đồ trình tự tập trung vào điều khiển thì biểu đồ cộng tác tập trung vào luồng dữ liệu. Các biểu đồ này đều liên quan đến các đối tượng cài đặt chức năng thực hiện trong UC. Hai biểu đồ này được xây dựng cho đối tượng lớp hay cả hai.

Khi xây dựng biểu đồ tương tác có nghĩa là ta gán trách nhiệm cho đối tượng. Thực hiện bổ sung thông điệp vào biểu đồ có nghĩa là gán trách nhiệm cho đối tượng nhận thông điệp. hãy đảm bảo rằng phải gán trách nhiệm phù hợp cho đối tượng tương ứng. Trong phần lớn các ứng dụng, đối tượng *màn hình* và đối tượng *mẫu báo cáo* thường không thực hiện tác nghiệp nào. Chúng chỉ được sử dụng để người dùng nhập liệu và quan sát thông tin. Do vậy, nếu logic tác nghiệp thay đổi sẽ không ảnh hưởng gì đến giao diện hệ thống và ngược lại, thay đổi giao diện sẽ không ảnh hưởng để tác nghiệp. Thí dụ, nếu cần in báo cáo về cân đối tài khoản, đối tượng *Tài khoản của ông A* không có trách nhiệm làm việc này, hãy gán trách nhiệm cho đối tượng khác, nó có khả năng tìm kiếm trong mọi tài khoản để hình thành báo cáo.

Từ biểu đồ tương tác, người thiết kế và người phát triển có thể xác định các lớp sẽ xây dựng, quan hệ giữa các lớp, thao tác và trách nhiệm của mỗi lớp. Biểu đồ tương tác trở thành nền tảng cho các công việc còn lại khi thiết kế. Biểu đồ trình tự là trật tự theo thời gian của các thông điệp, nó có ích khi ai đó muốn quan sát luồng logic trong kịch bản. Biểu đồ cộng tác có ích khi ta muốn quan sát giao tiếp giữa các đối tượng. Tổng quát thì biểu đồ tương tác chứa các thành phần sau đây:

• Đối tượng: biểu đồ tương tác sử dụng tên đối tượng, tên lớp hay cả hai. •

Liên kết: liên kết là hiện thực của kết hợp.

• Thông điệp: thông qua thông điệp lớp hay đối tượng có thể yêu cầu lớp hay đối tượng khác thực hiện chức năng cụ thể. thí dụ, form có thể yêu cầu chức năng Report tự in.

• Chú thích (notes) và ràng buộc như mọi biểu đồ khác.

4.2.1 - Biểu đồ trình tự

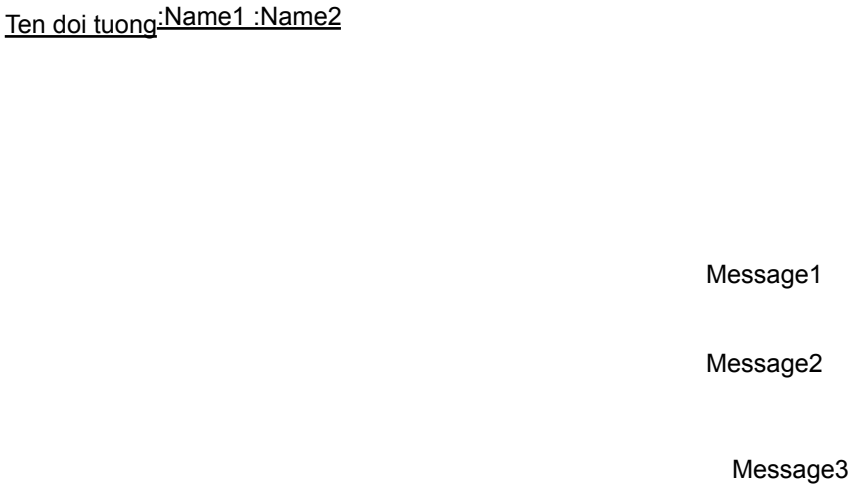
Biểu đồ trình tự (*sequence diagram*) là biểu đồ tương tác theo trật tự thời gian của các giao tiếp bằng thông điệp giữa các đối tượng; biểu đồ được đọc từ đỉnh xuống đáy. Mỗi UC có nhiều luồng dữ liệu. Mỗi biểu đồ trình tự biểu diễn một luồng dữ liệu.

TênĐốiTượng TênĐốiTượng : Lớp : Lớp <<Stereotype>>> TênĐốiTượng

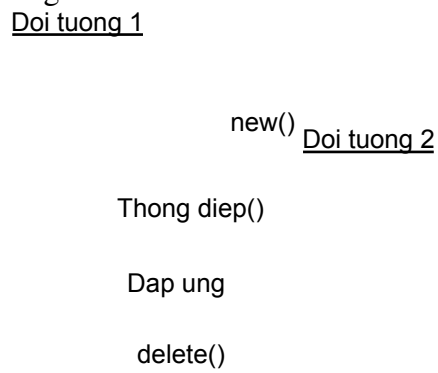
Hình 4.1 Biểu diễn đối tượng

Đọc biểu đồ trình tự bằng cách quan sát các đối tượng và thông điệp. Biểu đồ trình tự bao gồm các phần tử để biểu đồ đối tượng, thông điệp và thời gian. Đối tượng trong biểu đồ được biểu diễn bằng hình chữ nhật, trong hình chữ nhật là tên của nó (hình 4.1). Đối tượng được vẽ trong đỉnh biểu đồ. Thứ tự của các đối tượng trong biểu đồ được tổ chức sao cho biểu đồ được quan sát dễ dàng. Thời gian được biểu diễn bằng đường gạch gạch theo phương thẳng đứng (gọi là chu kỳ sống – *lifeline* – của đối tượng), bắt đầu từ đỉnh và kết thúc ở đáy biểu đồ. Hình chữ nhật hẹp dọc theo chu kỳ sống được gọi là *hoạt động*. Hoạt động biểu đồ thời gian thực thi một hành động tương ứng (hình 4.2a). Chú ý rằng trong biểu đồ trình tự do *Rational Rose* tạo ra, đôi khi hình chữ nhật này bị ẩn là chỉ để cho dễ quan sát biểu đồ.

Thông điệp được vẽ bằng mũi tên đóng đi từ chu kỳ sống của đối tượng này đến chu kỳ sống của đối tượng khác. Trên mũi tên là thông điệp. mỗi thông điệp là biểu diễn một đối tượng gọi hàm của đối tượng khác. Khi định nghĩa thao tác cho lớp sau này thì mỗi thông điệp sẽ trở thành thao tác. Một đối tượng còn có thể gửi thông điệp đến chính nó. Các thông điệp này gọi là phản thân, nó chỉ ra rằng đối tượng gọi chính thao tác của mình. Hình 4.2b mũi tên tập phần tử biểu đồ cơ bản của biểu đồ trình tự. Tuy rằng biểu tượng tác nhân kích hoạt biểu đồ trình tự nhưng nó không phải là phần tử của biểu đồ loạiại này [KALA01]. Trong quá trình tương tác, đối tượng mới có thể được lập hay đối tượng cũ bị hủy bỏ. Việc tạo lập hay hủy bỏ đối tượng có thể được biểu đồ trong biểu đồ trình tự (hình 4.3). Trong thí dụ này, thông điệp *new* tạo lập đối tượng 2 và thông điệp *delete* hủy bỏ nó. Hình 4.3 còn cho thấy đáp ứng do thông điệp cung cấp với tên *response*, nó có thể làm tham số cho thông điệp khác. Trên biểu đồ trình tự, đáp ứng được biểu diễn bởi đường nét đứt và mũi tên mở.



Hình 4.2 Thành phần cơ bản của biểu đồ trình tự



Hình 4.3 Tạo lập và bãi bỏ đối tượng

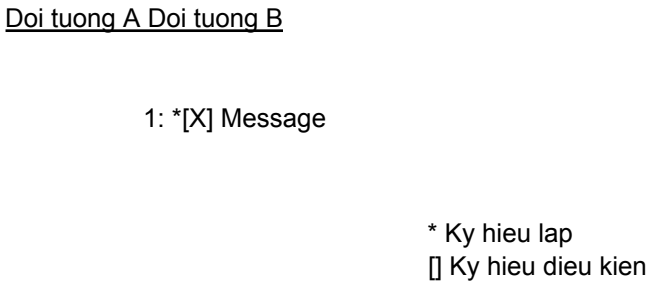
a)

Hình 4.4 Scripts trong biểu đồ trình tự

b)

Trong khi *Notes* được sử dụng để chú thích cho đối tượng thì *Scripts* được sử dụng để chú thích cho thông điệp. Chúng được đặt tại phía trái của biểu đồ trình tự, ngang với mức thông điệp cần chú thích. Có thể sử dụng *Scripts* để mô tả các logíc điều kiện trong biểu đồ, thí dụ lệnh *IF* hay trong vòng lặp *while*... Mã trình không được phát sinh cho *Scrips* nhưng nó cho người phát triển hệ thống biết được logic cần tuân thủ. Thí dụ bổ sung *Scripts* cho biểu đồ trình tự được mô tả trên hình 4.4.

Năm 1996, *Buschman* và cộng sự đề nghị bổ sung điều kiện vào thông điệp theo ký pháp *[condition] message()*. Biểu đồ trên hình 4.5 sử dụng ký pháp này. Vào thời gian đó, UML đã đưa ra biểu đồ hoạt động (*activity diagram*) phù hợp hơn cho thủ tục đa nhánh. Do vậy, việc sử dụng điều kiện trong biểu đồ trình tự không còn được khuyến cáo nữa [OEST00].



Hình 4.5 Thông điệp có điều kiện

Thông điệp là phương tiện giao tiếp giữa các đối tượng. Nó được biểu diễn trong biểu đồ hợp tác bởi mũi tên giữa nơi gửi và nơi nhận. Kiểu mũi tên chỉ ra loại thông điệp, nói cách khác là loại đồng bộ của thông điệp. UML có các loại đồng bộ thông điệp như sau đây:

Đơn: là giá trị mặc định của thông điệp. Phần tử biểu đồ này chỉ ra rằng thông điệp sẽ chạy trong luồng đơn của điều khiển. Cả biểu đồ trình tự và biểu đồ cộng tác đều sử dụng biểu tượng thông điệp đơn như trên hình 4.6a.

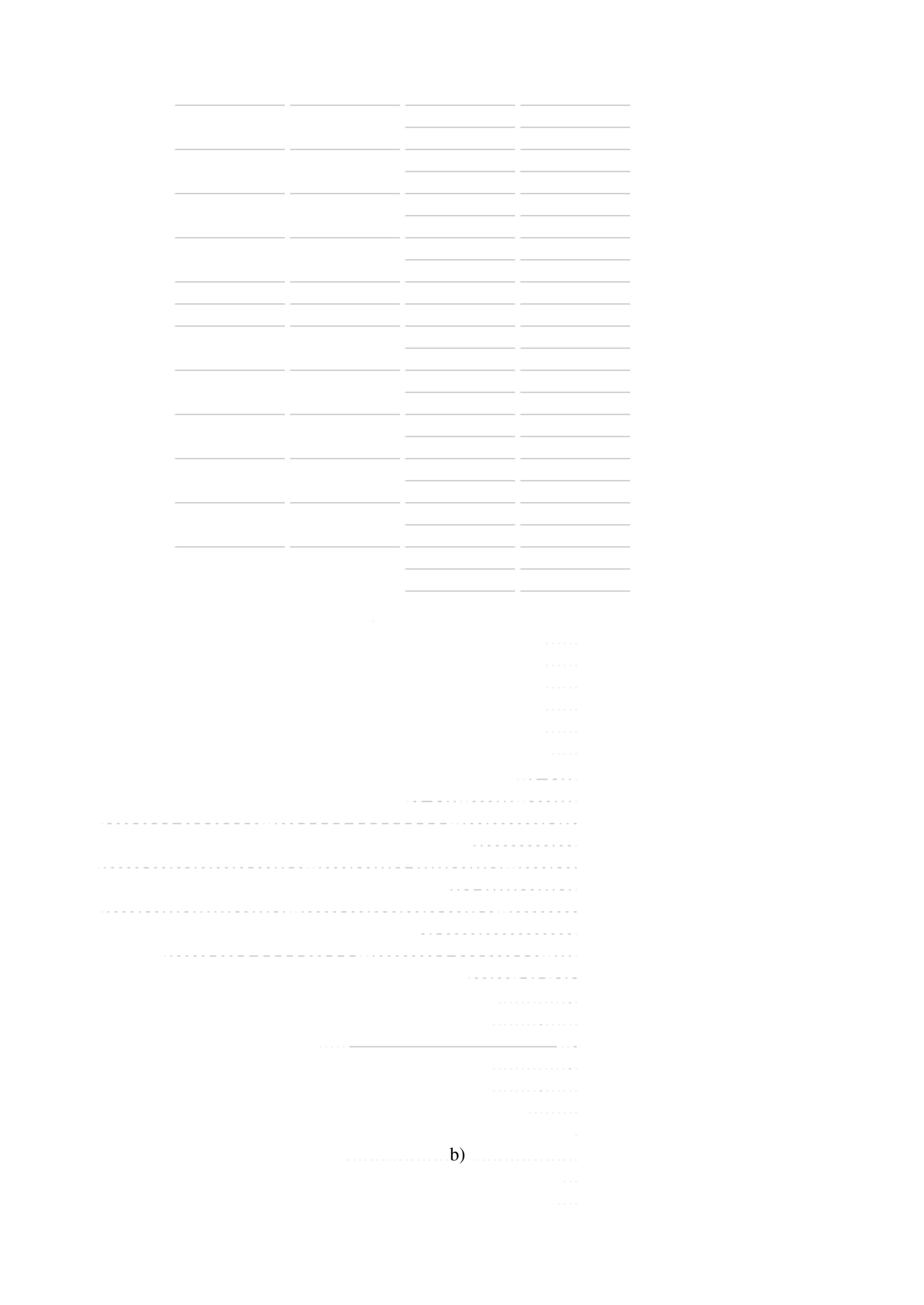
Đồng bộ: thuộc tính này được sử dụng khi đối tượng gửi thông điệp và chờ đến khi nó được xử lý xong. Cả biểu đồ trình tự và biểu đồ cộng tác đều sử dụng biểu tượng thông điệp đồng bộ như trên hình 4.6b.

Cản trở (Balking, Rendez-vous): đối tượng gửi thông điệp đến nơi nhận. Nếu nơi nhận chưa

xử lý ngay thì đối tượng gửi bãi bỏ thông điệp. Cả biểu đồ trình tự và biểu đồ cộng tác đều sử dụng biểu tượng thông điệp cản trở như trên hình 4.6c.



a)



b)

c)

d)

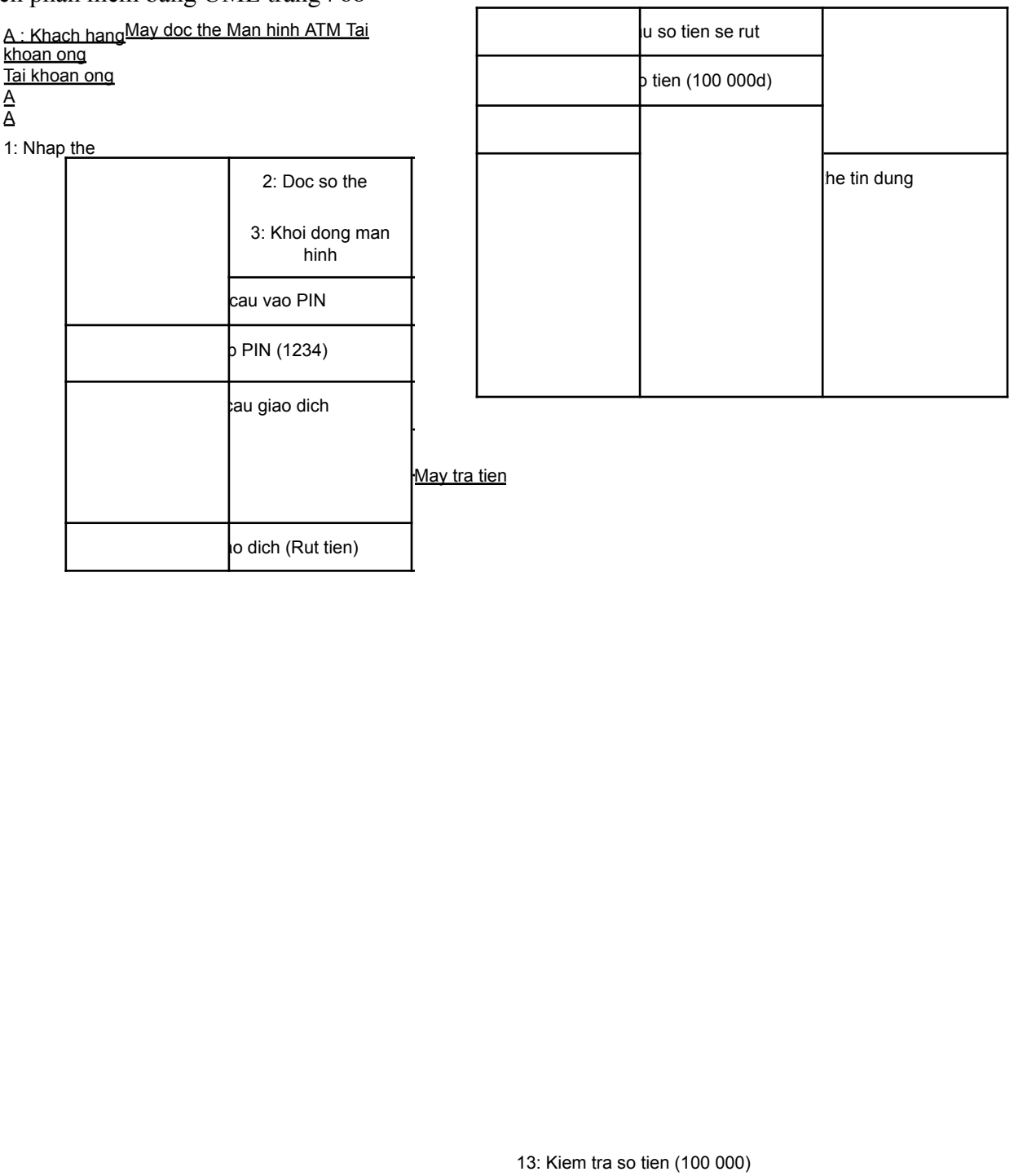
e)

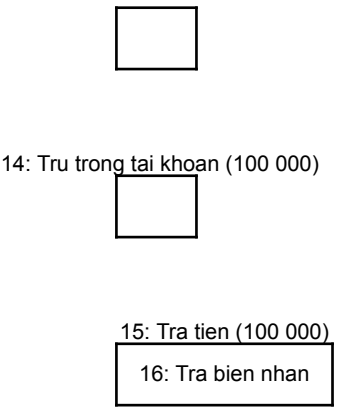
Hình 4.6 Các loại thông điệp

Hết hạn (time out): Đối tượng gửi thông điệp đến đối tượng nhận và chờ một khoảng thời

gian. Nếu sau thời gian đó mà thông điệp chưa được xử lý thì đối tượng gửi bãi bỏ chúng. Cả biểu đồ trình tự và biểu đồ cộng tác đều sử dụng biểu tượng thông điệp hết hạn như trên hình 4.6d.

Dị bộ: Đối tượng gửi thông điệp đến đối tượng nhận. Đối tượng gửi tiếp tục làm việc khác không quan tâm đến thông điệp đó có được xử lý hay không. Cả biểu đồ trình tự và biểu đồ cộng tác đều sử dụng biểu tượng thông điệp dị bộ như trên hình 4.6e.

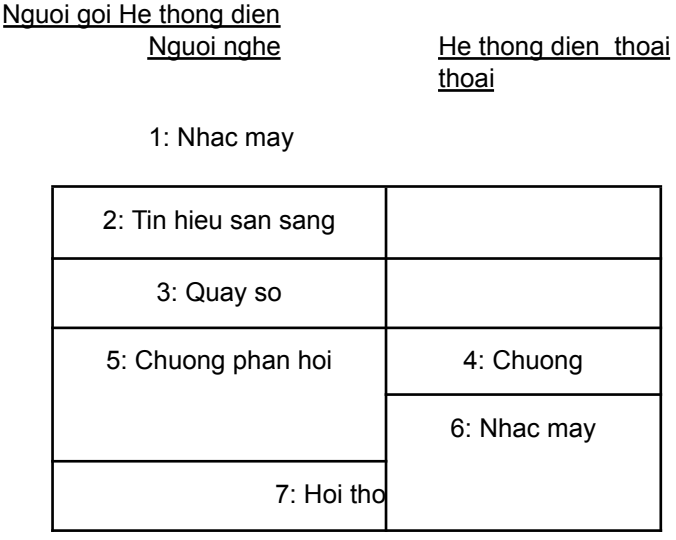


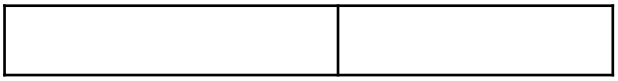


Hình 4.7 Biểu đồ trình tự ông A rút 100.000đ từ ATM

Hình 4.7 là thí dụ biểu đồ trình tự mô tả ông A rút 100.000đ từ máy rút tiền tự động ATM. Biểu đồ này có 5 đối tượng: Ông A (khách hàng), máy đọc thẻ tín dụng, màn hình ATM, tài khoản của ông A, máy chi trả. Đối tượng tác nhân (ông A) khởi động UC được vẽ tại góc bên trái của biểu đồ. Tiến trình rút tiền được bắt đầu khi ông A đặt thẻ tín dụng vào máy đọc thẻ. máy đọc thẻ đọc số hiệu thẻ và yêu cầu màn hình ATM tự khởi động. Màn hình ATM nhắc ông A nhập số căn cước cá nhân PIN, thí dụ nhập số 1234. ATM mở tài khoản của ông A. Số PIN của ông A được kiểm tra và màn hình ATM nhắc ông A thực hiện giao dịch. Ông A chọn *Rút tiền*. ATM nhắc ông A nhập số tiền muốn rút ra. Ông A nhập 100.000đ ATM kiểm tra xem số dư tài khoản của ông A có đủ số tiền muốn rút hay không. Rồi trừ số tiền trong tài khoản đi 100.000đ. ATM trả cho ông A 100.000đ và đẩy thẻ tín dụng ra khỏi máy đọc. Chú ý rằng trên biểu đồ này hình chữ nhật con dọc theo chu kỳ sống của đối tượng không được vẽ là để dễ quan sát biểu đồ. Các thông điệp số 2, 13 và 14 là các thông điệp phản thân.

Thí dụ về biểu đồ trình tự của hệ thống điện thoại được mô tả trên hình 4.8. biểu đồ này có 3 đối tượng: người gọi, hệ thống điện thoại và người nghe. Tiến trình gọi điện thoại bắt đầu khi người gọi *Nhấc máy gọi*. Hệ thống điện thoại gửi lại *Tín hiệu được phép gọi*. Người gọi *quay số*. Hệ thống điện thoại làm *Reo chuông* máy người bị gọi và *Gửi tín hiệu chuông trở lại người gọi*. Người bị gọi *Nhắc máy* để nghe. Người bị gọi *Hội thoại* với người gọi.



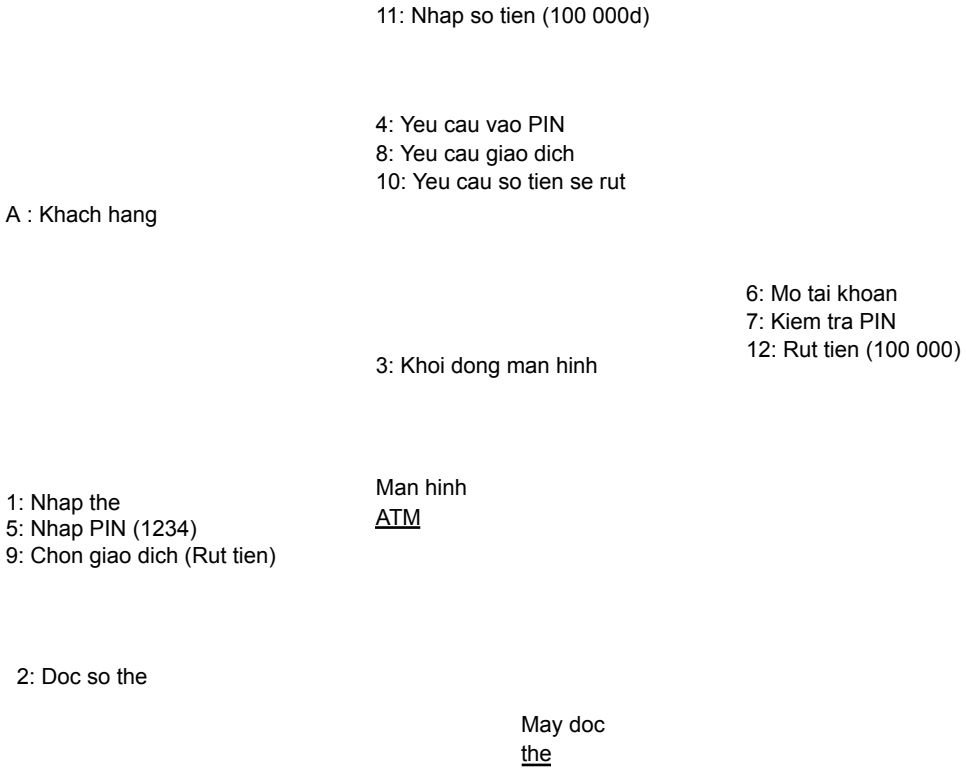


Hình 4.8 Biểu đồ trình tự gọi điện thoại

4.2.2 - Biểu đồ cộng tác

Tương tự như biểu đồ trình tự, biểu đồ cộng tác (*collaboration diagram*) chỉ ra luồng sự kiện xuyên qua kịch bản của UC. Trong khi biểu đồ trình tự có trật tự theo thời gian, thì biểu đồ cộng tác tập trung nhiều hơn vào quan hệ giữa các đối tượng, tập trung vào tổ chức cấu trúc của các đối tượng gửi hay nhận thông điệp.

Thí dụ trên hình 4.9 là biểu đồ cộng tác thể hiện luồng sự kiện ông A rút 100.000đ của hệ thống ATM. Hình này cho thấy các thông tin trên biểu đồ trình tự cũng có trên biểu đồ cộng tác, nhưng nó cho ta cách nhìn khác về luồng sự kiện. Biểu đồ này cho khả năng dễ quan sát quan hệ giữa các đối tượng nhưng khó quan sát hơn về trình tự thông tin. Do vậy, thông thường phải tạo cả hai biểu đồ cho cùng kịch bản.



17: Tra the tin dung ong A

May tra tien
13: Kiem tra so tien (100 000) 14: Tru trong tai khoan (100 000)

15: Tra tien (100 000)
16: Tra bien nhan

Tai khoan

Hình 4.9 Biểu đồ cộng tác ông A rút 100.000đ

Một sự khác nhau giữa biểu đồ cộng tác và biểu đồ trình tự đã đề cập trên là biểu đồ trình tự tập trung vào luồng điều khiển còn biểu đồ cộng tác thì không. Sự khác nhau giữa chúng là biểu đồ cộng tác hiển thị luồng dữ liệu (*Data flows*) còn biểu đồ trình tự thì không. Luồng dữ liệu được sử dụng để chỉ ra thông tin trở về khi đối tượng gửi thông điệp đến đối tượng khác (hình 4.10). Tổng quát thì không nên gán luồng dữ liệu cho mọi thông điệp trong biểu đồ cộng tác ví nó sẽ làm rối biểu đồ. Khi thông điệp cho lại cấu trúc dữ liệu thì nên gắn luồng dữ liệu. Khi ánh xạ thông điệp thành thao tác lớp, thông tin trong luồng dữ liệu sẽ được gắn vào thao tác. Tổng quát thì không nên quá tập trung vào luồng dữ liệu. Bổ sung chúng vào biểu đồ khi cho nó là có ý nghĩa thực sự cho việc phát triển hệ thống.

1: Thong diep

Du lieu A

Du lieu B 2

Doi tuong 1

Doi tuong

Hình 4.10 Luồng dữ liệu trong biểu đồ cộng tác

Biểu đồ trình tự và biểu đồ cộng tác chỉ phù hợp cho việc mô tả từng biến thể của thủ tục. Nó không phù hợp cho việc xác định đầy đủ các hành vi trên một biểu đồ. Các biểu đồ hoạt động (*activity diagram*) và biểu đồ trạng thái (*state diagram*) sẽ được đề cập trong các chương sau.

:Khach hang Doi tuong Mau

Doi tuong Mau

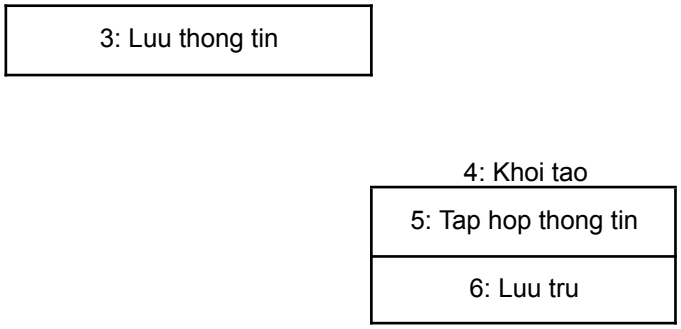
nhap lieu Doi tuong du lieu

Doi tuong du lieu

1: Mo mau nhap

nhap lieu

2: Nhap thong tin

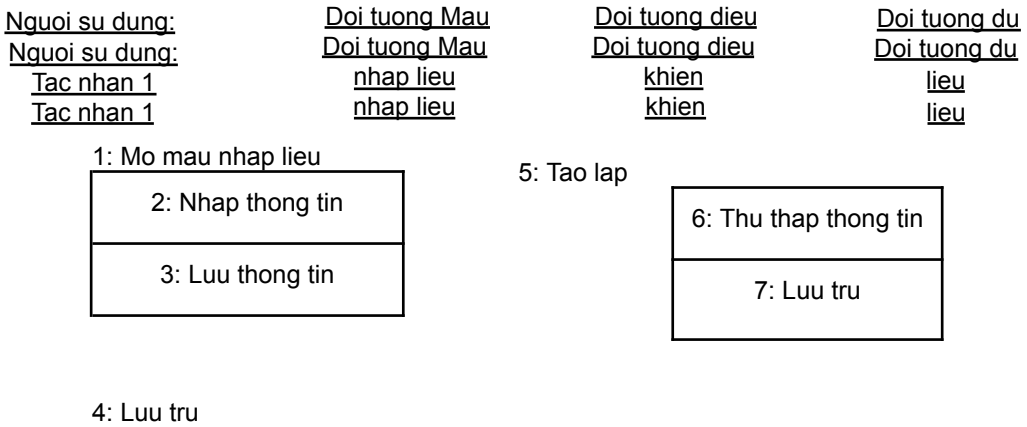


Hình 4.11 Biểu đồ trình tự bước thứ nhất

4.3 KỸ THUẬT XÂY DỰNG BIỂU ĐỒ TƯƠNG TÁC

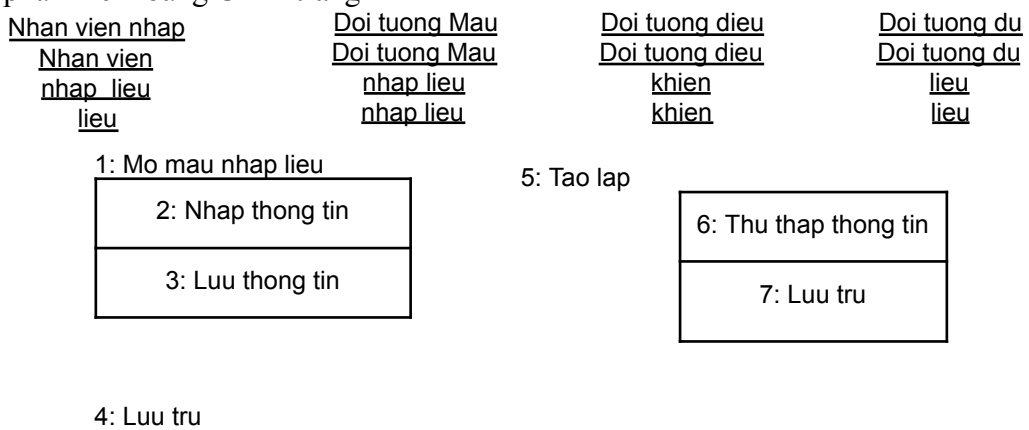
Có nhiều phương pháp xây dựng biểu đồ tương tác, nhưng tiệm cận hai bước là hay được sử dụng hơn cả để lập ra biểu đồ tương tác đầy đủ cho hệ thống liên quan đến CSDL. Bước thứ nhất chỉ tập trung vào các thông tin liên quan đến khách hàng. Không ánh xạ ngay thông điệp thành thao tác và không ánh xạ ngay đối tượng thành lớp. Biểu đồ tạo ra trong bước này dành cho nhà phân tích, khách hàng và những ai chỉ quan tâm đến luồng tác nghiệp, để biết luồng logic chạy trong hệ thống như thế nào. Biểu đồ trình tự điển hình sau bước thứ nhất được thể hiện trên hình 4.11.

Trong bước hai, khi khách hàng đã đồng ý với biểu đồ tạo ra trong bước một thì nó sẽ được bổ sung chi tiết hơn. Vào thời điểm này biểu đồ sẽ ít tác dụng cho khách hàng nhưng lại rất có ích với người phát triển hệ thống, kiểm tra chất lượng và các nhân viên khác của đội ngũ dự án. Một vài đối tượng mới có thể được thêm vào biểu đồ. Thông thường, mỗi biểu đồ tương tác đều có đối tượng điều khiển để điều khiển trình tự xuyên qua kịch bản. mọi biểu đồ tương tác của một uc đều chia sẻ cùng đối tượng điều khiển. sau khi bổ sung đối tượng điều khiển biểu đồ trình tự sẽ như trên hình 4.12.



Hình 4.12 biểu đồ trình tự với đối tượng điều khiển

Đối tượng điều khiển không xử lý tác nghiệp nào, chúng chỉ gửi thông điệp đến đối tượng khác. Đối tượng điều khiển có trách nhiệm điều phối công việc của đối tượng khác và được coi là đối tượng quản trị. Lợi thế của sử dụng đối tượng điều khiển là khả năng tách logic tác nghiệp ra khỏi logic trình tự. Nếu trình tự thay đổi thì chỉ đối tượng điều khiển thay đổi. Ta có thể bổ sung thêm đối tượng quản lý an toàn, quản lý hay kết nối CSDL. Rất nhiều đối tượng chỉ xây dựng một lần và có thể sử dụng lại trong các ứng dụng khác nhau. Sau đây là thí dụ về ứng dụng liên quan đến CSDL.



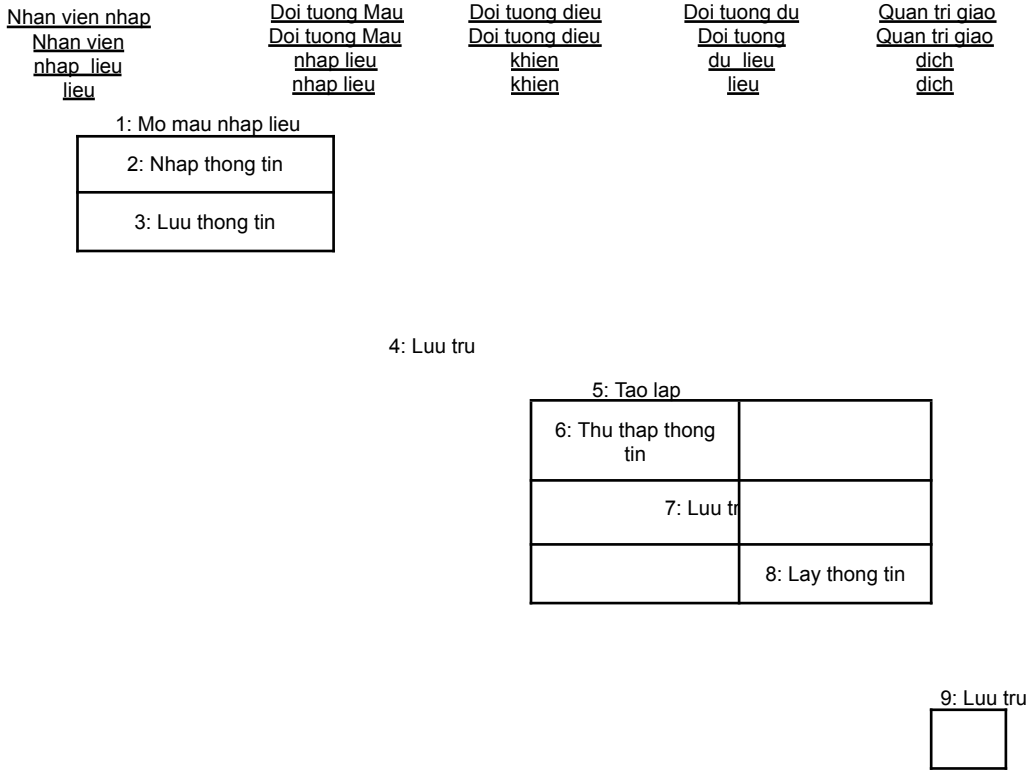
Hình 4.13 Logic ứng dụng tích hợp với logic CSDL

Có hai chức năng thường được sử dụng trong CSDL, đó là lưu trữ và khai thác thông tin. Giả sử rằng phải lưu trữ nhân viên mới có tên *Ông A* vào CSDL. Nếu đối tượng *Ông A* biết về CSDL thì nó tự lưu trữ vào CSDL. Cách khác là tách *Ông A* ra khỏi logic CSDL để đối tượng khác có trách nhiệm lưu trữ *Ông A* vào CSDL. Hình 4.13 là biểu đồ trình tự thể hiện trường hợp *Ông A* biết về CSDL. Logic ứng dụng và logic CSDL trong thí dụ này không tách rời nhau. Đối tượng *Ông A* vừa phải quan tâm đến logic ứng dụng và logic CSDL (lưu trữ *Ông A* vào CSDL và truy cập *Ông A* từ CSDL). Lợi thế của tiệm cận này là dễ mô hình hóa và cài đặt. tuy nhiên khi CSDL thay đổi nó sẽ kéo theo sự thay đổi tại rất nhiều nơi trong ứng dụng vì rất nhiều đối tượng chứa cả logic CSDL. Trong tiệm cận tách logic ứng dụng ra khỏi logic CSDL ta phải tạo ra đối tượng mới có trách nhiệm với CSDL. Các đối tượng này được gọi là đối tượng *Quản trị phiên giao dịch*

(*Transaction Manager*). Tại đây, trong khi đối tượng *Ông A* vẫn chứa logic tác nghiệp thì đối tượng *Quản trị phiên giao dịch* sẽ biết cách truy vấn *Ông A* từ CSDL và lưu trữ *Ông A* vào CSDL. Biểu đồ trình tự của cách tiếp cận này được trình bày trên hình 4.14.

Lợi thế của giải pháp này là dễ dàng sử dụng lại đối tượng *Ông A* cho ứng dụng với các CSDL khác nhau hay không có CSDL. Nó cho khả năng giảm thiểu các thay đổi, khi CSDL thay đổi sẽ không làm thay đổi logic ứng dụng, và ngược lại. Bất lợi của tiêm cận này là ta phải thêm thời gian để mô hình hóa và cài đặt hệ thống. công việc tiếp theo phía nhiệm vụ CSDL là bổ sung thêm các đối tượng quản lý lỗi, an toàn dữ liệu, giao tiếp giữa các tiến trình… Các chi tiết này không quan trọng với người sử dụng nhưng lại rất quan trọng với người phát triển hệ thống.

Sau khi bổ sung đầy đủ các đối tượng, bước tiếp theo là ánh xạ từng đối tượng vào lớp có sẵn hay lớp mới, ánh xạ từng thông điệp trong biểu đồ thành thao tác. Cuối cùng là thực hiện đặc tả đối tượng và đặc tả thông điệp như gán đồng bộ, gán tần xuất thông điệp và cách thức lưu trữ đối tượng…



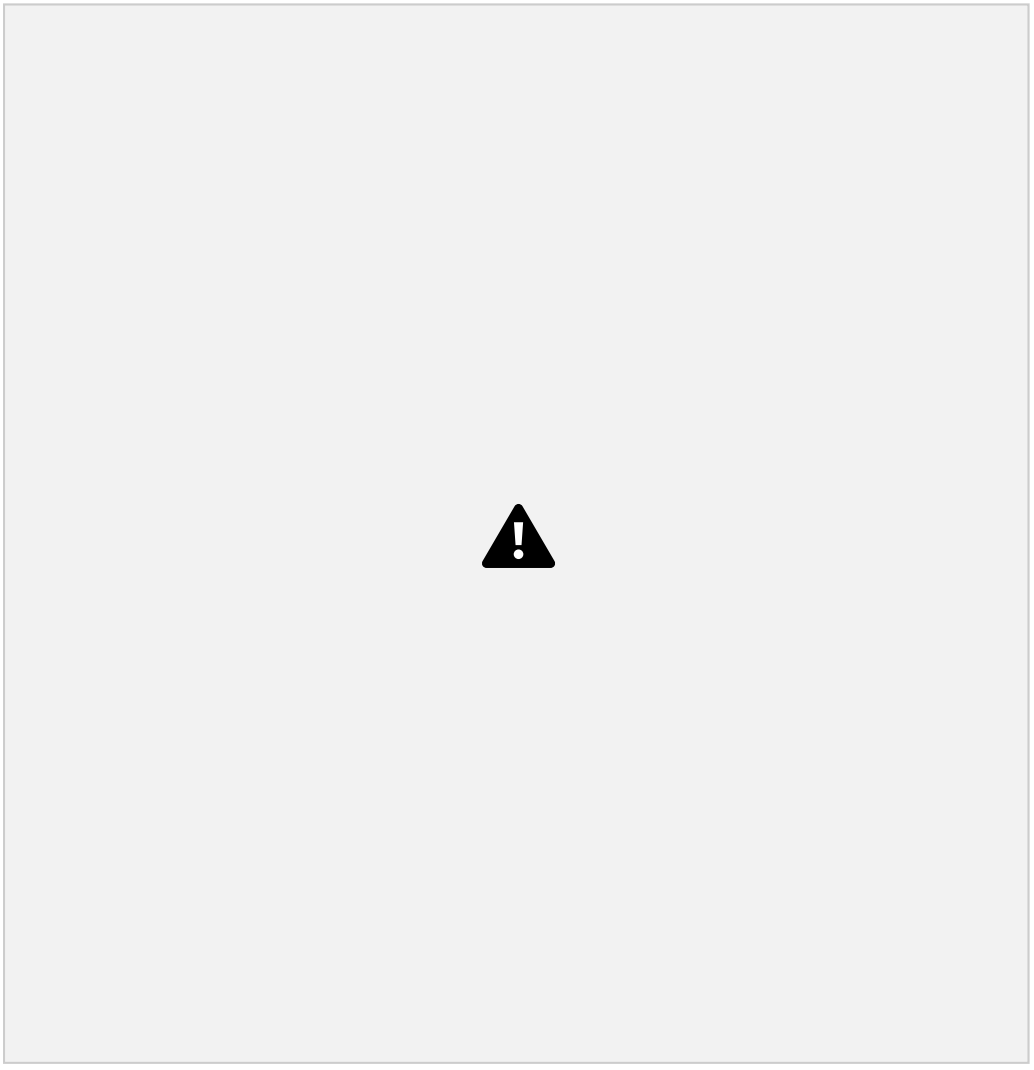
Hình 4.14 Logic ứng dụng tách khỏi logic CSDL

4.4 THỰC HÀNH

4.4.1 - Sử dụng Rational Rose

4.4.1.1 - Tạo lập biểu đồ trình tự

Các biểu đồ trình tự được tạo lập trong khung nhìn UC (*Use Case View*) hay khung nhìn logic (*Logical view*) của browser. Biểu đồ trình tự phải được vẽ trực tiếp trong UC như trên hình 4.15 hay trực tiếp trong gói.



Hình 4.15 Tạo lập biểu đồ trình tự mới

Tạo biểu đồ trình tự mới như sau:

1. Nhấn phím phải chuột trên gói hay UC trong browser
2. Chọn *New> Sequence diagram*
3. Đặt tên cho biểu đồ trình tự mới
4. Nhấn đúp trên biểu đồ trình tự mới trong browser để mở.

Mơ biểu đồ trình tự có sẵn:

1. Tìm biểu đồ trình tự trong khung nhìn UC của browser
2. Nhấn đúp chuột vào biểu đồ trình tự để mở nó.

Sử dụng các phím trên thanh công cụ để bổ sung các đối tượng, thông điệp vào biểu đồ hoặc di tác nhân, lớp từ browser vào biểu đồ trình tự. Chú ý rằng khi bỏ đối tượng khỏi biểu đồ. Lớp trong mô hình không bị xóa.

4.4.1.2 - Bãi bỏ biểu đồ trình tự

Trong quá trình làm việc có thể một số biểu đồ trình tự là quá cũ, không còn chính xác hay dư thừa. Để cho mô hình nhất quán, ta có thể hủy bỏ các biểu đồ trình tự đó đi. Trong Rose, việc hủy bỏ biểu đồ trình tự được thực hiện như sau:

1. Nhấn phím phải chuột trên biểu đồ trình tự trong browser
2. Chọn delete từ thực đơn

4.4.1.3 - Gắn tệp vào biểu đồ trình tự

Trong Rose, ta có thể gán tệp vào biểu đồ trình tự. Nội dung tệp có thể là mô tả kịch bản mà biểu đồ trình tự mô hình hóa, hoặc một vài dòng mã lệnh cài đặt logic trong biểu đồ hay mô tả các nhu cầu liên quan. Gắn tệp vào biểu đồ trình tự theo các bước sau đây:

1. Nhấn phím phải chuột trên biểu đồ trình tự trong browser
2. Chọn *New>File*
3. Sử dụng hộp thoại *Open* để chọn tên tệp sẽ gắn vào biểu đồ
4. Chọn *Open*

Bãi bỏ tệp gắn vào biểu đồ trình tự như sau:

1. Nhấn phím phải trên tệp trong browser
2. Chọn *Delete* từ thực đơn

4.4.1.4 - Tạo lập và hủy bỏ biểu đồ cộng tác

Tương tự biểu đồ trình tự, biểu đồ cộng tác được tạo lập trong browser, trực tiếp trong UC hay trong gói. Cách tạo lập khác là chuyển đổi tự động biểu đồ trình tự có sẵn sang biểu đồ cộng

tác nhờ **nhấn phím F5**. Trình tự hủy bỏ biểu đồ cộng tác được thực hiện tương tự như hủy bỏ biểu đồ trình tự vừa mô tả trên đây.

4.4.1.5 - Bổ sung và hủy bỏ tác nhân trong biểu đồ tương tác

Mỗi biểu đồ trình tự và biểu đồ cộng tác đều có một tác nhân. Đối tượng tác nhân lá kích thích ngoài để hệ thống thực hiện một vài chức năng. Đối tượng tác nhân của biểu đồ tương tác bao gồm các tác nhân thao tác với UC trong biểu đồ UC

Tạo lập đối tượng tác nhân trên biểu đồ tương tác như sau:

1. Mở biểu đồ tương tác
2. Chọn tác nhân trong browser
3. Di tác nhân từ Browser đến biểu đồ đang mở

Hủy bỏ đối tượng tác nhân trên biểu đồ thao tác theo trình tự sau:

1. Chọn tác nhân trên biểu đồ tương tác
2. Chọn *Edit>Delete from Model*, hay nhấn phím *Ctrl+D*

Chú ý rằng việc hủy bỏ đối tượng khỏi biểu đồ sẽ không hủy bỏ lớp trong mô hình.

4.4.1.6 - Bổ sung đối tượng vào biểu đồ tương tác

Công việc tiếp sau khi biểu đồ tương tác được tạo lập là bổ sung đối tượng khác vào biểu đồ. Ta phải tìm các đối tượng tham gia vào biểu đồ trình tự hay biểu đồ cộng tác cụ thể bằng cách tìm các danh từ trong tài liệu về luồng sự kiện và kịch bản. bổ sung đối tượng vào biểu đồ trình tự theo các bước sau đây:

1. Chọn phím *Object* trên thanh công cụ
2. Nhấn phím trái chuột trong vùng biểu đồ nơi sẽ đăt đối tượng. Trong biểu đồ trình tự, các đối tượng đặt tại hàng đỉnh
3. Nhập tên cho đối tượng mới
4. Sau khi vẽ các đối tượng trong biểu đồ, ta có thể sắp xếp lại chúng bằng nhấn-di-nhả đối tượng, bổ sung đối tượng vào giữa các đối tượng đã vẽ trước đó.

Trong biểu đồ cộng tác, bổ sung đối tượng theo trình tự sau:

1. Chọn phím *Object* trên thanh công cụ
2. Nhấn phím chuột trong vùng biểu đồ nơi sẽ đặt đối tượng. Trong biểu đồ cộng tác, các đối tượng đặt tại bất kỳ đâu.
3. Nhập tên đối tượng mới

4.4.1.7 - Bãi bỏ đối tượng trong biểu đồ tương tác

Biểu đồ tương tác được huỷ bỏ theo trình tự sau:

1. Chọn đối tượng
2. Chọn *Edit > Delete from Model*, hay *Ctrl+D*

4.4.1.8 - Đặc tả đối tượng

Cửa sổ đặc tả đối tượng trong Rose cho nhập các thông tin sau: tên đối tượng, lớp của đối tượng, lưu trữ, đa bản, tài liệu. Mở cửa sổ đặc tả đối tượng như sau:

1. Nhấn phím phải chuột vào biểu đồ trình tự và cộng tác
2. Chọn thực đơn *Open Specification*

4.4.1.9 - Đặt tên đối tượng

Mỗi đối tượng trên biểu đồ phải có tên duy nhất. Trong khi tên lớp là rất khái quát như *Nhân viên*, *Công ty* thì tên đối tượng lại rất cụ thể như *Ông A*, *Microsoft*…

Đặt tên đối tượng như sau

1. Nhấn phím phải chuột trên đối tượng trong biểu đồ
2. Chọn *Open Specification*
3. Nhập tên vào cửa sổ *Name*. Mỗi đối tượng trên biểu đồ có tên duy nhất. Thực hiện tương tự cho việc nhập tài liệu của đối tượng.

4.4.1.10 - Ánh xạ đối tượng vào lớp

Mỗi đối tượng trong biểu đồ trình tự và biểu đồ hợp tác có thể ánh xạ vào một lớp. Thí dụ đối tượng tài khoản *Ông A* ánh xạ vào lớp *Tài khoản*. Việc này được thực hiện trong cửa sổ đặc tả đối tượng (*Object Specification*). Khi chọn lớp cho đối tượng, ta có thể chọn lớp có sẵn hay lập lớp mới.

Ánh xạ đối tượng vào lớp có sẵn như sau:

1. Nhấn phím phải chuột trên đối tượng trong biểu đồ tương tác
2. Chọn *Open Specification*
3. Nhập tên lớp hay chọn tên lớp trong hộp *Class*
4. Khi đã ánh xạ đối tượng vào lớp, tên lớp sẽ xuất hiện cùng tên đối tượng trên biểu đồ.

Bãi bỏ ánh xạ lớp cho đối tượng theo trình tự sau đây:

1. Nhấn phím phải chuột trên đối tượng trong biểu đồ tương tác
2. Chọn thực đơn *Open Specification*
3. Chọn (*Unspecified*) trong hộp danh sách *Class*

Tạo lập lớp mới cho đối tượng:

1. Nhấn phím phải chuột trên đối tượng trong biểu đồ tương tác
2. Chọn thực đơn *Open Specification*
3. Chọn <*New*> trong hộp danh sách *Class*. Nhập thông tin cho lớp mới trong cửa sổ đặc tả.

Chỉ hiển thị tên lớp trên biểu đồ:

1. Nhấn phím phải chuột trên đối tượng trong biểu đồ tương tác
2. chọn thực đơn *Open Specification*
3. nhập tên đối tượng vào cửa sổ *Name*
4. chọn (*Unspecified*) trong hộp danh sách *Class*

Chỉ hiển thị tên lớp trên biểu đồ:

1. Nhấn phím phải chuột trên đối tượng trong biểu đồ tương tác
2. Chọn thực đơn *Open Specification*
3. Xóa tên đối tượng trong cửa sổ *Name*

4.4.1.11 - Thiết lập lưu trữ đối tượng

Rose cho khả năng thiết lập thuộc tính lưu trữ cho mỗi đối tượng trong biểu đồ. Các thuộc tính đó là:

Lưu trữ (Persistent): đối tượng lưu trữ là đối tượng sẽ được lưu trữ trong CSDL, hay lưu trữ dưới khuôn dạng khác.

Tĩnh (Static): là đối tượng tồn tại trong bộ nhớ cho đến khi chương trình kết thúc *Tạm thời (Transient)*: là đối tượng chỉ tồn tại trong bộ nhớ trong khoảng thời gian ngắn

Thiết lập thuộc tính lưu trữ đối tượng như sau:

1. Nhấn phím phải chuột trên đối tượng trong biểu đồ tương tác
2. Chọn thực đơn *Open Specification*
3. Chọn phím radio tương ứng trong vùng *Persistance*

4.4.1.12 - Sử dụng nhiều bản đối tượng

Rational Rose có khả năng sử dụng một biểu tượng để biểu diễn nhiều hiện thực của một lớp. Thí dụ, để biểu diễn danh sách nhân viên trong biểu đồ tương tác ta có thể sử dụng biểu tượng đa hiện thực (*multiple instances*). Ký pháp của UML như sau:

Nhan
vien

Gán thuộc tính đa hiện thực cho đối tượng theo các bước sau:

1. Nhấn phím phải chuột trên đối tượng trong biểu đồ tương tác
2. Chọn thực đơn *Open Specification*
3. Đánh dấu *on* hay *off* trong hộp *Multiple Instances*. Rose sẽ sử dụng bên trong phù hợp (đơn hay đa hiện thực) trong biểu đồ cộng tác, biểu tượng đơn hiện thực trong biểu đồ trình tự

4.4.1.13 - Bổ sung thông điệp vào biểu đồ trình tự

Trong biểu đồ trình tự, thông điệp được vẽ giữa các đường *lifelines* của đối tượng hay đường *lifelines* của chính đối tượng. thông điệp sẽ hiển thị từ trên xuống đáy biểu đồ

Trình tự bổ sung thông điệp như sau:

1. Chọn phím *Object Message* trên thanh công cụ.
2. Di chuột từ *lifeline* của đối tượng hay tác nhân gửi thông điệp đến đối tượng hay tác nhân nhận thông điệp
3. Nhập tên thông điệp

4.4.1.14 - Thứ tự thông điệp trong biểu đồ trình tự

Thực hiện sắp xếp lại bằng nhấn, di, thả. Thứ tự thông điệp được tự động sắp

xếp ***4.4.1.15 - Đánh số thông điệp trong biểu đồ trình tự***

Mặc định, số thứ tự thông điệp được gán trên biểu đồ tương tác và là ẩn. Để hiển thị / ẩn thứ tự thông điệp ta làm như sau:

1. Chọn thực đơn Tool > Options
2. Chọn DiagramTab
3. Đặt thuộc tính on / off cho Sequence Numbering